

TUGAS AKHIR - ES234849

ANALISIS KINERJA ARSITEKTUR INTEGRASI DATA ETL, ELT, DAN HIBRIDA PADA *DATA WAREHOUSE* BERBASIS *POSTGRESQL*

LUTHFAN ARYANANDA PURWITO

NRP 5026221166

Dosen Pembimbing

Faizal Johan Atletiko, S.Kom., MT.

NIP 197704082002121001

Program Studi S1 Sistem Informasi

Departemen Sistem Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



TUGAS AKHIR - ES234849

ANALISIS KINERJA ARSITEKTUR INTEGRASI DATA ETL, ELT, DAN HIBRIDA PADA *DATA WAREHOUSE* BERBASIS *POSTGRESQL*

LUTHFAN ARYANANDA PURWITO

NRP 5026221166

Dosen Pembimbing

Faizal Johan Atletiko, S.Kom., MT.

NIP 197704082002121001

Program Studi S1 Sistem Informasi

Departemen Sistem Informasi

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

2026



FINAL PROJECT - QQXXXXXX

THE TITLE OF THE FINAL PROJECT IS WRITTEN BRIEFLY, CLEARLY AND DESCRIBING THE MAIN THEME

STUDENT NAME

NRP XXXXXXXXXXXXX

Advisor

Advisor Name and Academic Title

NIP XXXXXXXXXXXXX

Study Program <Name of Study Program>

Department of <Name of Department>

Faculty of <Name of Faculty>

Institut Teknologi Sepuluh Nopember

Surabaya

Year

LEMBAR PENGESAHAN

ANALISIS KINERJA ARSITEKTUR INTEGRASI DATA ETL, ELT, DAN HIBRIDA PADA DATA WAREHOUSE BERBASIS POSTGRESQL

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat
memperoleh gelar Sarjana pada
Program Studi S-1 Sistem Informasi
Departemen Sistem Informasi
Fakultas Teknologi Elektro dan Informatika Cerdas
Institut Teknologi Sepuluh Nopember

Oleh : **LUTHFAN ARYANANDA PURWITO**

NRP. 5026221166

Disetujui oleh Tim Penguji Tugas Akhir :

- | | |
|---|------------|
| 1. Faizal Johan Atletiko, S.Kom., MT. | Pembimbing |
| 2. Radityo Prasetyanto Wibowo, S.Kom., M.Kom. | Penguji |
| 3. Irmasari Hafidz, S.Kom., M.Sc. | Penguji |

SURABAYA

Bulan, Tahun

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP :

Program studi :

Dosen Pembimbing / NIP :

dengan ini menyatakan bahwa Tugas Akhir dengan judul “_____”
adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, _____

Mengetahui
Dosen Pembimbing

Mahasiswa

Nama dosen pembimbing
NIP.

Nama mahasiswa
NRP.

ABSTRAK

ANALISIS KINERJA ARSITEKTUR INTEGRASI DATA ETL, ELT, DAN HIBRIDA PADA DATA WAREHOUSE BERBASIS POSTGRESQL

Nama Mahasiswa / NRP : Luthfan Aryananda Purwito / 5026221166
Departemen : Sistem Informasi FTEIC - ITS
Dosen Pembimbing : Faizal Johan Atletiko, S.Kom., MT.

Abstrak

Pertumbuhan volume dan varietas data menimbulkan tantangan kompleks bagi organisasi dan perusahaan di berbagai sektor. sehingga menuntut mereka untuk mengembangkan strategi data yang efisien. Pendekatan *Extract, Transform, Load* (ETL) telah lama menjadi standar yang diandalkan dalam transformasi data di dunia *enterprise* dan bisnis, namun masih sering menghadapi banyak kendala dan keterbatasan dari berbagai aspek, seperti skalabilitas dan kinerja saat menangani data bervolume besar. Sebagai alternatif, arsitektur *Extract, Load, Transform* (ELT) muncul untuk memanfaatkan kekuatan dari mesin basis data target. Beberapa perusahaan—terutama industri dengan regulasi yang ketat seperti keuangan dan kesehatan—menerapkan pendekatan hibrida untuk menggabungkan kekuatan kedua arsitektur, yaitu tata kelola data dari ETL, serta kecepatan dan skalabilitas dari ELT. Namun, studi komparatif antara ketiga arsitektur ini masih terbatas, terutama penerapannya di lingkungan lokal (*on-premise*) dan pengujian dengan jenis data yang bervariasi. Penelitian ini bertujuan untuk mengevaluasi dan membandingkan kinerja arsitektur ETL, ELT, dan pendekatan hibrida dalam memproses data terstruktur (CSV, Parquet) dan semi-terstruktur (JSON) pada *data warehouse* berbasis PostgreSQL. Penelitian dilakukan untuk mengevaluasi dan membandingkan kinerja arsitektur ETL, ELT, dan pendekatan hibrida dalam menangani *dataset benchmark* TPC-H. Arsitektur ETL disimulasikan menggunakan Python (*PySpark*), sedangkan ELT dan hibrida menggunakan Python dan *dbt* (*data build tool*). Kinerja arsitektur tersebut dievaluasi berdasarkan metrik waktu pemrosesan total dan efisiensi penggunaan sumber daya (CPU dan memori). Hasil penelitian tugas akhir ini diharapkan dapat menjadi referensi yang komprehensif dan berguna untuk penelitian atau pengembangan sistem serupa di masa depan.

Kata kunci: *Integrasi Data, ETL, ELT, Hibrida, PostgreSQL, dbt, Python, TPC-H*

ABSTRACT

A COMPARATIVE ANALYSIS OF ETL, ELT, AND HYBRID DATA INTEGRATION ARCHITECTURE PERFORMANCE IN POSTGRESQL-BASED DATA WAREHOUSES

Student Name / NRP : Luthfan Aryananda Purwito / 5026221166
Department : Sistem Informasi FTEIC - ITS
Advisor : Faizal Johan Atletiko, S.Kom., MT.

Abstract

The rapid growth of data volume and variety poses complex challenges for organizations across various sectors, necessitating the development of efficient data strategies. Historically, the Extract, Transform, Load (ETL) approach has served as the industry standard for enterprise data integration; however, it frequently encounters significant limitations regarding scalability and performance when processing large-scale data volumes. As an alternative, the Extract, Load, Transform (ELT) architecture has emerged to leverage the computational power of the target database engine. Furthermore, certain organizations—particularly in highly regulated industries such as finance and healthcare—adopt a hybrid approach to combine the strengths of both architectures: the robust data governance of ETL and the speed and scalability of ELT. However, comparative studies across these three architectures remain limited, particularly regarding their application in on-premise environments and performance testing with diverse data types. This study aims to evaluate and compare the performance of ETL, ELT, and hybrid architectures in processing structured (CSV, Parquet) and semi-structured (JSON) data within a PostgreSQL-based data warehouse environment. The research utilizes the TPC-H benchmark dataset for evaluation. The ETL architecture is simulated using Python (PySpark), while the ELT and hybrid architectures are implemented using Python and dbt (data build tool). Performance is evaluated based on total processing latency metrics, resource usage efficiency (CPU and memory). The findings of this study are expected to serve as a comprehensive reference for future research and the development of similar data integration systems.

Keywords: *Data Integration, ETL, ELT, Hybrid, PostgreSQL, dbt, Python, TPC-H*

DAFTAR ISI

LEMBAR PENGESAHAN	i
ABSTRAK	iii
ABSTRACT	iv
DAFTAR ISI	v
DAFTAR GAMBAR	vii
DAFTAR TABEL	viii
DAFTAR SIMBOL	ix
BAB 1 PENDAHULUAN	10
1.1 Latar Belakang	10
1.2 Rumusan Masalah	12
1.3 Batasan Masalah	12
1.4 Tujuan	13
1.5 Manfaat	13
BAB 2 TINJAUAN PUSTAKA	14
2.1 Hasil Penelitian Terdahulu	14
2.2 Dasar Teori	18
2.2.1 Karakteristik dan Tantangan pada Big Data	18
2.2.2 Data Warehouse	19
2.2.3 Data Lake	19
2.2.4 Integrasi Data	20
2.2.5 Extract, Transform, Load (ETL)	21
2.2.6 Extract, Load, Transform (ELT)	22
2.2.7 Arsitektur Hibrida	23
2.2.8 Format Penyimpanan Data	25
2.2.9 Data Build Tool (dbt)	27
2.2.10 Python	28
2.2.11 PostgreSQL	29
2.2.12 TPC-H (TPC Benchmark™ H)	29
BAB 3 METODOLOGI	32
3.1 Uraian Metodologi	32
3.1.1 Identifikasi Masalah	33

3.1.2	Studi Literatur	33
3.1.3	Identifikasi Kebutuhan	33
3.1.4	Akuisisi Data	34
3.1.5	Perancangan Arsitektur ETL	35
3.1.6	Perancangan Arsitektur ELT	36
3.1.7	Perancangan Arsitektur Hibrida	37
3.1.8	Tahap Pengujian	37
3.1.9	Tahap Analisis Hasil	39
3.1.10	Penyusunan Tugas Akhir	39
3.2	Urutan Pelaksanaan Penelitian	40
DAFTAR PUSTAKA		74
LAMPIRAN		77

DAFTAR GAMBAR

Gambar 2.1 Klasifikasi tantangan konseptual dari big data (Sivarajah et al., 2017).....	19
Gambar 2.2 Extract, Transform, Load (ETL) (DataForge, 2023)	22
Gambar 2.3 Extract, Load, Transform (ELT) (DataForge, 2023)	23
Gambar 2.4 Model ECLT (Farhan et al., 2024).....	24
Gambar 2.5 Struktur File Parquet (Apache Parquet: Documentation, 2024)	27
Gambar 2.6 Arsitektur Penerapan dbt.....	28
Gambar 2.7 Skema TPC-H (TPC Benchmark H Standard Specification Book, 1993).....	30
Gambar 3.1 Diagram Alir Metodologi Tugas Akhir	32
Gambar 3.2 Proses Ekstraksi pada ETL	35
Gambar 3.3 Proses Transformasi pada ETL	36
Gambar 3.4 Proses Pemuatan pada ETL.....	36

DAFTAR TABEL

Tabel 2.1 Penelitian Terdahulu 1	14
Tabel 2.2 Penelitian Terdahulu 2	15
Tabel 2.3 Penelitian Terdahulu 3	15
Tabel 2.4 Penelitian Terdahulu 4	16
Tabel 2.5 Penelitian Terdahulu 5	17
Tabel 2.6 Penelitian Terdahulu 6	17
Tabel 2.7 Perkiraan Ukuran Basis Data TPC-H dengan SF = 1 (TPC Benchmark H Standard Specification Book, 1993)	31
Tabel 3.1 Spesifikasi Lingkungan Pengujian.....	34
Tabel 3.2 Perkiraan Ukuran Basis Data TPC-H dengan SF = 50	34
Tabel 3.3 Jadwal Pelaksanaan Penelitian Tugas Akhir	41

DAFTAR KODE

Kode 4.1 Konfigurasi Spark Job Group pada Arsitektur ETL.....	48
Kode 4.2 Implementasi Ekstraksi Data Multi-Format pada Arsitektur ETL.....	49
Kode 4.3 Implementasi Penanganan Kesalahan Ekstraksi File pada ETL	49
Kode 4.4 Inisiasi Entitas Tabel TPC-H pada Fase Transformasi	50
Kode 4.5 Implementasi Transformasi TPC-H Query 9 pada Arsitektur ETL	51
Kode 4.6 Implementasi Pemuatan Data ke PostgreSQL pada Arsitektur ETL	52
Kode 4.7 Implementasi Fungsi Utama Pipeline ETL	53
Kode 4.8 Konfigurasi Spark Job Group pada Arsitektur ELT.....	54
Kode 4.9 Implementasi Ekstraksi Data Multi-Format pada Arsitektur ELT.....	54
Kode 4.10 Implementasi Kesalahan Penanganan Ekstraksi File pada ELT	55
Kode 4.11 Implementasi Pemuatan Data Mentah pada Arsitektur ELT	55
Kode 4.12 Konfigurasi Koneksi PostgreSQL pada Arsitektur ELT.....	56
Kode 4.13 Struktur Direktori dbt pada Arsitektur ELT	57
Kode 4.14 Konfigurasi sources.yml pada Arsitektur ELT	58
Kode 4.15 Model Staging stg_elt_lineitem.sql pada Arsitektur ELT.....	59
Kode 4.16 Model Staging stg_elt_orders.sql pada Arsitektur ELT.....	59
Kode 4.17 Model Staging stg_elt_part.sql pada Arsitektur ELT.....	59
Kode 4.18 Model Staging stg_elt_partsupp.sql pada Arsitektur ELT	60
Kode 4.19 Model Staging stg_elt_supplier.sql pada Arsitektur ELT	60
Kode 4.20 Model Staging stg_elt_nation.sql pada Arsitektur ELT.....	60
Kode 4.21 Model Mart mart_elt_query9.sql pada Arsitektur ELT	61
Kode 4.22 Perintah Eksekusi Transformasi dbt pada Arsitektur ELT.....	62
Kode 4.23 Konfigurasi Spark Job Group pada Tahap Ekstraksi Arsitektur Hibrida.....	63
Kode 4.24 Implementasi Ekstraksi Data Multi-Format pada Arsitektur Hibrida.....	64
Kode 4.25 Konfigurasi Spark Job Group pada Tahap Pembersihan Arsitektur Hibrida ...	65
Kode 4.26 Implementasi Pembersihan Data pada Arsitektur Hibrida.....	66
Kode 4.27 Implementasi Pemuatan Data pada Arsitektur Hibrida.....	67
Kode 4.28 Struktur Direktori dbt pada Arsitektur Hibrida	69
Kode 4.29 Implementasi Model Staging stg_hybrid_lineitem.sql pada Arsitektur Hibrida	69
Kode 4.30 Implementasi Model Staging stg_hybrid_orders pada Arsitektur Hibrida	70

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Pertumbuhan teknologi informasi yang sangat cepat menyebabkan peningkatan signifikan dalam volume serta kecepatan produksi data. Menurut Reinsel et al. (2018), diproyeksikan bahwa jumlah data yang dihasilkan secara global akan mencapai 175 *zettabyte* pada tahun 2025, naik secara substansial dari 33 *zettabyte* pada tahun 2018. Data tersebut berasal dari beragam sumber digital seperti perangkat *Internet of Things* (IoT), aplikasi bisnis, platform media sosial, hingga sistem transaksi daring. Variasi format data—baik terstruktur, semi-terstruktur, maupun tidak terstruktur—menimbulkan tantangan kompleks bagi organisasi dan perusahaan di berbagai sektor. Studi literatur yang dilakukan oleh Mehmood & Anees (2020) menyebutkan bahwa perbedaan dan keragaman data (heterogenitas) merupakan salah satu faktor utama yang mempersulit proses pengolahan aliran data dalam konteks *big data*. Sampai saat ini, organisasi masih berupaya menemukan pendekatan yang paling efisien dan efektif untuk melakukan pemindahan serta transformasi data mereka. Urgensi ini meningkat seiring dengan pertumbuhan data yang sangat besar dan kebutuhan terhadap analisis secara *real-time*, di mana nilai suatu data sering kali bergantung pada tingkat kebaruan dan ketepatan waktunya. Studi tersebut juga menekankan bahwa transformasi data merupakan komponen penting dalam pemrosesan data, sehingga berbagai metode dan perangkat terus dikembangkan untuk meningkatkan efisiensinya.

Secara historis, pendekatan *Extract, Transform, Load* (ETL) telah lama digunakan sebagai standar industri untuk menjawab tantangan integrasi data. Konsep ini pertama kali dipopulerkan oleh Ralph Kimball pada tahun 1996 dan berperan penting dalam arsitektur *data warehouse* konvensional, di mana data dari berbagai sistem operasional diekstrak, dibersihkan, dan diubah dalam staging area khusus, sebelum dimuat ke dalam *data warehouse* untuk analisis (Kimball & Ross, 2013). Dengan metodologi *dimensional modelling* dan konsep *star schema*, ETL telah menjadi standar yang diandalkan dalam transformasi data di dunia *enterprise* dan bisnis, menawarkan performa yang optimal dalam analisis data terstruktur di lingkungan *data warehouse* berbasis *Relational Database Management System* (RDBMS).

Penelitian terkait ETL di ranah akademik dan industri telah meluas, tetapi mayoritas fokusnya masih terbatas pada data terstruktur dengan volume yang relatif kecil; masih sedikit yang membahas performa ETL pada data skala besar dan tidak/semi-terstruktur. Dokumentasi model ETL yang diajukan juga sering tidak mencukupi atau bahkan tidak tersedia (Dhaouadi et al., 2022). Selain itu, pergeseran ke era *big data* membawa tantangan baru terkait dengan karakteristik *big data* yang dikenal sebagai “5V”: *Volume* (jumlah data yang dihasilkan sangat besar), *Variety* (keberagaman tipe data yang bisa berupa data terstruktur, semi-terstruktur, dan tidak terstruktur), *Velocity* (kecepatan waktu memproses data), *Value* (nilai yang dapat diberikan oleh data yang dikumpulkan), serta *Veracity* (kualitas data, termasuk akurasi atau tingkat kebenaran data) (Younas, 2019). Karakteristik 5V tersebut tidak bisa ditangani secara optimal oleh pendekatan ETL tradisional. Salah satu masalah besar yang muncul adalah performa waktu eksekusi yang menurun drastis saat *data warehouse* harus menangani data yang berukuran sangat besar, apalagi jika data tersebut tidak terstruktur. Secara spesifik, *bottleneck* tersebut seringkali berpusat di tahap Transformasi (T) itu sendiri. Ketika berhadapan dengan data bervolume besar atau analitis yang kompleks, pengembang biasanya menulis sebuah *User-Defined Functions* (UDF) atau kode custom yang ditulis di luar *data warehouse*. Penelitian oleh

Ali et al. (2019) menunjukkan bahwa UDF sering dianggap sebagai aktivitas “*black-box*” oleh alat ETL, sehingga tidak dioptimalkan maupun diparalelkan. Hal tersebut menjadi *bottleneck* dalam alur kerja ETL. Sivarajah et al. (2017) mengidentifikasi bahwa organisasi mengalami tantangan yang cukup signifikan dalam hal kompleksitas integrasi data (*data integration complexities*) dan keterbatasan infrastruktur seperti *data warehouse* yang tidak memadai untuk mengelola data yang masif. Kondisi tersebut menyebabkan penurunan performa analitis secara keseluruhan. Farhan et al. (2024) mengidentifikasi *bottleneck* pada data yang tidak terstruktur sebagai penyebab penurunan performa signifikan dalam eksekusi *query* di *data warehouse* modern, sehingga diperlukan pengembangan model baru yang dapat meningkatkan kinerja *query* pada data berskala besar.

Menanggapi tantangan ini, dalam beberapa tahun terakhir telah terjadi pergeseran signifikan menuju pendekatan *Extract, Load, Transform* (ELT). Pergeseran ini didorong oleh migrasi organisasi ke lingkungan *cloud data warehouse* (seperti Snowflake, BigQuery, dan Redshift), yang arsitekturnya dirancang untuk memproses data dalam skala besar dan format yang beragam secara lebih efisien (dbt Labs, 2025). ELT menerapkan *schema-on-read*, yang memungkinkan data mentah untuk dimuat terlebih dahulu ke sistem target tanpa perlu transformasi awal, sehingga memberikan fleksibilitas dalam melakukan analisis dan eksplorasi data secara paralel dengan memanfaatkan kekuatan mesin komputasi *cloud* dari sistem target (dbt Labs, 2025). Hal itu diperkuat oleh survei yang dilakukan oleh Haryono et al. (2020) yang menunjukkan hasil bahwa ELT lebih unggul jika dibandingkan dengan ETL dari beberapa aspek, terutama kinerja, pemrosesan data bervolume besar, dukungan untuk data tidak terstruktur, biaya rendah, dan fleksibilitas.

Keunggulan utama dari pendekatan ini adalah peningkatan performa yang drastis. Praktik di industri menunjukkan bahwa implementasi arsitektur ELT mampu mereduksi waktu pemrosesan data secara signifikan, berkisar antara 50% hingga 90% tergantung pada karakteristik beban kerja dan kualitas implementasi. Meski demikian, tingkat adopsi ELT berbeda-beda antar sektor industri. Sektor teknologi, ritel, dan media menjadi yang paling banyak mengimplementasikan ELT karena kebutuhan mereka akan analisis yang lincah (*agile*) terhadap sumber data yang beragam. Sebaliknya, industri dengan regulasi ketat seperti di bidang keuangan dan kesehatan cenderung lebih berhati-hati dan sering kali mempertahankan pendekatan hibrida karena mereka masih berada di bawah tuntutan kepatuhan (*compliance*) dan silsilah data (*data lineage*). Dari sisi biaya, efisiensi penggunaan infrastruktur sangat bergantung pada pola penggunaan komputasi *cloud* dan optimasi *query*. Walaupun berpotensi menurunkan biaya hingga 30-40%, beberapa organisasi justru melaporkan peningkatan biaya operasional. Meski memiliki banyak kelebihan, ELT juga membawa kelemahan baru, yakni dari segi tata kelola (*governance*) dan adanya risiko untuk terbentuk *data swamp* jika pengelolaannya tidak dilakukan dengan baik. (Integrate.io, 2025)

Setiap pendekatan, baik ETL, ELT maupun hibrida, mempunyai kelebihan dan keterbatasannya masing-masing, baik dari aspek performa, biaya, skalabilitas, maupun keamanan. Metode hibrida muncul sebagai respons atas kebutuhan industri modern, dengan menggabungkan kontrol dan keamanan transformasi dari ETL, dengan fleksibilitas dan skalabilitas dari ELT di lingkungan *cloud* (Ong et al., 2017). Namun, pendekatan hibrida membawa kelemahan dari segi kompleksitas, baik secara implementasi maupun pemeliharaan, karena organisasi harus mengelola dua lingkungan pemrosesan secara bersamaan. Meskipun arsitektur integrasi data terus berevolusi, penelitian yang melakukan perbandingan secara

komprehensif antara model ETL, ELT, dan hibrida, masih terbatas, khususnya menyangkut integrasi data skala besar dengan format campuran (data terstruktur dan semi-terstruktur), serta pemanfaatan teknologi *cloud* yang makin meluas dalam industri saat ini.

Berdasarkan kesenjangan tersebut, tugas akhir ini berfokus pada analisis komparatif performa, skalabilitas, dan kemampuan adaptasi terhadap karakteristik 5V *big data* (*Volume, Velocity, Variety, Value, Veracity*) antara arsitektur ETL, ELT, dan hibrida. Ketiga model ini akan evaluasi melalui beberapa indikator performa, seperti kecepatan waktu proses, efisiensi penggunaan memori, skalabilitas, dan tata kelola. Hasil perbandingan ini diharapkan dapat menjadi referensi bagi profesional dan organisasi dalam memilih arsitektur *data pipeline* yang paling sesuai dengan kondisi lingkungan *big data* masing-masing.

1.2 Rumusan Masalah

Berdasarkan kesenjangan yang sudah diidentifikasi pada latar belakang, tugas akhir ini bertujuan untuk melakukan analisis komparatif secara kuantitatif dan kualitatif terhadap arsitektur data modern. Secara spesifik, fokus dari penelitian ini dirumuskan dalam pertanyaan-pertanyaan berikut:

1. Bagaimana perbandingan waktu pemrosesan total antara arsitektur ETL, ELT, dan hibrida dalam menangani beban kerja dengan bentuk data terstruktur dan semi-terstruktur?
2. Bagaimana perbandingan efisiensi penggunaan sumber daya (CPU dan memori) pada ketiga arsitektur tersebut selama proses integrasi data berlangsung?
3. Bagaimana perbandingan efisiensi penggunaan ruang penyimpanan *disk* pada ketiga arsitektur tersebut pada tahap penyimpanan data mentah dan setelah dimuat ke *data warehouse*?
4. Bagaimana perbandingan skalabilitas (*scalability*) dari ketiga arsitektur tersebut dalam merespons peningkatan volume data secara signifikan?

1.3 Batasan Masalah

Agar dapat berjalan secara lebih terarah dan hasilnya sesuai dengan tujuan yang ditetapkan, maka batasan masalah tugas akhir ini dibatasi pada beberapa aspek berikut:

1. Analisis akan difokuskan pada *dataset* TPC-H yang dikonversi menjadi dua jenis format: data terstruktur (dalam format CSV dan parquet) dan semi-terstruktur (dalam format JSON). Tugas akhir ini tidak akan menggunakan data tidak terstruktur seperti gambar, audio, foto, maupun video.
2. Metrik pengujian:
 - a. Waktu pemrosesan total
 - b. Penggunaan CPU
 - c. Penggunaan memori
 - d. Penggunaan ruang penyimpanan
3. Platform dan teknologi:
 - a. *Data warehouse* dengan menggunakan PostgreSQL untuk menyimulasikan lingkungan industri modern sekaligus meminimalisir biaya yang perlu digunakan.
 - b. Arsitektur ETL akan disimulasikan menggunakan Python untuk tahap transformasi data.

- c. Arsitektur ELT akan disimulasikan menggunakan Python untuk ekstraksi dan pemuatan data, dan *dbt (data build tool)* untuk melakukan transformasi data setelah data masuk di *data warehouse*.
- d. Arsitektur hibrida akan disimulasikan menggunakan kombinasi dari kedua pola, yakni menggunakan Python untuk ekstraksi serta pembersihan data sebelum dimuat dan melakukan pemuatan, kemudian menggunakan *dbt* untuk transformasi di dalam *data warehouse*.

1.4 Tujuan

Tujuan dari penulisan tugas akhir ini adalah sebagai berikut:

1. Untuk membandingkan waktu pemrosesan total antara arsitektur ETL, ELT, dan hibrida dalam menangani beban kerja dengan bentuk data terstruktur dan semi-terstruktur.
2. Untuk membandingkan efisiensi penggunaan sumber daya (CPU dan memori) pada ketiga arsitektur tersebut selama proses integrasi data berlangsung.
3. Untuk membandingkan efisiensi penggunaan ruang penyimpanan *disk* pada ketiga arsitektur tersebut pada tahap penyimpanan data mentah dan setelah dimuat ke *data warehouse*.
4. Untuk membandingkan skalabilitas (*scalability*) dari ketiga arsitektur tersebut dalam merespons peningkatan volume data secara signifikan.

1.5 Manfaat

Berdasarkan latar belakang, rumusan masalah, dan tujuan yang telah diuraikan, manfaat dari tugas akhir ini adalah sebagai berikut:

1. Praktis:
 - a. Memberikan rekomendasi teknis berbasis data dalam memilih arsitektur integrasi data yang paling optimal, berdasarkan karakteristik data dan kebutuhan spesifik mereka.
2. Akademis:
 - a. Menyajikan wawasan dan data perbandingan kuantitatif antara arsitektur ETL, ELT, dan hibrida yang dapat mengisi kesenjangan riset di bidang integrasi data berskala besar.

BAB 2 TINJAUAN PUSTAKA

2.1 Hasil Penelitian Terdahulu

Sub-bab ini berisi informasi mengenai beberapa penelitian terdahulu yang relevan dengan tugas akhir ini, di mana analisis yang dilakukan mencakup metodologi yang telah digunakan, temuan yang didapatkan, serta kesenjangan yang masih ada dalam literatur tersebut. Berikut merupakan beberapa informasi yang didapatkan:

Tabel 2.1 Penelitian Terdahulu 1

Jurnal	<i>Big Data and Cognitive Computing</i>
Judul	<i>A Model for Enhancing Unstructured Big Data Warehouse Execution Time</i>
Peneliti	Marwa Salah Farhan, Amira Youssef, Laila Abdelhamid
Penerbit	<i>Multidisciplinary Digital Publishing Institute</i>
Tahun	(2024)
Ringkasan	Penelitian ini bertujuan untuk mengatasi masalah utama dari arsitektur <i>data warehouse</i> tradisional, yaitu kinerja yang lambat dalam memproses data dalam volume besar, terutama data yang tidak terstruktur seperti teks. Menurut peneliti, model ETL dan ELT murni yang ada saat ini masih kurang efisien untuk kasus penggunaan tersebut. Sebagai solusi, mereka mengusulkan model baru yang disebut ECLT dengan 4 tahap: <i>extract</i> (Ekstrak data mentah dengan Spark), <i>clean</i> (Pembersihan data menggunakan PySpark), <i>load</i> (Konversi data teks yang sudah bersih menjadi data berbentuk tabel menggunakan RegEx dan memuatnya ke tabel Hive), dan <i>transform</i> (Transformasi analitis dilakukan di dalam <i>warehouse</i> Hive menggunakan Spark SQL). Hasil eksperimen yang telah digunakan menggunakan dua <i>dataset</i> teks (Spark Log dan Artikel Wikipedia) menunjukkan bahwa ECLT dapat secara signifikan lebih baik dari model ELT dan ETL dari segi waktu eksekusi dan penggunaan memori.
Keterkaitan	Penelitian ini secara langsung menyajikan perbandingan kinerja kuantitatif antara model ETL, ELT, dan beberapa jenis hibrida. Peneliti melakukan pengujian secara <i>on-premise</i> , sama seperti yang dilakukan pada tugas akhir ini. Penelitian ini juga memberikan justifikasi yang kuat terhadap pendekatan dengan model hibrida sebagai solusi potensial. Metrik pengujian yang digunakan juga sangat relevan dengan metrik yang digunakan dalam tugas akhir ini, yakni pada metrik waktu eksekusi dan penggunaan memori.
Kesenjangan	Meski dilakukan secara <i>on-premise</i> seperti yang akan dilakukan pada tugas akhir ini, penelitian ini menggunakan <i>tools</i> yang berbeda, yakni Spark, PySpark, dan Hive. Penelitian ini juga hanya berfokus pada data yang tidak terstruktur (teks) dan mengandalkan transformasi berbentuk RegEx.

Tabel 2.2 Penelitian Terdahulu 2

Jurnal	Journal of Computational Intelligence and Robotics
Judul	<i>ETL vs. ELT: Optimizing Data Integration for Retail and Insurance Analytics</i>
Peneliti	Venkatesha Prabhu Rambabu, Chandrashekar Althathi, Amsa Selvaraj
Penerbit	<i>The Science Brigade Publishers</i>
Tahun	(2023)
Ringkasan	Penelitian ini melakukan analisis komparatif antara integrasi data dengan metode ETL dan ELT untuk kebutuhan industri ritel dan asuransi dari beberapa aspek, yaitu performa, skalabilitas, dan efisiensi. Dari segi performa, yang dijadikan metrik pengukuran adalah kecepatan dan efektivitas pemrosesan data, terutama dengan <i>dataset</i> bervolume besar dan transformasi kompleks. Dari segi skalabilitas, penelitian tersebut menganalisis seberapa baik ETL dan ELT beradaptasi dengan volume data yang terus berkembang. Dari segi efisiensi penggunaan sumber daya, hal-hal yang dipertimbangkan adalah penggunaan sumber daya, biaya, dan dampak operasional masing-masing metode. Selain aspek-aspek kuantitatif tersebut, penelitian ini juga mempertimbangkan tingkat kepatuhan dan integritas data untuk masing-masing metode. Hasil yang disimpulkan adalah ELT, yang memanfaatkan kekuatan <i>cloud data warehouse</i> , umumnya lebih unggul dalam menangani data bervolume besar dan menawarkan fleksibilitas yang lebih tinggi dibandingkan metode ETL.
Keterkaitan	Penelitian ini secara langsung membandingkan antara ETL dan ELT, yang merupakan inti dari topik tugas akhir ini. Metrik perbandingan yang digunakan juga sama, yaitu performa/kinerja, skalabilitas, dan efisiensi sumber daya. Penelitian ini juga menunjukkan bahwa ELT adalah pendekatan modern untuk mengatasi masalah skalabilitas pada ETL.
Kesenjangan	Penelitian ini membandingkan ETL dan ELT secara kualitatif, tidak seperti tugas akhir ini yang juga membandingkannya secara kuantitatif. Penelitian ini juga tidak mempertimbangkan pendekatan secara hibrida, hanya berfokus pada salah satu dari ETL/ELT murni. Penelitian ini juga tidak memberikan pengujian kuantitatif spesifik tentang dampak perbedaan bentuk data (terstruktur, semi-terstruktur, tidak terstruktur) terhadap kinerjanya.

Tabel 2.3 Penelitian Terdahulu 3

Prosiding	2020 International Conference on Informatics, Multimedia, Cyber and Information System (ICIMCIS)
Judul	Comparison of the E-LT vs ETL Method in Data Warehouse Implementation: A Qualitative Study
Peneliti	Edward Manopo Haryono, Fahmi, Adi Setiawan Tri W, Indra Gunawan, Achmad Nizar Hidayanto, Untung Rahardja
Penerbit	<i>Institute of Electrical and Electronics Engineers Inc. (IEEE)</i>
Tahun	(2020)
Ringkasan	Penelitian ini melakukan studi kualitatif dalam bentuk survei kepada para profesional yang bekerja di sektor <i>warehouse</i> di berbagai industri

	(asuransi, perbankan, konsultan, <i>startup</i> , jaringan, dan pertambangan) untuk membandingkan metode ETL dan ELT dalam implementasi <i>data warehouse</i> . Hasil survei menunjukkan metode ELT dianggap lebih unggul dalam 17 aspek, terutama pada kinerja, pemrosesan data bervolume besar, dukungan untuk data tidak terstruktur, biaya rendah, dan fleksibilitas. Sebaliknya, ETL hanya unggul dalam aspek-aspek seperti <i>scope</i> , stabilitas, kemudahan implementasi awal, serta kualitas dan kebersihan data.
Keterkaitan	Penelitian ini membandingkan secara langsung antara ETL dan ELT, yang merupakan inti dari tugas akhir ini. Banyak metrik perbandingan yang diidentifikasi (seperti kinerja, dukungan data tidak terstruktur, kompleksitas, dan biaya) yang sangat relevan dengan metrik yang digunakan dalam tugas akhir ini.
Kesenjangan	Kesenjangan utama dari penelitian ini adalah dari segi metodologi. Penelitian ini bersifat kualitatif, berdasarkan pengalaman dan persepsi dari profesional (survei). Penelitian ini tidak menyajikan data kinerja kuantitatif seperti penggunaan sumber daya atau latensi pemrosesan. Selain itu, penelitian tidak mengevaluasi arsitektur hibrida dan tidak menguji dampak format data yang berbeda-beda (CSV, JSON, Parquet) terhadap kinerja kedua arsitektur tersebut.

Tabel 2.4 Penelitian Terdahulu 4

Jurnal	Journal of Information Systems Engineering and Management
Judul	Optimizing ETL Processes for High-Volume Data Warehousing in Financial Applications
Peneliti	Samyukta Rongala
Penerbit	Journal of Information Systems Engineering and Management (JISEM)
Tahun	(2025)
Ringkasan	Penelitian ini berfokus pada pengoptimalan proses ETL untuk data bervolume besar di sektor finansial. Peneliti mengidentifikasi beberapa tantangan utama seperti volume data yang masif, varietas data (terstruktur, semi-terstruktur, tidak terstruktur), dan kebutuhan akan kinerja dengan latensi rendah untuk aplikasi <i>real-time</i> seperti deteksi penipuan. Artikel ini juga menyajikan analisis <i>bottleneck</i> yang secara visual menampilkan bahwa tahap transformasi adalah tahap yang memiliki latensi tertinggi jika dibandingkan dengan ekstraksi dan pemuatan.
Keterkaitan	Penelitian ini fokus pada evaluasi dan optimalisasi performa ETL dengan metrik performa kuantitatif, sehingga cocok dengan tugas akhir ini yang juga mengevaluasi performa ETL. Penelitian ini juga menyajikan bukti visual yang menunjukkan bahwa transformasi adalah <i>bottleneck</i> utama dalam proses ETL.
Kesenjangan	Penelitian ini hanya berfokus pada mengoptimalkan ETL saja, tidak membahas maupun membandingkannya dengan arsitektur ELT ataupun hibrida. Artikel ini juga tidak menyebutkan ELT ataupun hibrida sebagai solusi potensial untuk <i>bottleneck</i> transformasi pada ETL. Tugas akhir ini dapat menguji apakah mengganti arsitektur ke ELT/hibrida adalah solusi yang lebih baik daripada sekedar mencoba untuk mengoptimalkan ETL.

Tabel 2.5 Penelitian Terdahulu 5

Prosiding	<i>3rd IEEE International Virtual Conference on Innovations in Power and Advanced Computing Technologies, i-PACT 2021</i>
Judul	<i>Heterogeneous Data Integration with ELT and Analytical MPP Database for Data Analysis Application</i>
Peneliti	Sivabalan S., Minu R. I.
Penerbit	<i>Institute of Electrical and Electronics Engineers Inc. (IEEE)</i>
Tahun	(2021)
Ringkasan	Penelitian ini membahas integrasi data dari sumber yang heterogen (beragam) untuk aplikasi analisis data. Peneliti secara langsung membandingkan antara ETL yang tradisional dengan ELT yang sudah lebih modern. Argumen utama yang disampaikan peneliti adalah bahwa ETL memiliki masalah dari segi kinerja dan biaya lisensi/hardware yang tinggi, karena proses transformasi data membutuhkan server ETL yang khusus. Sebagai solusi, peneliti mengusulkan penggunaan metode ELT yang dikombinasikan dengan basis data <i>Analytical Massively Parallel Processing</i> (MPP) seperti <i>cloud data warehouse</i>. Dalam model ini, transformasi dilakukan di dalam basis data MPP, memanfaatkan skalabilitas dan performa komputasi <i>cloud</i>. Di dalam prosiding ini juga dicantumkan grafik perbandingan kinerja yang menunjukkan bahwa ELT memiliki latensi pemrosesan jauh lebih rendah meski volume data bertumbuh.
Keterkaitan	Penelitian ini secara langsung membandingkan kinerja metode ETL dan ELT, dan mendapatkan kesimpulan bahwa ELT lebih unggul dalam hal kinerja dan skalabilitas, di mana itu sangat berkaitan dengan tugas akhir ini. Penelitian ini juga mengidentifikasi secara eksplisit biaya perangkat, dan menyebutkan bahwa <i>bottleneck</i> ETL terjadi karena sangat bergantung pada server yang digunakan untuk melakukan transformasi.
Kesenjangan	Penelitian ini berfokus pada arsitektur ETL dan ELT, tidak menyebutkan pendekatan secara hibrida. Meskipun menyebutkan data heterogen, penelitian ini tidak memberikan dampak spesifik format file terhadap kinerja masing-masing arsitektur.

Tabel 2.6 Penelitian Terdahulu 6

Judul	Perbandingan Performa Teknologi <i>Data Lake</i> dan <i>Data Warehouse</i> dalam Pengelolaan Data <i>Stack Overflow</i> Menggunakan Arsitektur <i>Hybrid Hub-and-Spoke</i> dan <i>Kimball Method</i>
Peneliti	Ihsan Kamil Al Khozi
Penerbit	Institut Teknologi Sepuluh Nopember
Tahun	(2025)
Ringkasan	Penelitian ini bertujuan untuk membandingkan performa antara <i>data warehouse</i> dan <i>data lake</i>. Metrik utama yang digunakan adalah waktu pembangunan sistem, durasi pemrosesan ETL, <i>Query per Second</i> (QPS), dan efisiensi penyimpanan. Untuk <i>data warehouse</i>, peneliti menggunakan <i>SQL Server 2019</i> sebagai tempat penyimpanannya dan <i>Pentaho Data Integration</i> sebagai alat ETL. Untuk <i>data lake</i>, peneliti

	menggunakan MinIO sebagai tempat penyimpanannya, Python untuk melakukan ETL, dan DuckDB sebagai mesin analitis. Dengan menggunakan dataset Stack Overflow 50 GB, penelitian ini menghasilkan kesimpulan bahwa <i>data lake</i> unggul secara signifikan dari segi QPS dan efisiensi penyimpanan, sedangkan <i>data warehouse</i> unggul dari segi kecepatan ETL jika data yang diperoleh dari sumber sudah terstruktur.
Keterkaitan	Penelitian ini melakukan analisis komparatif dengan menggunakan beberapa metrik yang relevan dengan tugas akhir ini, yakni durasi proses ETL dan efisiensi sumber daya. Penelitian ini juga bersifat kuantitatif dan dilakukan secara lokal (<i>on-premise</i>), sama seperti tugas akhir ini.
Kesenjangan	Penelitian ini lebih berfokus pada perbandingan performa antara <i>data warehouse</i> dan <i>data lake</i> , bukan metode integrasi data seperti ETL atau ELT. Oleh karena itu, teknologi yang digunakan juga berbeda (SQL Server/Pentaho dan MinIO/DuckDB).

2.2 Dasar Teori

Sub-bab ini berisi tentang penjelasan secara detail mengenai dasar teori yang digunakan dalam tugas akhir ini. Tujuannya adalah untuk memberikan dasar ilmiah bagi penelitian tugas akhir dan menguraikan kerangka pemikiran yang digunakan.

2.2.1 Karakteristik dan Tantangan pada Big Data

Big data adalah *dataset* dengan volume yang besar dan kompleks yang sulit dikelola, diproses, atau dianalisis menggunakan metode pemrosesan data yang berisi data terstruktur, semi-terstruktur, dan tidak terstruktur dengan volume sangat besar yang dihasilkan dari berbagai sumber, termasuk platform media sosial, sensor, perangkat digital, dan kegiatan komersial (Singh et al., 2025)

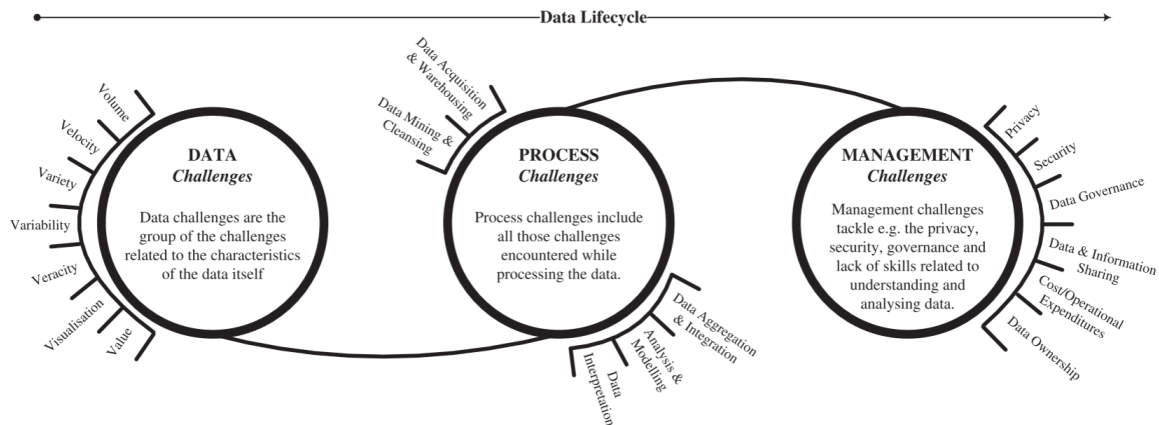
Menurut Naeem et al. (2022), karakteristik *big data* biasanya disebut dengan istilah “5V”:

1. *Volume* (jumlah data yang sangat besar)
2. *Variety* (keragaman tipe dan sumber data, mulai dari data terstruktur hingga data tidak terstruktur)
3. *Velocity* (kecepatan akuisisi dan pemrosesan data)
4. *Veracity* (kualitas, keakuratan, dan keandalan data)
5. *Value* (potensi nilai atau wawasan dan pengetahuan yang bisa diambil untuk pengambilan keputusan)

Kelima karakteristik (5V) inilah yang pada akhirnya menimbulkan serangkaian tantangan unik dan kompleks dalam akuisisi, pemrosesan, dan manajemen data. Sivarajah et al. (2017) dalam *Journal of Business Research* menunjukkan berbagai tantangan yang dihadapi organisasi dalam mengadopsi *big data*, yang dapat dikategorikan menjadi 3:

1. Tantangan data, yang berkaitan dengan karakteristik data tersebut (termasuk karakteristik 5V dari *big data*).
2. Tantangan proses, yang berkaitan dengan pertanyaan “bagaimana”, seperti bagaimana mengintegrasikan data, bagaimana mentransformasikan data, hingga bagaimana mengambil data.

3. Tantangan manajemen, yang berkaitan dengan privasi, keamanan, tata kelola, dan aspek-aspek etika.



Gambar 2.1 Klasifikasi tantangan konseptual dari big data (Sivarajah et al., 2017)

Meskipun teknologi dan platform sudah berkembang, tantangan utama *big data* di organisasi tetap pada masalah kebutuhan platform penyimpanan data dengan kecepatan baca dan tulis yang tinggi, kurangnya pemahaman terhadap *big data*, masalah integrasi, tata kelola (*governance*), standardisasi, *metadata*, serta pengelolaan kualitas dan keamanan data. Oleh karena itu, penelitian dan pengembangan *big data* saat ini banyak berfokus pada pembuatan kerangka tata kelola, *best practice*, dan pengembangan platform analitis untuk mendukung kebutuhan keberlanjutan bisnis.

2.2.2 Data Warehouse

Data warehouse adalah repositori terpusat dari sebuah organisasi, yang dirancang untuk menyimpan data yang bersifat nonvolatil (tidak mudah berubah), terintegrasi, bervariasi secara waktu, dan berorientasi subjek. *Data warehouse* dikembangkan pada tahun 1990-an untuk melakukan proses ETL yang menggabungkan data dari berbagai sumber menjadi satu untuk dapat dianalisis dengan beberapa perangkat. (Harby & Zulkernine, 2025)

Data warehouse mengorganisir data menjadi beberapa tabel dimensi dan tabel fakta. *Data warehouse* dibentuk berdasarkan pertanyaan 4W yang mendasari tabel fakta, yakni “What, Who, When, Where” (Khine & Wang, 2018). Dimensi pada *data warehouse* mencerminkan perspektif dari mana data dapat dilihat. Biasanya, dimensi dimodelkan sebagai hierarki tingkatan, di mana hierarki tersebut berisi elemen-elemen (atau anggota). Sedangkan, fakta merupakan data kuantitatif yang terkait dengan dimensi-dimensi, yang biasanya disebut dengan metrik. *Data warehouse* menggunakan struktur multidimensional yang disebut kubus data (*data cube*), yang memungkinkan pengambilan fakta pada tingkat granularitas yang berbeda-beda. *Data warehouse* mengikuti prinsip OLAP (*Online Analytical Processing*). Dalam prinsip ini, proses analisis data dilakukan menggunakan dimensi. Operasi OLAP memungkinkan data untuk diagregasi, difilter, dan dirujuk menggunakan dimensi. (Vallejos et al., 2018)

2.2.3 Data Lake

Data Lake adalah repositori terpusat dengan skalabilitas tinggi yang dapat menyimpan data mentah yang terstruktur, semi-terstruktur, atau tidak terstruktur dari berbagai sumber. Konsep Data Lake diperkenalkan oleh James Dixon pada tahun 2010. Konsep tersebut menjawab

permasalahan *data warehouse* yang membutuhkan biaya dan waktu yang besar untuk melakukan prapemrosesan, karena *data warehouse* membutuhkan struktur yang terdefinisikan sebelumnya, dan transformasi data yang lebih kompleks. (Harby & Zulkernine, 2025)

Data Lake telah berkembang sebagai repositori *big data* yang menyimpan data mentah dengan volume yang besar, dan memiliki berbagai fungsionalitas yang dibantu dengan deskripsi metadata yang lengkap. Meski *data lake* merupakan salah satu bentuk tempat penyimpanan data *enterprise* seperti pada umumnya, namun fitur-fitur analisis data yang disediakan oleh *data lake* berbeda dengan *data warehouse*. Semua data mentah yang dihasilkan oleh organisasi akan disimpan di dalam suatu struktur data yang disebut *data lake*, dalam bentuk format aslinya. Biasanya data yang sudah berada di dalam lake dialirkan ke *data warehouse* terlebih dahulu untuk dilakukan pemrosesan dan analisis data yang kompleks, agar kemudian data tersebut siap untuk digunakan. (Nambiar & Mundra, 2022)

Meski menjadi alternatif yang relatif lebih baik dari *data warehouse*, *data lake* juga membawa beberapa potensi masalah. Salah satu yang terbesar adalah risiko menjadi *data swamp* jika tidak dikelola dengan baik. *Data swamp* merupakan *data lake* yang berantakan sehingga tidak dapat digunakan untuk analisis sebelum dilakukan pemrosesan, yang akan memakan waktu lebih banyak (Brackenbury et al., 2018). Tidak ada prosedur yang mencegah masuknya data yang salah maupun duplikat. *Data lake* juga tidak dapat memberikan jaminan performa, tata kelola, keamanan, dan kontrol akses. Oleh karena banyaknya permasalahan yang dimiliki *data lake*, keberadaannya tidak serta-merta menggantikan *data warehouse*. Untuk menggunakan data dari kedua sistem ini secara efektif, diperlukan proses integrasi data. (Khin & Wang, 2018).

2.2.4 Integrasi Data

Integrasi data adalah konsep fundamental dalam manajemen data modern, yang menjadi landasan bagi *data warehousing* dan analitis bisnis. Secara formal, konsep integrasi data didefinisikan sebagai "masalah penggabungan data yang berada di sumber-sumber otonom yang berbeda, dan menyediakan pandangan terpadu (*unified view*) atas data tersebut kepada pengguna" (Beneventano et al., 2019). Definisi ini menyoroti dua tantangan utama:

1. Sumber otonom, yang menjelaskan bahwa data berasal dari sumber yang berbeda-beda (misalnya, basis data operasional, API eksternal, file log) yang tidak dirancang untuk berintegrasi satu sama lain
2. Pandangan terpadu, yang menjelaskan bahwa tujuan utama integrasi data adalah untuk menyajikan data dari sumber yang beragam seolah-olah berasal dari satu sumber yang terpadu, sehingga memudahkan proses analisis data.

Neang et al. (2021) dalam penelitiannya yang berjudul "*Data Integration as Coordination: The Articulation of Data Work in an Ocean Science Collaboration*" memberikan sudut pandang terhadap integrasi data sebagai pekerjaan koordinasi, bukan pertukaran. Mereka memandang proses penggabungan beberapa data (integrasi data) bukan hanya sebagai proses transaksi satu kali di mana data ditukar atau diberikan, tetapi proses ini lebih rumit dan membutuhkan koordinasi yang baik secara terus-menerus. Dalam praktiknya, masalah integrasi data menjadi semakin kompleks karena karakteristik "5V" pada *big data*. Organisasi harus menggabungkan data terstruktur, semi-terstruktur, dan tidak terstruktur. Kompleksitas integrasi data bergantung pada berbagai faktor, termasuk model, format, dan tingkat keakuratan data. Untuk mengatasi

tantangan ini, berbagai arsitektur telah dikembangkan. Dua arsitektur paling dominan yang dapat mengatasi masalah ini adalah ETL (*Extract, Transform, Load*) dan ELT (*Extract, Load, Transform*).

2.2.5 Extract, Transform, Load (ETL)

Extract, Transform, dan Load (ETL) merupakan rangkaian proses inti dalam integrasi data untuk memindahkan dan menyiapkan data untuk kebutuhan analisis. Menurut IBM (n.d.), ETL merupakan proses integrasi data yang mencakup tahap penggabungan, pembersihan, dan pengelolaan data dari beberapa sumber menjadi satu *dataset* yang konsisten di dalam suatu *data warehouse*, *data lake*, atau sistem target lainnya.

Kimball & Ross (2013) dalam buku fundamental mereka “*The Data Warehouse Toolkit*” mendefinisikan ETL sebagai segala sesuatu yang berada di antara sistem sumber operasional dan area *data warehouse / business intelligence* (DW/BI), yang bertujuan untuk memastikan agar data terintegrasi, bersih, konsisten, dan siap untuk dianalisis secara efektif. Dalam proses ini, data diekstrak, dibersihkan, dilakukan transformasi, dan diintegrasikan agar siap digunakan dalam kebutuhan analisis bisnis. Proses ETL bertanggung jawab terhadap konsistensi data, keakuratan, kesesuaian format, serta eliminasi duplikasi, yang merupakan aspek krusial agar *data warehouse* dapat menunjang proses analisis bisnis secara optimal.

Menurut Kimball & Ross (2013), berikut merupakan tiga tahapan utama ETL:

1. Ekstraksi (*Extract*)

Tahap ekstraksi adalah tahap pertama atau ‘E’ dari ETL, yang merupakan proses memasukkan data ke lingkungan *data warehouse*. Dalam tahap ini, data dari sumber dibaca dan dipahami untuk kemudian dimasukkan ke sistem ETL untuk kebutuhan manipulasi data selanjutnya. Pada tahap ini, data sudah menjadi milik *data warehouse*, tidak lagi berada di sistem sumber.

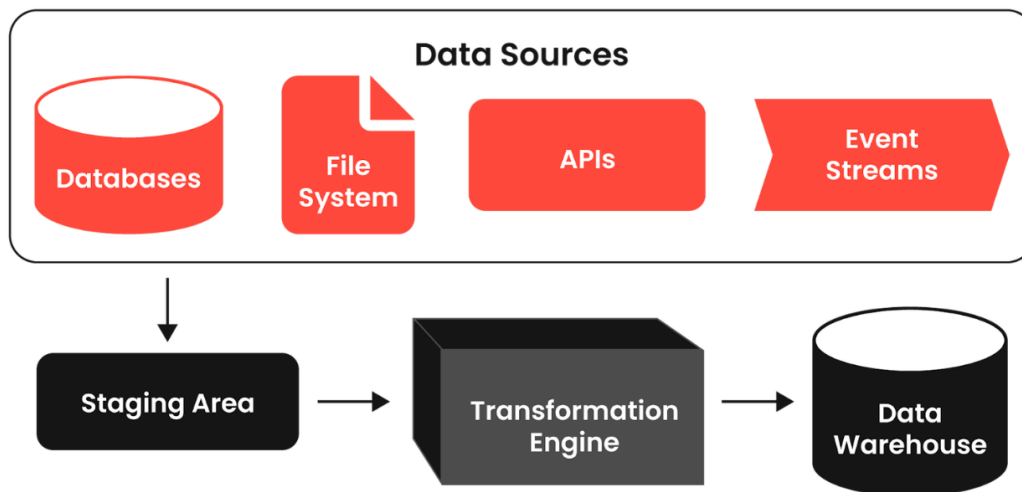
2. Transformasi (*Transform*)

Setelah data berhasil diekstraksi ke sistem ETL, data tersebut kemungkinan besar membutuhkan transformasi (atau tahap ‘T’ pada ‘ETL’). Transformasi yang biasanya dilakukan mencakup pembersihan data (standarisasi data dengan melakukan parsing ke format yang sesuai, mengelola data yang hilang, kesalahan penulisan, dan lain sebagainya). Selain pembersihan data, pada tahap transformasi ini juga bisa dilakukan pembuatan metadata, untuk meningkatkan kualitas data seiring waktu.

3. Pemuatan (*Load*)

Tahap terakhir dari ETL, yang mencakup penataan fisik dan pemuatan data ke dalam model dimensional pada area presentasi di *data warehouse*.

Secara arsitektural, ciri khas ETL adalah proses transformasi (T) yang terjadi setelah ekstraksi (E) namun sebelum pemuatan (L). Proses ini tidak dilakukan di dalam sistem *warehouse* target. Data yang diekstrak akan kemudian dimuat terlebih dahulu di lingkungan yang disebut area *staging*. Di server inilah semua transformasi berat dilakukan, termasuk pembersihan, agregasi, dan penggabungan data dilakukan. Setelah data sudah bersih dan dilakukan transformasi baru dilakukan pemuatan ke *data warehouse* target. (DataForge, 2023)



Gambar 2.2 Extract, Transform, Load (ETL) (DataForge, 2023)

Seiring perkembangan teknologi, tantangan ETL semakin kompleks dengan munculnya *big data* yang memiliki karakteristik data bervariasi dengan volume yang lebih besar. Farhan et al. (2024) menunjukkan adanya kebutuhan untuk dikembangkan alternatif dari proses ETL ini, mengintegrasikan sumber data yang lebih banyak, serta menyesuaikan proses transformasi agar mampu menangani data terstruktur maupun tidak terstruktur dalam skala besar secara efisien.

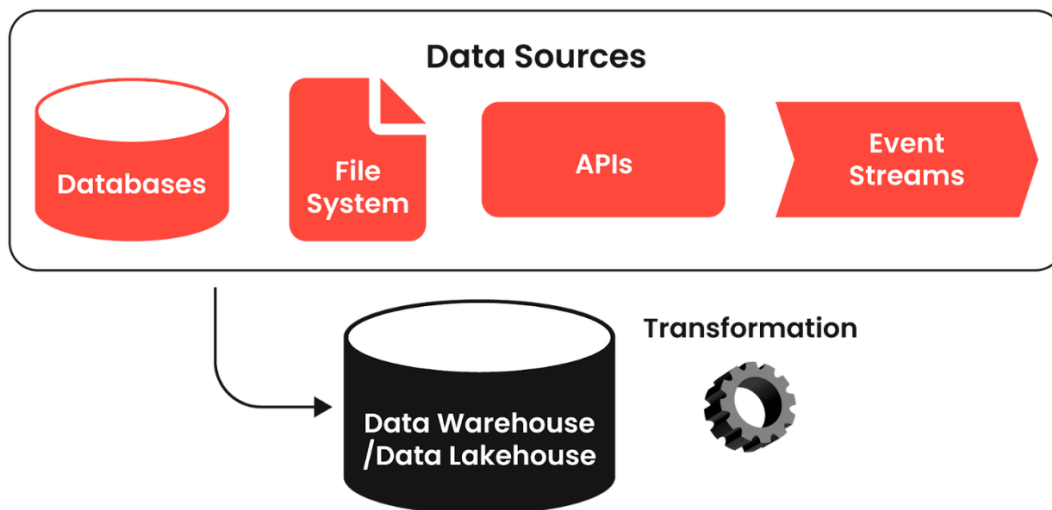
Dalam beberapa literatur terbaru, tren integrasi data telah beralih sebagian ke ELT (*Extract, Load, Transform*), terutama dalam ekosistem awan (*cloud*) yang lebih cocok untuk pemrosesan paralel dan dengan volume data yang lebih besar. Namun, ETL masih relevan di banyak kasus *enterprise*, karena kebutuhan tata kelola, kepatuhan (*compliance*), dan transformasi data kritis yang kompleks.

Secara umum, ETL masih menjadi fondasi proses integrasi data yang vital, dan penelitian masih terus dilakukan untuk mengembangkan optimalisasi performa, dan hibridisasi ETL dengan teknologi baru untuk lebih menyesuaikan dengan kebutuhan industri modern.

2.2.6 Extract, Load, Transform (ELT)

Extract, Load, Transform (ELT) merupakan proses integrasi data di mana data mentah diekstrak dari berbagai sumber, kemudian langsung dimuat ke dalam sistem data target, tanpa perlu melakukan transformasi di area *staging* yang terpisah (Google Cloud, n.d.). ELT merupakan alternatif dari metode yang telah dikembangkan sebelumnya, yaitu ETL (*Extract, Transform, Load*) yang memiliki permasalahan dari segi fleksibilitas, biaya, dan ketidakmampuannya dalam mengelola data dengan kompleksitas yang tinggi.

Berbeda dengan ETL, arsitektur ELT tidak memerlukan area *staging* terpisah untuk melakukan transformasi datanya. Sebagai gantinya, ELT memuat data mentah langsung ke *data warehouse* tujuan. Proses transformasi (T) pada ELT kemudian dilakukan di dalam *data warehouse* di mana data disimpan dan diproses, memanfaatkan kekuatan dan skalabilitas dari platform tersebut. Berikut merupakan penjelasan mengenai masing-masing prosesnya:



Gambar 2.3 Extract, Load, Transform (ELT) (DataForge, 2023)

1. Ekstraksi (*Extract*)

Tahap awal ini melibatkan pengumpulan data mentah dari sumber aslinya. Sumber-sumber ini bisa sangat beragam, termasuk basis data (SQL dan NoSQL), aplikasi perusahaan (seperti CRM dan ERP), platform SaaS, API, dan file *log*. Proses ekstraksi berfokus pada pengambilan data dari sistem-sistem ini secara efisien.

2. Pemuatan (*Load*)

Pada tahap kedua, data mentah yang telah diekstraksi dimuat, sering kali dalam format aslinya atau dengan pemrosesan minimal, langsung ke dalam sistem penyimpanan berkapasitas tinggi. Target umum untuk data mentah ini adalah *data lake* atau *data warehouse* modern yang dapat menangani volume besar data terstruktur, semi-terstruktur, dan tidak terstruktur.

3. Transformasi (*Transform*)

Tahap akhir ini terjadi setelah data tersimpan dengan aman di dalam sistem target. Menggunakan daya komputasi dari *data warehouse* atau *data lake*, data mentah tersebut dibersihkan, distrukturkan, diperkaya, dan diubah ke dalam format yang sesuai untuk analitis, pelaporan, dan *machine learning*. Transformasi dapat mencakup pemfilteran, penggabungan, agregasi, standardisasi format, dan penurunan titik data baru.

Proses ELT menawarkan fleksibilitas karena transformasi tidak ditetapkan sebelum pemuatan. *Data scientist*, misalnya, dapat mengakses data mentah untuk mengeksplorasi pola-pola yang tidak terduga atau melakukan analisis *ad-hoc*, sementara tim inteligensi bisnis dapat membangun *dataset* yang telah ditransformasi untuk pelaporan. Namun, proses ELT juga memiliki beberapa kelemahan, terutama dari segi tata kelola, keamanan, serta adanya potensi mengubah *data lake* menjadi *data swamp* jika data selalu dimuat apa adanya tanpa adanya penstrukturan dan tata kelola yang baik (Harby & Zulkernine, 2025).

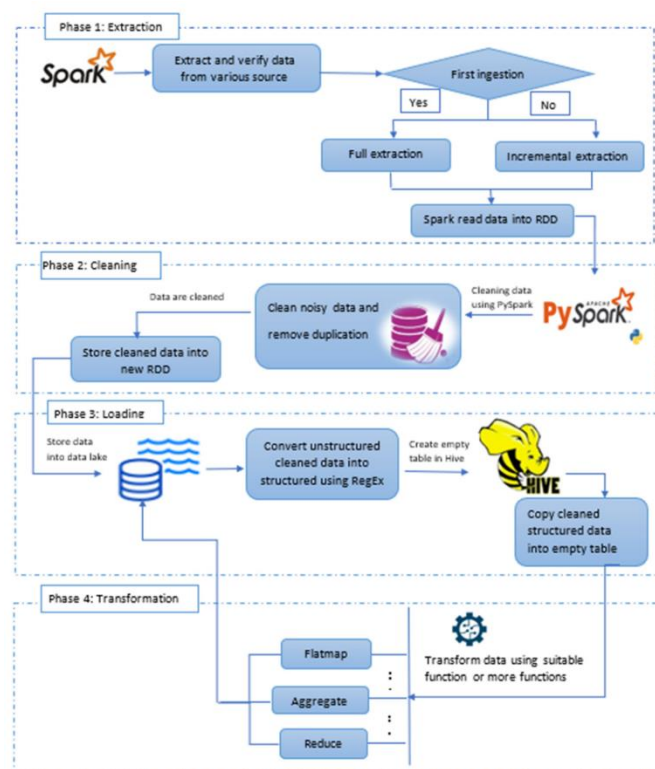
2.2.7 Arsitektur Hibrida

Extract, Transform, Load (ETL) dan *Extract, Load, Transform* (ELT) merupakan dua arsitektur integrasi data yang populer. Namun, masih terdapat kekurangan dari masing-masing metode. Meski ETL unggul dari segi tata kelola, kepatuhan, dan kemampuannya untuk mentransformasi data secara kompleks, ETL kurang dapat diandalkan jika volume data yang digunakan sangat besar, munculnya data semi-terstruktur atau tidak terstruktur, maupun adanya

tuntutan *real-time*. Di sisi lain, ELT menjadi alternatif yang baik bagi ETL karena kemampuannya untuk menangani data semi-terstruktur maupun tidak terstruktur, dan dapat memenuhi kebutuhan industri untuk melakukan analisis *real-time*. (Singhal & Aggarwal, 2022)

Untuk menanggapi kekurangan dari metode ETL dan ELT, penelitian telah mengembangkan pendekatan arsitektur hibrida. Pendekatan arsitektur hibrida ini bertujuan untuk mengambil kekuatan dari kedua model. Secara umum, arsitektur hibrida menerapkan sebagian transformasi (seperti validasi dan pembersihan data) sebelum data dimuat ke dalam sistem target (seperti ETL), sekaligus memanfaatkan kekuatan *engine* dari sistem tujuan yang lebih kuat untuk menangani beban transformasi yang berat (seperti agregasi dan penggabungan) setelah data dimuat (seperti ELT). (Farhan et al., 2024)

Salah satu implementasi pendekatan arsitektur hibrida yang terdokumentasi dalam literatur adalah model ECLT (*Extract, Clean, Load, dan Transform*) yang diusulkan oleh Farhan et al. (2024).



Gambar 2.4 Model ECLT (Farhan et al., 2024)

Pada dasarnya, model ECLT yang diusulkan oleh Farhan et al. (2024) didasari oleh temuan bahwa pendekatan ELT murni masih belum efisien untuk mengelola data yang tidak terstruktur (seperti teks). Pada dasarnya, model ini merupakan pendekatan hibrida yang membagi langkah transformasi menjadi dua tahap. Tahap pertama adalah transformasi ringan (yang disebut langkah “*Clean*”) yang dilakukan sebelum data dimuat ke dalam sistem target. Tahap ini berfokus pada validasi kualitas data, seperti pembersihan data, penggantian nama kolom, penyesuaian tipe data, penghapusan duplikasi, dan penghapusan *noise* (data pengganggu). Proses ini dieksekusi di luar *data warehouse*. Kemudian, tahap kedua adalah transformasi berat (yang disebut langkah “*Transform*”) yang dilakukan setelah data bersih dimuat di dalam sistem target. Transformasi yang dilakukan pada tahap ini lebih bersifat analitis, seperti agregasi dan

penggabungan data. Beban ini diserahkan ke mesin dari *data warehouse* tujuan. Penerapan model ECLT ini terbukti dapat meminimalisir waktu pemrosesan data jika dibandingkan dengan penerapan ETL atau ELT murni.

2.2.8 Format Penyimpanan Data

Dalam arsitektur data modern, faktor *variety* (keberagaman tipe data) merupakan salah satu tantangan utama. Pemilihan format penyimpanan file merupakan hal yang sangat krusial dan dapat memengaruhi biaya penyimpanan, kecepatan pemuatan data, dan kinerja *query* analisis. Terdapat berbagai jenis format file, beberapa di antaranya adalah CSV, JSON, dan parquet.

2.2.8.1 Comma-Separated Values (CSV)

Comma-Separated Values (CSV) merupakan format file berbasis teks yang paling umum untuk menyimpan data tabular dalam struktur baris dan kolom yang sederhana. Format ini menjadi pilihan utama untuk impor dan ekspor dalam basis data dan aplikasi *spreadsheet* karena kesederhanaan dan kompatibilitasnya lintas platform (Microsoft, 2025). Format ini disimpan dengan ekstensi file *.csv*. Ciri khas format CSV adalah penggunaan *delimiter*, yaitu karakter/tanda yang berfungsi sebagai pemisah antar kolom dalam suatu baris. Seperti namanya, tanda pemisah yang sering digunakan dalam format CSV adalah tanda koma (,). Struktur CSV juga relatif sederhana, baris pertama biasanya berisi nama-nama kolom (*header*), dan baris-baris berikutnya menyimpan data aktual di mana setiap nilai dipisahkan oleh *delimiter*.

2.2.8.2 JavaScript Object Notation (JSON)

JavaScript Object Notation (JSON) adalah format pertukaran data berbasis teks yang ringan dan efisien. Format ini dirancang agar mudah dibaca oleh manusia sekaligus mudah untuk diurai (*parse*) oleh mesin. Meskipun sintaksnya diturunkan dari JavaScript, JSON bersifat independen terhadap bahasa pemrograman, yang menjadikannya ideal sebagai format pertukaran data universal. (json.org, n.d.)

JSON disimpan dalam file dengan ekstensi *.json*. Secara struktural, dokumen JSON berisi pasangan kunci-nilai (*key-value*) yang disusun dari tiga komponen dasar. Komponen dasar tersebut mencakup *objects* (kumpulan pasangan kunci-nilai yang diapit kurung kurawal {}), *arrays* (kumpulan nilai terurut dan diapit oleh kurung siku []), dan nilai itu sendiri (yang dapat berupa *string*, angka, objek, atau *Boolean* (Bourhis et al., 2020). JSON mampu merepresentasikan data kompleks karena mendukung struktur bersarang (*nested*), seperti objek di dalam objek atau array di dalam objek. Kemampuan inilah yang mengklasifikasikannya sebagai format data semi-terstruktur.

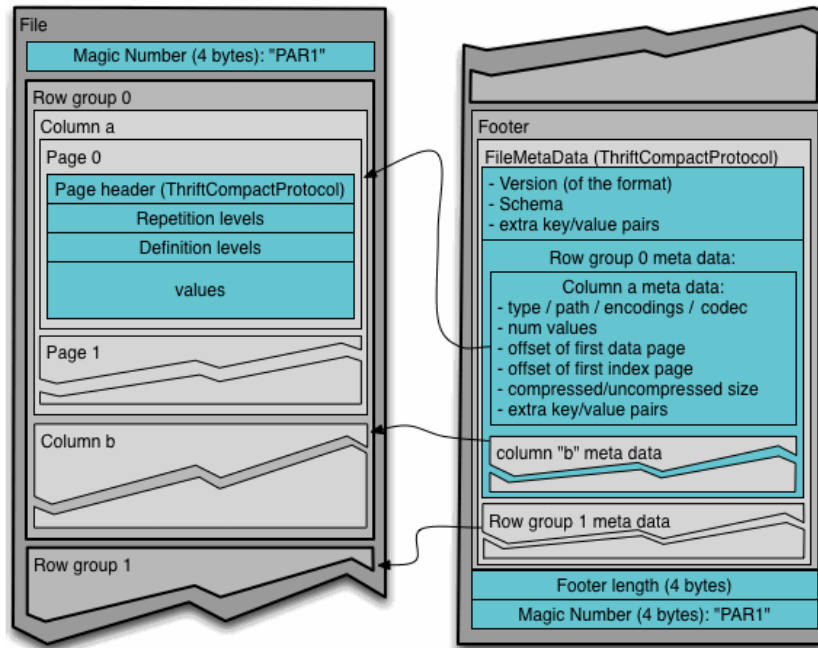
Di balik fleksibilitasnya, penggunaan JSON memiliki beberapa tantangan. Tidak adanya kerangka kerja JSON yang universal berpotensi menimbulkan ketidakkonsistenan data selama proses integrasi atau migrasi antar sistem. Ketidakkonsistenan itu juga dapat meningkatkan kompleksitas dalam melakukan *query* secara signifikan. Selain itu, pemrosesan data JSON juga dapat menjadi tantangan dari segi performa, terutama dalam menangani volume data yang sangat besar, yang berpotensi menyebabkan *bottleneck* pada operasi berintensitas tinggi. (Bourhis et al., 2020)

2.2.8.3 Apache Parquet

Apache Parquet adalah format penyimpanan kolumnar yang dirancang khusus untuk menyimpan, mengelola, dan menganalisis *big data* secara efisien. File berformat Parquet menggunakan ekstensi *.parquet*. Format ini menawarkan kompresi dan skema *encoding* berperforma tinggi untuk menangani data yang kompleks. Sebagai format yang dapat mendeskripsikan diri sendiri, Parquet juga menyertakan skema dan metadata langsung di dalam filenya, sehingga memungkinkan evolusi skema seiring perubahan kebutuhan bisnis. Parquet didukung oleh berbagai *platform* pemrosesan data, termasuk Apache Spark, Hive, dan Google BigQuery. (*Apache Parquet: Documentation*, 2024)

Format data yang digunakan menyesuaikan kebutuhan masing-masing organisasi. Banyak format file tradisional yang umumnya mengatur data dalam baris (*row-based*). Format berbasis baris efektif untuk pemrosesan data transaksional OLTP (*Online Transaction Processing*) yang berfokus pada optimalisasi transfer data dan keterbacaan. Sebaliknya, pendekatan yang diambil Parquet mengelompokkan data dalam kolom. Struktur ini ideal diimplementasikan untuk analisis data OLAP (*Online Analytical Processing*) berskala besar. Hal ini karena proses analisis dapat dilakukan lebih selektif dengan hanya membaca kolom-kolom yang diperlukan, tanpa harus membaca seluruh baris data, sehingga dapat meningkatkan kecepatan *query* secara signifikan. (IBM, 2025)

Parquet memiliki beberapa karakteristik utama yang membedakannya dengan format yang lain. Karakteristik pertamanya yaitu Parquet menyimpan data dalam format penyimpanan kolumnar, sehingga memungkinkan Parquet untuk membaca nilai hanya pada kolom-kolom yang relevan, di mana itu dapat mengurangi waktu *query* analisis. Parquet juga mengoptimalkan penyimpanan data melalui teknik kompresi yang tergolong efisien, dengan menghapus informasi yang duplikat yang tidak diperlukan dan menyimpan data dalam format yang lebih hemat ruang. Karakteristik lainnya adalah Parquet juga memiliki cetak biru berupa skema dan metadata yang mendeskripsikan dirinya sendiri, sehingga memungkinkan model data berkembang seiring perubahan kebutuhan bisnis. (IBM, 2025)



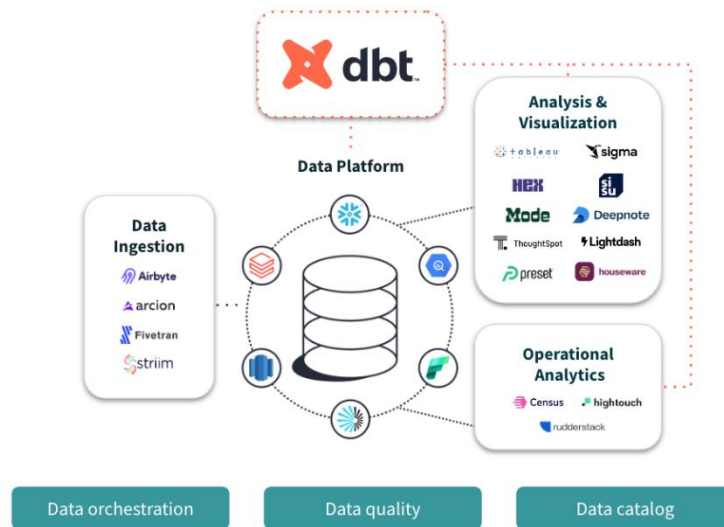
Gambar 2.5 Struktur File Parquet (Apache Parquet: Documentation, 2024)

Metadata yang disimpan pada file berformat Parquet mencatat informasi vital seperti skema file, tipe data, statistik kolom, dan lokasi setiap barisnya. Metadata ini yang membantu pengoptimalan *query* yang lebih efisien. Saat *query* dieksekusi, mesin *query* akan membaca metadata terlebih dahulu untuk mengidentifikasi kolom mana yang relevan untuk dibaca dengan *query* tersebut. Dengan itu, hanya potongan kolom yang diperlukan yang dibaca dari penyimpanan, dan data dikompresi dan diterjemahkan sesuai kebutuhan, menghindari pembacaan penuh dan secara drastis meningkatkan performa analisis. (IBM, 2025)

Proses pemrosesan data dalam format Apache Parquet terdiri dari beberapa tahapan. File Parquet biasanya mengatur data ke dalam beberapa komponen. Data diorganisasikan ke dalam *row groups* (Kelompok Baris), yang kemudian dipecah lebih lanjut menjadi *column chunks* (Potongan Kolom). Unit paling kecil dalam Parquet adalah *page* (Halaman), yang dapat mencakup ribuan nilai. Proses penulisan standar melibatkan langkah-langkah seperti *encoding* (pengodean) dan *compression* (kompresi) data sebelum data tersebut ditulis ke dalam file. (Shang et al., 2022)

2.2.9 Data Build Tool (dbt)

dbt (*data build tool*) merupakan sebuah alat *open-source* modern yang secara khusus dirancang untuk membantu analis data melakukan transformasi data, terutama dalam penerapan metode ELT (*extract, load, transform*). Tidak seperti alat ETL pada biasanya yang mencakup tahap ekstraksi dan pemuatan, dbt hanya berfokus pada tahap “T” atau transformasi data. dbt dijalankan setelah data mentah dimuat ke dalam *cloud data warehouse* atau *data lake*, sehingga memungkinkan *engineer* untuk berfokus pada pemodelan data (transformasi) dengan lebih baik. (dbt Labs, n.d.)



Gambar 2.6 Arsitektur Penerapan dbt

dbt memungkinkan analis data dan *engineer* untuk menulis kode SQL modular untuk melakukan transformasi data mentah di *cloud data warehouse* seperti Snowflake, BigQuery, Redshift, dan banyak lagi. Dengan itu, perubahan pada satu model akan secara otomatis menyesuaikan ke semua dependensinya, mengurangi kesalahan ketika terjadi perubahan di proses bisnis. Kekuatan utama dbt terletak pada kemampuannya untuk mengelola skrip SQL dalam alur kerja analitis dengan lebih baik. Selain itu, dbt juga dapat mengotomasi eksekusi transformasi SQL sekaligus menyediakan pengujian dan dokumentasi. (Zagni, 2023)

2.2.10 Python

Dalam bukunya yang berjudul “Beginning Programming with Python for Dummies”, Mueller (2014) menyebutkan bahwa Python merupakan salah satu bahasa pemrograman yang paling populer dari sekian banyaknya bahasa pemrograman yang ada saat ini. Python kerap digunakan karena berbagai alasan, seperti penggunaan kode yang efisien dan tingkat kesulitan yang rendah untuk memahaminya. Bahasa Python dapat digunakan di berbagai skenario, seperti pembuatan prototipe aplikasi dengan cepat, melakukan *scripting* di aplikasi web, berurusan dengan file dengan format XML, berinteraksi dengan basis data, hingga melakukan perhitungan matematis hingga saintifik dengan berbagai *library* yang tersedia.

Dalam konteks rekayasa dan analisis data, kekuatan utama Python terletak pada ekosistem *library* yang bervariasi, serbaguna, dan sangat matang. Dengan *library* yang tersedia, Python dapat menangani data-data tabular seperti basis data dan melakukan transformasi data yang kompleks. Kemampuan ini mencakup membaca dan menulis file dengan berbagai format, serta melakukan transformasi data seperti pembersihan, penyesuaian tipe data, penggabungan (*join*), pengurutan, hingga agregasi data. Dengan kemampuan tersebut, Python juga dapat digunakan sebagai *framework* untuk menyimulasikan proses Transformasi (T) pada arsitektur ETL. Kemampuannya untuk melakukan transformasi data secara fleksibel memungkinkan seluruh transformasi dan pemodelan data (termasuk *join* dan agregasi) dilakukan di antara sistem sumber dan target. (Lau et al., 2023)

2.2.11 PostgreSQL

PostgreSQL merupakan sebuah server basis data *open source* yang canggih. Sebagai RDBMS (*Relational Database Management System*), fungsi dasar PostgreSQL adalah untuk penyimpanan dan pengambilan data yang diatur dalam struktur yang seragam. Proyek PostgreSQL berawal dari pengembangan basis data Ingres oleh tim yang dipimpin oleh Michael Stonebraker pada tahun 1977-1985 di *University of California* di Berkeley. Sebagai perangkat lunak *open source*, pengembangannya tidak bergantung pada suatu perusahaan, namun dilakukan oleh tim global yang terdiri dari para pemrogram di Internet. (Momjian, 2001)

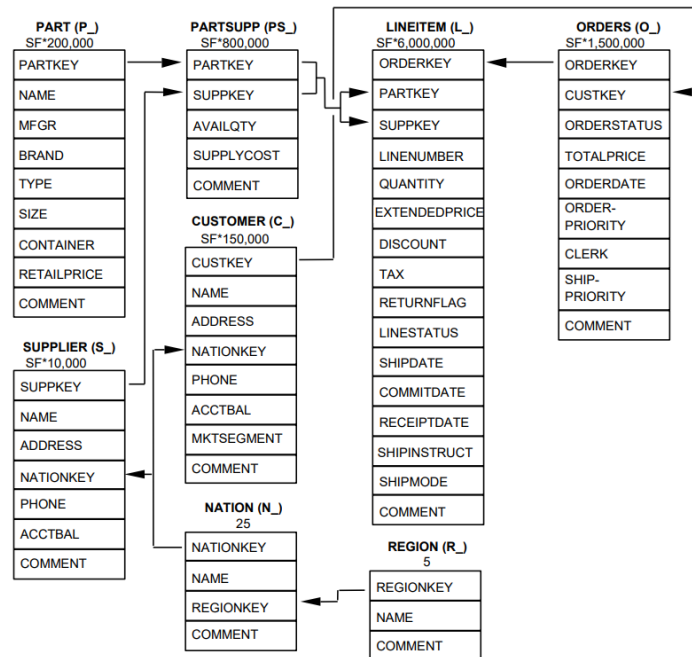
Interaksi utama dalam PostgreSQL dilakukan dengan menggunakan SQL (*Structured Query Language*). Beberapa perintah dasar SQL meliputi:

1. CREATE TABLE : Membuat tabel baru
2. INSERT : Menambahkan data (baris) baru ke dalam tabel
3. SELECT : Mengambil data dari tabel
4. UPDATE : Melakukan modifikasi data yang sudah ada di dalam tabel
5. DELETE : Menghapus data (baris) dari tabel
6. DROP TABLE : Menghapus tabel beserta strukturnya.

Selain perintah dasar SQL, PostgreSQL juga mendukung perintah-perintah SQL yang lebih canggih, seperti transaksi (yang dapat memberikan kapabilitas “semua atau tidak sama sekali” saat melakukan modifikasi data), *join* (penggabungan tabel yang dilakukan dengan membandingkan nilai kolom), maupun indeks (digunakan untuk meningkatkan performa). (Momjian, 2001)

2.2.12 TPC-H (TPC Benchmark™ H)

TPC Benchmark™ H merupakan salah satu *benchmark* standar industri yang dikembangkan oleh *Transaction Processing Performance Council* (TPC). Data yang digunakan dalam *benchmark* ini juga telah disesuaikan agar dapat mensimulasikan lingkungan bisnis secara universal dan relevan untuk berbagai sektor industri. TPC-H dirancang khusus untuk mengukur kinerja sistem basis data terutama dalam menangani tantangan di lingkungan sistem pendukung keputusan modern, seperti kebutuhan untuk menangani data bervolume sangat besar dan mengeksekusi *query* yang kompleks untuk mendapatkan wawasan bisnis yang krusial bagi industri. (*TPC-H Official Homepage*, n.d.)



Gambar 2.7 Skema TPC-H (TPC Benchmark H Standard Specification Book, 1993)

TPC Benchmark H Standard Specification Book (1993) dalam dokumen spesifikasi standar “TPC Benchmark™ H Standard Specification” menyatakan bahwa *benchmark* TPC-H berfokus pada simulasi lingkungan bisnis pada tahap analisis, yang lebih menekankan pada komputasi tren dan penyediaan data yang sudah diolah untuk mendukung keputusan bisnis yang tepat. Hal ini berbeda dengan *benchmark* TPC lainnya yang umumnya memodelkan sisi operasional bisnis (OLTP), di mana transaksi dilakukan secara *real-time*. Alih-alih menerima aliran data mentah secara terus-menerus seperti pada OLTP, basis data TPC-H menerima data secara periodik. (TPC Benchmark H Standard Specification Book, 1993)

TPC memiliki program generator data resmi yang dinamakan sebagai DBGEN. Program ini dapat membuat skema basis data yang biasanya terdiri dari delapan tabel, yaitu *Supplier*, *Part*, *PartSupp*, *Customer*, *Orders*, *LineItem*, *Nation*, dan *Region*. Tabel-tabel tersebut juga dapat diisi dengan data yang valid dan acak, namun tetap konsisten dengan data bisnis yang cukup realistis, sesuai dengan standar dan spesifikasi yang dimiliki oleh TPC-H. Pembuatan volume data dikontrol dengan parameter yang disebut dengan *Scale Factor* (SF). SF merupakan nilai numerik yang menentukan ukuran basis data TPC-H yang akan dihasilkan. Dengan mengubah nilai SF, peneliti dapat dengan mudah mengatur skala data yang akan digunakan untuk menguji suatu sistem. Secara umum, SF dari suatu data berbanding lurus dengan ukuran data yang akan dihasilkan (dalam *Gigabyte*). Sebagai contoh, SF 1 akan menghasilkan data dengan ukuran sekitar 1 GB, dan SF 100 akan menghasilkan data dengan ukuran sekitar 100 GB. Berikut merupakan tabel perkiraan ukuran masing-masing tabel pada basis data TPC-H dengan *Scale Factor* 1:

Tabel 2.7 Perkiraan Ukuran Basis Data TPC-H dengan SF = 1 (TPC Benchmark H Standard Specification Book, 1993)

Nama Tabel	Kardinalitas (dalam satuan baris)	Panjang baris (dalam bytes)	Ukuran tabel (dalam MB)
SUPPLIER	10000	159	2
PART	200000	155	30
PARTSUPP	800000	144	110
CUSTOMER	150000	179	26
ORDERS	1500000	104	149
LINEITEM	6001215	112	641
NATION	25	128	< 1
REGION	5	124	< 1
Total	8661245		956

Perlu dicatat bahwa ukuran dari beberapa tabel seperti *Nation* dan *Region* bersifat konstan, sehingga tidak dipengaruhi oleh ukuran SF. Kardinalitas (jumlah baris) *LineItem* yang tertera pada tabel 2.7 juga bukan merupakan kelipatan yang mutlak dari SF, karena setiap pesanan dapat memiliki jumlah *lineitem* yang berbeda-beda (dengan rata-rata jumlah baris *item* per pesanan adalah 4). (TPC Benchmark H Standard Specification Book, 1993)

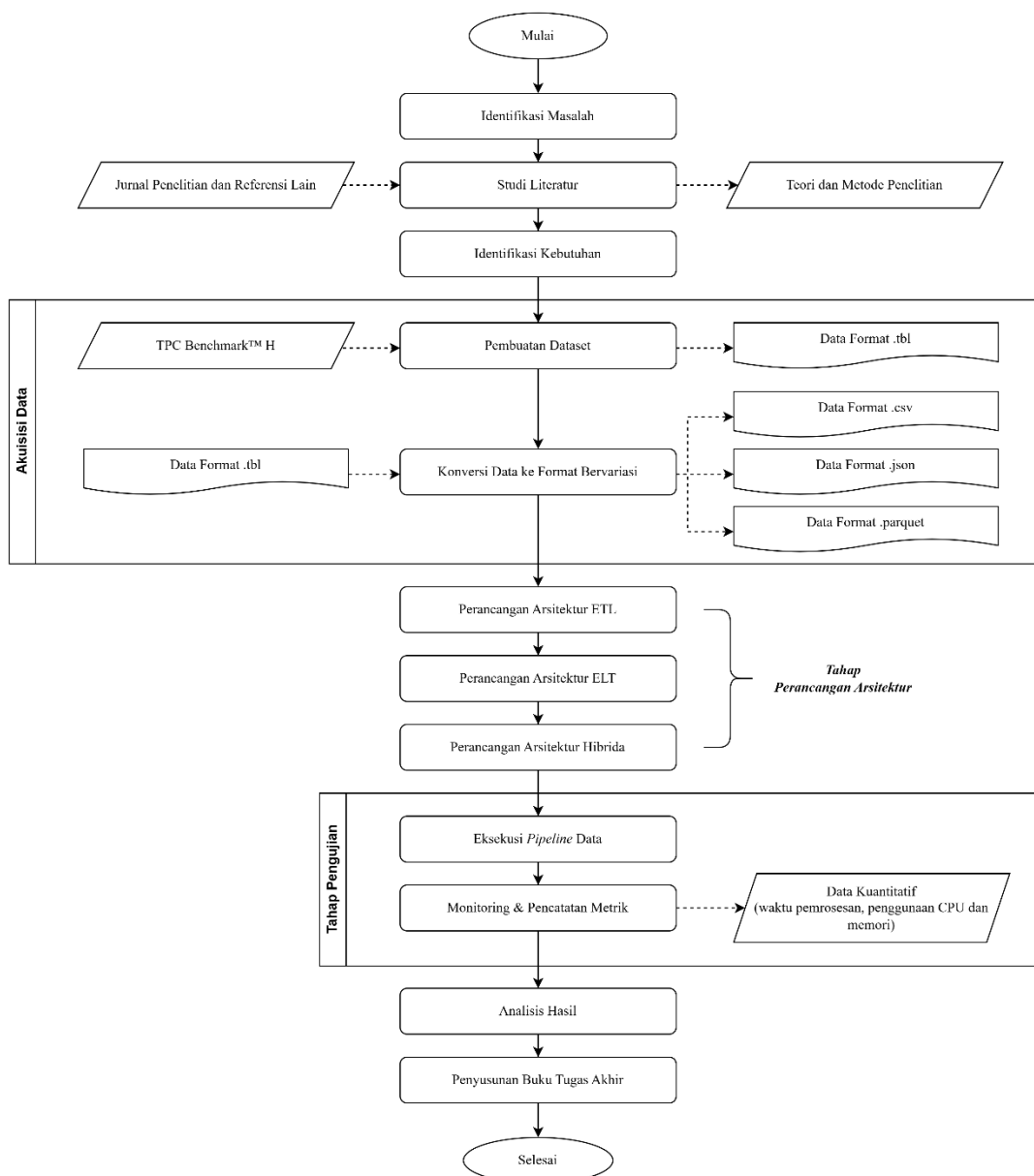
Dalam buku TPC Benchmark H Standard Specification Book (1993), disebutkan bahwa terdapat 22 *query* standar yang disediakan oleh TPC. *Query* 1 dan 6 berfokus pada agregasi data pada jumlah yang besar, yang menguji kecepatan pemrosesan CPU dan efisiensi pemindaian tabel. *Query* 2, 9, dan 14 melibatkan operasi *join* yang kompleks antara tabel fakta dan beberapa tabel dimensi, yang akan menguji efisiensi strategi *join*. *Query* 3, 4, dan 13 menguji kemampuan sistem dalam mengelola pengurutan (*sorting*) dan pengelompokan (*grouping*) data hasil *join*. *Query* 5 dan 10 melibatkan *join* beberapa tabel dan filter. *Query* 17, 20, dan 21 dirancang untuk menjadi sangat sulit dioptimasi.

BAB 3 METODOLOGI

Bagian ini berisi mengenai terkait diagram metodologi beserta penjelasan masing-masing tahapan metodologi yang digunakan pada tugas akhir ini. Selain itu, juga akan dipaparkan mengenai urutan pelaksanaan penelitian.

3.1 Uraian Metodologi

Berdasarkan studi literatur yang telah dilakukan, bagian ini menjelaskan mengenai metodologi yang digunakan dalam pengerjaan penelitian tugas akhir ini. Alur pengerjaan tugas akhir ini mencakup langkah-langkah pada diagram di bawah ini:



Gambar 3.1 Diagram Alir Metodologi Tugas Akhir

Berdasarkan diagram alir pada gambar 3.1, pengerjaan tugas akhir ini dibagi menjadi beberapa tahapan. Masing-masing tahap akan dijabarkan secara lebih detail pada subbab selanjutnya.

3.1.1 Identifikasi Masalah

Penelitian tugas akhir diawali dengan mengidentifikasi permasalahan. Pada tahap awal ini, dilakukan identifikasi dari topik yang dibahas pada tugas akhir ini untuk merumuskan masalah, menentukan tujuan penelitian, dan batasan masalah yang akan digunakan pada tugas akhir ini. Berdasarkan identifikasi kesenjangan pada penelitian terdahulu, di mana jumlah penelitian yang melakukan perbandingan implementasi praktis antara arsitektur ETL dan ELT, dan bahkan yang dengan implementasi pendekatan hibrida, masih minim. Oleh karena itu, dirumuskan tujuan untuk menganalisis dan melakukan analisis komparatif antara arsitektur tersebut secara kuantitatif.

3.1.2 Studi Literatur

Tahap ini dilakukan untuk mengetahui bagaimana implementasi masing-masing arsitektur ETL, ELT, maupun hibrida yang sudah ada. Studi literatur ini dilakukan dengan mengumpulkan, menganalisis, dan mempelajari hasil penelitian terdahulu yang relevan dengan topik yang dibahas pada tugas akhir ini, serta memperdalam pemahaman teoritis dari buku-buku terkait. Hasil temuan dari studi literatur ini dapat memperkuat latar belakang, memberi dasar dalam penentuan rumusan masalah, batasan masalah, tujuan, dan manfaat dari penelitian tugas akhir ini.

3.1.3 Identifikasi Kebutuhan

Pada tahap ini, dilakukan identifikasi kebutuhan sistem dan data secara mendalam untuk memastikan seluruh proses penelitian dan pengujian berjalan sesuai dengan tujuan yang telah ditetapkan sebelumnya. Sistem pengujian akan menggunakan PostgreSQL sebagai sistem target, dan beberapa perangkat lunak untuk membentuk masing-masing arsitektur *pipeline*.

1. Arsitektur ETL akan menggunakan Python untuk keseluruhan tahapannya, yang mencakup ekstraksi, transformasi, dan pemuatan data. *Library* yang akan digunakan adalah *PySpark*, yang dapat mendukung ekstraksi dari sistem sumber ke *DataFrame*, transformasi data, serta pemuatan ke sistem target.
2. Arsitektur ELT akan menggunakan Python untuk tahap ekstraksi dan pemuatan data, dan *dbt* untuk tahap transformasi data di dalam sistem target.
3. Arsitektur hibrida akan menggunakan Python untuk tahap ekstraksi data, pembersihan data secara ringan, dan pemuatan data. Tahap transformasi akan dilakukan dengan *dbt* di dalam sistem target.

Data yang digunakan pada tugas akhir ini adalah data yang dikembangkan dengan alat TPC-H DBGEN (dengan format awal *.tbl*). Pemilihan *benchmark* TPC-H didasarkan oleh karakteristik skemanya yang merepresentasikan basis data relasional dari sistem operasional mendekati format normalisasi ketiga (3NF). Karakteristik ini mendukung tujuan penelitian tugas akhir untuk melakukan operasi transformasi yang berat dan kompleks, seperti penggabungan (*join*) banyak tabel dan agregasi, untuk mengubah data mentah menjadi bentuk dimensional yang siap dianalisis. Kondisi ini sangat ideal untuk memberikan beban kerja yang maksimal dan mengukur efisiensi sumber daya (waktu, CPU, maupun memori) pada tahap transformasi dalam arsitektur ETL, ELT, dan hibrida. Pendekatan ini berbeda dengan *benchmark* lain seperti TPC-DS yang datanya cenderung untuk menguji performa kueri analitis akhir dibandingkan menguji ketahanan sebuah *pipeline* data.

Untuk menguji skalabilitas dan performa arsitektur pada beban kerja yang berbeda, dataset akan dikembangkan dengan dua variasi ukuran berdasarkan parameter Scale Factor (SF), yaitu SF 10 (sekitar 10 *Gigabytes*) dan SF 50 (sekitar 50 *Gigabytes*). Data tersebut kemudian dikonversi terlebih dahulu menjadi format CSV, JSON, dan Parquet untuk mengetahui dampak keragaman data (terstruktur dan semi-terstruktur) terhadap performa masing-masing arsitektur.

Untuk memastikan pengujian kinerja arsitektur berjalan secara lancar dan menghindari *bottleneck*, perangkat keras yang digunakan juga dipastikan dapat menangani keseluruhan proses dengan data berskala besar serta pengujian performa secara komprehensif. Spesifikasi perangkat keras yang digunakan adalah sebagai berikut:

Tabel 3.1 Spesifikasi Lingkungan Pengujian

Model	Dell Latitude 5480
Prosesor (CPU)	Intel(R) Core(TM) i5-7300U 2.60GHz
Memori (RAM)	8 GB
Penyimpanan	Solid State Drive (SSD) 256 GB
Sistem Operasi	Windows 10 Pro

3.1.4 Akuisisi Data

Pada tahap ini, yang dilakukan adalah akuisisi/pengumpulan data. Data yang digunakan adalah data *benchmark* TPC-H, sebuah data simulasi berstandar industri yang dapat diatur sesuai volume data yang diinginkan.

3.1.4.1 Pembuatan Data

Pembuatan data dilakukan dengan program DBGEN yang merupakan generator data pada *benchmark* TPC-H. Program ini dapat digunakan untuk menghasilkan data berstandar industri dengan volume atau *scale factor* (SF) yang diinginkan. Untuk mengukur skalabilitas masing-masing arsitektur, data awal yang akan digunakan untuk tugas akhir ini akan memiliki ukuran 50 GB, sehingga *Scale Factor* (SF) yang digunakan adalah SF = 50.

Tabel 3.2 Perkiraan Ukuran Basis Data TPC-H dengan SF = 50

Nama Tabel	Kardinalitas (dalam satuan baris)	Panjang baris (dalam bytes)	Ukuran tabel (dalam MB)
SUPPLIER	500000	7950	100
PART	10000000	7750	1500
PARTSUPP	40000000	7200	5500
CUSTOMER	7500000	8950	1300
ORDERS	75000000	5200	7450
LINEITEM	300060750	5600	32050
NATION	25	128	< 1
REGION	5	124	< 1
Total	433060780		47900

3.1.4.2 Konversi Data ke Format yang Dibutuhkan

Data mentah awal yang dihasilkan dari generator TPC-H (DBGEN) berformat *.tbl* dengan pemisah karakter berupa *pipe* (“|”). Agar dapat digunakan untuk menguji kinerja arsitektur

dengan berbagai karakteristik data, data mentah tersebut akan dikonversi menjadi tiga format, yaitu format CSV, JSON, dan Parquet.

Proses konversi ini akan dilakukan menggunakan skrip *python* dengan memanfaatkan *library DuckDB*. Skrip ini akan membaca masing-masing file *.tbl*, memetakan skema dan tipe data yang sesuai, dan menyimpannya kembali ke dalam tiga format target sebagai berikut:

1. Format CSV (*Comma-Separated Values*) dengan pemisah koma dan tidak terkompresi
2. Format JSON (*JavaScript Object Notation*), lebih tepatnya JSONL (*Newline Delimited JSON*), di mana setiap baris dalam file JSON merepresentasikan satu objek JSON. Ukuran yang terdapat dalam file ini akan lebih besar dari format yang lain, karena bersifat tidak terstruktur.
3. Format Apache Parquet, dengan menggunakan kompresi *Snappy* standar. Format ini merepresentasikan standar penyimpanan data modern yang bersifat kolumnar (berbasis kolom) dan biner, yang dirancang untuk efisiensi dalam analisis skala besar.

Seluruh file tersebut akan disimpan dalam direktori penyimpanan lokal yang terstruktur berdasarkan masing-masing format file untuk memudahkan akses pada saat pengujian berlangsung.

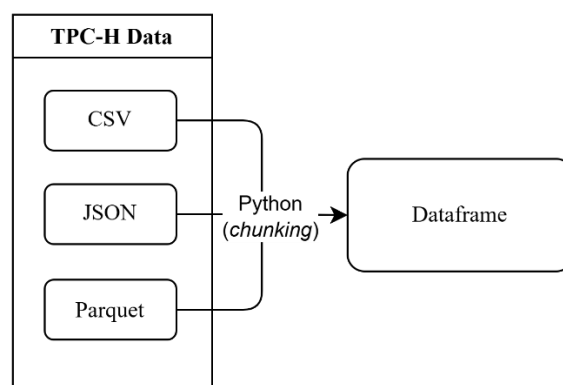
3.1.5 Perancangan Arsitektur ETL

Pada arsitektur *Extract, Transform, Load* (ETL) murni, proses transformasi data dilakukan di luar *data warehouse*, sebelum tahap pemuatan data ke dalam sistem target. Pada tugas akhir ini, arsitektur ETL akan diterapkan menggunakan bahasa pemrograman Python dengan *library PySpark* untuk melakukan pemrosesannya.

Alur kerja untuk arsitektur ETL yang akan diimplementasikan pada tugas akhir ini adalah sebagai berikut:

1. Ekstraksi (*extraction*)

Ekstraksi merupakan langkah pertama dalam ETL. Berikut merupakan gambaran yang akan dilakukan pada tahap ini:



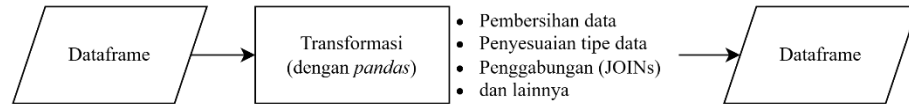
Gambar 3.2 Proses Ekstraksi pada ETL

Langkah pertama adalah membaca data sumber TPC-H (yang sudah dikonversi menjadi format CSV, JSON, dan Parquet) dari penyimpanan lokal. Pembacaan data dilakukan dengan skrip Python secara bertahap (*chunking*) untuk mengelola penggunaan memori

(RAM) pada perangkat lokal. Data yang telah diekstrak akan diubah menjadi bentuk *dataframe*.

2. Transformasi (*transformation*)

Data yang telah diekstrak dan diubah menjadi *dataframe* kemudian diproses untuk transformasi.

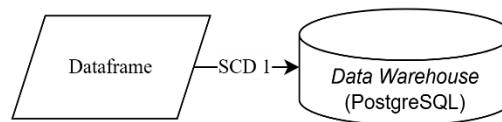


Gambar 3.3 Proses Transformasi pada ETL

Tahap transformasi mencakup pembersihan data (penanganan data duplikat dan *null*), penyesuaian tipe data, operasi penggabungan, dan transformasi lainnya untuk membentuk tabel dimensi dan fakta dalam *data warehouse*. Transformasi *dataframe* juga dilakukan menggunakan *library PySpark*.

3. Pemuatan (*loading*)

Setelah data selesai ditransformasi menjadi bentuk akhir (misalnya tabel fakta), data tersebut kemudian dimuat ke dalam tabel di sistem target PostgreSQL.



Gambar 3.4 Proses Pemuatan pada ETL

Pada tahap ini, sistem PostgreSQL hanya berfungsi sebagai tempat penyimpanan yang pasif. Pemuatan yang dilakukan pada tugas akhir ini menerapkan SCD 1 (*Update/Overwrite*).

3.1.6 Perancangan Arsitektur ELT

Pada arsitektur *Extract, Load, Transform* (ELT) murni, data mentah dimuat terlebih dahulu ke dalam sistem target, kemudian melakukan transformasi di dalam sistem target tersebut. Pada tugas akhir ini, proses ekstraksi dan pemuatan data akan dilakukan menggunakan bahasa pemrograman Python, dan transformasi di dalam mesin PostgreSQL akan dilakukan menggunakan *dbt (data build tool)* dengan adaptor *dbt-postgres*.

Alur kerja yang dilakukan dengan arsitektur ELT ini adalah sebagai berikut:

1. Ekstraksi (*extraction*)

Skrip Python akan membaca data sumber TPC-H (yang sudah dikonversi menjadi format CSV, JSON, dan Parquet) dari penyimpanan lokal, sama seperti ekstraksi yang dilakukan pada arsitektur ETL. Pembacaan data juga akan dilakukan secara bertahap (*chunking*), untuk mengelola penggunaan memori (RAM) pada perangkat lokal.

2. Pemuatan (*loading*)

Data mentah yang sudah diekstraksi kemudian langsung dimuat ke dalam sistem PostgreSQL target, tanpa melakukan transformasi atau pembersihan apapun pada data. Tujuan utama dari tahap ini pada arsitektur ELT adalah untuk memindahkan data secepat mungkin ke sistem tujuan.

3. Transformasi (*transformation*)

Berbeda dengan ETL, transformasi berat seperti penggabungan, agregasi, dan pemfilteran dilakukan oleh mesin PostgreSQL. Dengan ini, beban komputasi bergeser dari aplikasi (Python) ke mesin sistem manajemen basis data.

3.1.7 Perancangan Arsitektur Hibrida

Arsitektur hibrida pada tugas akhir ini sebagian besar menerapkan sistem ELT, namun menambahkan tahap pembersihan data dan transformasi ringan terlebih dahulu sebelum data dimuat ke sistem target. Transformasi berat (seperti penggabungan, agregasi, dan transformasi yang kompleks) kemudian dilakukan di dalam sistem basis data.

Alur kerja untuk arsitektur hibrida yang akan diimplementasikan dalam tugas akhir ini adalah sebagai berikut:

1. Ekstraksi (*Extraction*)

Tahap ekstraksi data akan dilakukan dengan cara yang sama seperti pada ETL/ELT murni. Pembacaan data dilakukan menggunakan skrip Python dari sumber TPC-H yang sudah dikonversi menjadi tiga format penyimpanan lokal. Pembacaan data juga akan dilakukan secara bertahap (*chunking*), untuk mengelola penggunaan memori (RAM) pada perangkat lokal. Data yang telah diekstrak akan diubah menjadi bentuk *dataframe* agar dapat dilakukan proses pembersihan data di tahap selanjutnya.

2. Pembersihan (*Cleaning*)

Setelah data diekstrak dan diubah menjadi bentuk *dataframe*, dilakukan transformasi ringan yang bersifat baris-per-baris, seperti penyesuaian tipe data. Tahap ini tidak mencakup operasi berat seperti penggabungan dan agregasi yang kompleks, untuk menjaga penggunaan memori yang rendah.

3. Pemuatan (*Loading*)

Data yang telah dibersihkan dimuat ke dalam basis data PostgreSQL.

4. Transformasi (*Transformation*)

Setelah data dimuat, dilakukan transformasi berat yang biasanya digunakan untuk kebutuhan analisis, yang mencakup penggabungan dan agregasi yang kompleks serta kalkulasi metrik analitis di dalam PostgreSQL menggunakan dbt.

3.1.8 Tahap Pengujian

Pada tahap ini, akan dilakukan pengujian untuk masing-masing metode integrasi data yang bertujuan untuk mengevaluasi ketiga arsitektur yang akan diimplementasikan. Pengujian dilakukan dengan data TPC-H yang sudah dikonversi menjadi tiga format file yang berbeda. Pengujian dirancang untuk mengumpulkan data kuantitatif yang menjadi metrik perbandingan kinerja masing-masing arsitektur. Proses pengujian terdiri dari dua mekanisme, yaitu eksekusi *pipeline* integrasi data serta monitoring dan pencatatan hasil metrik pengujian.

3.1.8.1 Skenario Pengujian

Berdasarkan metrik pengujian yang melibatkan tiga arsitektur (ETL, ELT, dan hibrida) dan variasi format file yang digunakan untuk pengujian, terbentuk berbagai kombinasi eksperimen. Eksekusi *pipeline* data akan dilakukan mengikuti skenario yang dirancang untuk masing-masing kombinasi:

Tabel 3.3 Skenario Kombinasi Eksperimen

Arsitektur	Format Data
ETL	CSV
	JSON
	Parquet
ELT	CSV
	JSON
	Parquet
Hibrida	CSV
	JSON
	Parquet

Kesembilan skenario tersebut juga akan dijalankan secara berulang untuk setiap ukuran SF (10 dan 50). Selain variasi arsitektur dan format data, pengujian juga dilakukan dengan skenario variasi volume data bertingkat untuk mengukur skalabilitas sistem. Variasi volume data yang digunakan dibagi menjadi dua tingkatan beban kerja:

1. Skala kecil (SF 10, ± 10 GB) : Tabel *lineitem* berjumlah $\sim 60.000.000$ baris, dan orders berjumlah $\sim 15.000.000$ baris.
2. Skala besar (SF 50, ± 50 GB) : Tabel *lineitem* berjumlah $\sim 300.060.750$ baris, dan orders berjumlah $\sim 75.000.000$ baris.

Setiap skenario akan menghasilkan keluaran akhir berupa tabel *warehouse* final, dan dijalankan secara *cold start* untuk memastikan akurasi pengukuran sumber daya. Metrik yang diukur mencakup waktu pengujian beserta penggunaan sumber daya komputasi seperti CPU, memori, dan penyimpanan.

3.1.8.2 Definisi Beban Kerja

Dari 22 *query* standar dari *TPC-H Benchmark*, dipilih *query* 9 sebagai yang paling efektif untuk tugas akhir ini. *Query* ini menggabungkan hampir seluruh skema inti TPC-H, yaitu *lineitem*, *orders*, *part*, *supplier*, *partsupp*, dan *nation*. Terdapat *query* SELECT, subquery, GROUP BY, ORDER BY, dan perhitungan matematis di dalamnya. Dengan itu, perbedaan kinerja antara ETL dan ELT dapat lebih terlihat.

3.1.8.3 Monitoring dan Pencatatan Metrik Hasil Pengujian

Setelah dilakukan pengujian, tahap selanjutnya adalah monitoring dan pencatatan metrik hasil pengujian. Pengumpulan metrik hasil pengujian akan dilakukan dengan mengukur waktu pemrosesan dan beban komputasi dari dua sisi. Pada sisi aplikasi, metrik akan didapatkan dengan memanfaatkan *log* yang dihasilkan dari Python dan dbt. Metrik pada sisi server basis data akan dihitung dari *log* yang dihasilkan dari mesin PostgreSQL saat dilakukan proses pengujian. Metrik yang akan dimonitor selama proses pengujian mencakup:

1. Waktu Pemrosesan
Metrik ini digunakan untuk membandingkan total durasi waktu yang dibutuhkan untuk menjalankan setiap skenario *pipeline*. Waktu pemrosesan akan dihitung dengan granularitas satuan detik. Untuk ETL, waktu dihitung dari awal hingga akhir eksekusi

skrip Python. Untuk ELT, waktu dihitung dari gabungan waktu ekstraksi dan pemuatan, serta durasi eksekusi transformasi di dalam basis data (diambil dari log eksekusi)

2. Pengujian Efisiensi Sumber Daya

Metrik ini akan digunakan untuk membandingkan penggunaan sumber daya pada setiap kombinasi pengujian arsitektur integrasi data dengan masing-masing format file. Pengujian efisiensi sumber daya akan berfokus pada penggunaan CPU dan memori. Penggunaan CPU akan dihitung dengan satuan persentase CPU (%) dan memori akan dihitung dengan satuan *megabyte* (MB).

3. Penggunaan Ruang Penyimpanan

Metrik ini akan digunakan untuk mengetahui ukuran file CSV/JSON/Parquet. Pengukuran awal file tersebut dilakukan dengan alat *command line*, sedangkan pada fase *data warehouse* diukur menggunakan fungsi internal PostgreSQL untuk menghitung total ukuran tabel beserta indeksnya.

Data metrik hasil pengujian tersebut akan dikumpulkan dan disajikan dalam bentuk tabel yang terstruktur untuk diproses lebih lanjut pada tahap analisis hasil.

3.1.9 Tahap Analisis Hasil

Setelah seluruh skenario pengujian berhasil dijalankan dan data metrik hasil pengujian terkumpul, tahap selanjutnya adalah melakukan analisis komparatif yang mendalam. Proses analisis ini akan dibantu oleh visualisasi dalam bentuk grafik untuk menunjukkan total durasi pemrosesan dan penggunaan sumber daya, untuk memudahkan perbandingan kinerja antar arsitektur secara intuitif.

Proses-proses analisis yang akan dilakukan adalah sebagai berikut:

1. Analisis Perbandingan Durasi Pemrosesan

Analisis ini berfokus pada perbandingan total durasi waktu pemrosesan antar arsitektur. Data waktu eksekusi akan divisualisasikan dalam bentuk grafik, untuk menunjukkan perbedaan kinerja yang signifikan antar skenario. Analisis ini bertujuan untuk mengidentifikasi arsitektur mana yang paling cepat untuk setiap format file. Dari analisis ini juga akan didapatkan tahap mana yang paling

2. Analisis Penggunaan Sumber Daya

Analisis ini dilakukan untuk membandingkan penggunaan sumber daya antara proses aplikasi (skrip Python) dan proses basis data (PostgreSQL). Dari analisis ini, dapat diketahui titik *bottleneck* utama pada setiap arsitektur, seperti misalnya ETL mengalami *bottleneck* pada memori (RAM) karena penggunaannya oleh Python untuk transformasi.

3.1.10 Penyusunan Tugas Akhir

Tahap terakhir adalah penyusunan buku tugas akhir. Buku ini berisi seluruh aspek dari tugas akhir ini, termasuk latar belakang permasalahan, metodologi dan arsitektur yang digunakan, dokumentasi seluruh proses kegiatan dan pengujian, penjabaran temuan, hingga hasil analisis penelitian tugas akhir ini. Laporan ini menyajikan informasi tentang penelitian tugas akhir secara detail, sehingga diharapkan dapat menjadi referensi yang komprehensif dan berguna untuk penelitian atau pengembangan sistem serupa di masa depan.

3.2 Urutan Pelaksanaan Penelitian

Pelaksanaan tugas akhir ini akan dilaksanakan dalam kurun waktu 5 bulan. Timeline pelaksanaannya adalah sebagai berikut:

Tabel 3.4 Jadwal Pelaksanaan Penelitian Tugas Akhir

No	Jenis Kegiatan	Bulan / Minggu																			
		Bulan ke-1				Bulan ke-2				Bulan ke-3				Bulan ke-4				Bulan ke-5			
		I	II	III	IV	I	II	III	IV	I	II	III	IV	I	II	III	IV	I	II	III	IV
1	Studi Literatur																				
2	Identifikasi Kebutuhan																				
3	Pengumpulan Data																				
4	Perancangan Arsitektur ETL																				
5	Perancangan Arsitektur ELT																				
6	Perancangan Arsitektur Hibrida																				
7	Tahap Pengujian																				
8	Tahap Analisis Hasil																				
9	Penyusunan Tugas Akhir																				

BAB 4 HASIL DAN PEMBAHASAN

Bagian ini akan membahas secara rinci mengenai seluruh proses yang tahapannya telah dijelaskan pada bab 3. Mulai dari perancangan, implementasi, dan pengujian kinerja arsitektur integrasi data dengan *data warehouse* berbasis PostgreSQL. Masing-masing tahapan tersebut akan dijelaskan secara rinci pada setiap subbab.

4.1 Persiapan Lingkungan dan Data Eksperimen

4.1.1 Lingkungan Sistem Pengujian

Sebelum melakukan implementasi penelitian ini, dilakukan persiapan lingkungan sistem pengujian berdasarkan kebutuhan yang telah diidentifikasi sebelumnya. Hal ini untuk memastikan seluruh proses implementasi dan pengujian berjalan sesuai dengan tujuan yang telah ditetapkan sebelumnya. Kebutuhan yang diidentifikasi mencakup beberapa aspek: kebutuhan lingkungan untuk implementasi sistem, kebutuhan untuk pengukuran performa, kebutuhan data yang akan dijadikan objek penelitian, serta arsitektur sistem yang akan digunakan.

Identifikasi kebutuhan teknis pertama yaitu kebutuhan perangkat keras dan perangkat lunak yang akan digunakan untuk implementasi. Untuk mendukung proses perancangan dan implementasi perbandingan kinerja arsitektur integrasi data dengan data berskala besar, ditetapkan perangkat keras yang akan digunakan adalah sebagai berikut:

Tabel 4.1 Spesifikasi Perangkat Keras

Model	Dell Latitude 5480
Prosesor (CPU)	Intel(R) Core(TM) i5-7300U 2.60GHz
Memori (RAM)	8 GB
Penyimpanan	Solid State Drive (SSD) 256 GB
Sistem Operasi	Windows 10 Pro

Adapun perangkat lunak yang akan mendukung proses implementasi dan pengujian ini adalah sebagai berikut:

Tabel 4.2 Spesifikasi Perangkat Lunak

Relational Database Management System (RDBMS)	PostgreSQL
Sistem Operasi dan Lingkungan Virtual	Windows Subsystem for Linux (WSL) 2
Alat Persiapan Data	TPC-H DBGEN (Database Generator)
Integrasi dan Pemrosesan Data	Apache Spark (PySpark)
Transformasi di arsitektur ELT & Hibrida	dbt (Data Build Tool)

Penelitian ini akan menggunakan objek penelitian berupa dataset berskala besar, yaitu data yang dikembangkan dengan menggunakan alat TPC-H DBGEN. Ukuran data yang dikembangkan akan memiliki dua variasi, yaitu 10 *Gigabytes* (SF=10) dan 50 *Gigabytes* (SF=50).

4.1.2 Hasil Akuisisi dan Pembuatan Data

Dalam penelitian tugas akhir ini, objek yang digunakan adalah *dataset* yang dikembangkan dengan menggunakan standar *benchmarking* industri, yaitu *dataset* TPC-H. *Dataset* ini dipilih juga karena dapat dikembangkan dengan skala besar. Selain itu, *dataset* yang terstandar ini digunakan untuk memastikan penelitian yang dilakukan valid, dapat direproduksi, dan diakui secara akademis dibandingkan penggunaan data *dummy* yang bersifat sembarangan.

Proses pengembangan data dilakukan dengan menggunakan alat bawaan TPC-H yaitu DBGEN. DBGEN dioperasikan sepenuhnya melalui *command-line interface* (CLI). Alat ini memungkinkan pembuatan data dalam berbagai ukuran secara khusus melalui pengaturan parameter *Scale Factor* (SF). Pembuatan data dikonfigurasi menggunakan *Scale Factor* tertentu, yaitu SF=50 yang akan menghasilkan data sekitar 50 GB, guna menguji ketahanan dan skalabilitas arsitektur integrasi data terhadap volume data yang besar.

Pembuatan data diawali dengan melakukan kompilasi terhadap kode sumber TPC-H pada lingkungan Ubuntu 24.04.1 LTS yang dijalankan di dalam Windows Subsystem for Linux (WSL) 2 hingga menghasilkan *executable file*. Penggunaan lingkungan WSL 2 dipilih karena menggunakan kernel Linux asli yang mendukung eksekusi skrip Bash secara *native*, yang sangat krusial dalam proses integrasi data berskala besar. Data kemudian dikembangkan dengan mengeksekusi perintah “./dbgen -s 50”. Penggunaan parameter -s (*scale factor*) ini bertujuan untuk menentukan total volume data yang dihasilkan, yang dalam penelitian tugas akhir ini ditetapkan sebesar 50 GB untuk mensimulasikan beban data berskala besar.

Proses eksekusi DBGEN menghasilkan delapan tabel utama dengan format bawaan *.tbl*, yaitu berkas teks yang memiliki pemisah berupa karakter *pipe* (“|”). Tabel-tabel tersebut mencakup data-data kebutuhan bisnis seperti *supplier*, *part*, *partsupp*, *customer*, *lineitem*, *orders*, *nation*, dan *region*. Skema masing-masing tabel yang dihasilkan akan dijelaskan lebih rinci pada tabel-tabel di bawah.

Tabel 4.3 Skema tabel ‘supplier’

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	s_suppkey	INTEGER		NO	Primary key
2	s_name	VARCHAR	25	NO	
3	s_address	VARCHAR	40	NO	
4	s_nationkey	INTEGER		NO	Foreign key ke ‘n_nationkey’
5	s_phone	VARCHAR	15	NO	
6	s_acctbal	NUMERIC	12, 2	NO	
7	s_comment	VARCHAR		NO	

Tabel *supplier* berisi 500.000 baris data (untuk SF = 50) yang terbagi menjadi 7 kolom. Kolom *s_suppkey* menjadi *primary key*, sementara *s_nationkey* berfungsi sebagai *foreign key* yang menghubungkan tabel ini dengan tabel *nation*. Seluruh kolom dalam tabel ini memiliki batasan *NOT NULL* sesuai dengan spesifikasi TPC-H, untuk menjamin integritas data selama proses pengujian performa berlangsung.

Tabel 4.4 Skema tabel 'part'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	p_partkey	INTEGER		NO	Primary key
2	p_name	VARCHAR	55	NO	
3	p_mfgr	VARCHAR	25	NO	
4	p_brand	VARCHAR	10	NO	
5	p_type	VARCHAR	25	NO	
6	p_size	INTEGER		NO	
7	p_container	VARCHAR	10	NO	
8	p_retailprice	NUMERIC	12, 2	NO	
9	p_comment	VARCHAR	231	NO	

Kemudian terdapat tabel *part* yang berisi 10.000.000 baris data. Tabel ini terdiri atas 9 kolom, dengan *p_partkey* berperan sebagai primary key. Seperti pada tabel lainnya, seluruh kolom dalam tabel *part* ini memiliki batasan NOT NULL mengikuti spesifikasi TPC-H, sehingga menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.5 Skema tabel 'partsupp'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	ps_partkey	INTEGER		NO	Foreign key ke 'p_partkey'
2	ps_suppkey	INTEGER		NO	Foreign key ke 's_suppkey'
3	ps_availqty	INTEGER		NO	
4	ps_supplycost	NUMERIC	12, 2	NO	
5	ps_comment	VARCHAR	199	NO	

Selanjutnya adalah tabel *partsupp* yang merupakan tabel jembatan antara tabel *part* dan tabel *supplier*. Tabel ini berisi 40.000.000 baris data, yang terbagi menjadi 5 kolom. Kolom *ps_partkey* berperan sebagai *foreign key* yang menghubungkannya dengan tabel *part*, dan *ps_suppkey* yang merupakan *foreign key* untuk menghubungkannya dengan tabel *supplier*. Seperti pada tabel lainnya, seluruh kolom dalam tabel *partsupp* ini memiliki batasan NOT NULL mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.6 Skema tabel 'customer'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	c_custkey	INTEGER		NO	Primary Key
2	c_name	VARCHAR	25	NO	
3	c_address	VARCHAR	40	NO	
4	c_nationkey	INTEGER		NO	Foreign key ke 'n_nationkey'
5	c_phone	VARCHAR	15	NO	
6	c_acctbal	NUMERIC	12, 2	NO	
7	c_mktsegment	VARCHAR	10	NO	
8	c_comment	VARCHAR	117	NO	

Tabel *customer* berisi 7.500.000 baris data, yang terbagi menjadi 8 kolom. Kolom *c_custkey* berperan sebagai *primary key*, dan kolom *c_nationkey* berperan sebagai *foreign key* untuk menghubungkan tabel ini dengan tabel *nation*. Seperti pada tabel lainnya, seluruh kolom dalam tabel *customer* ini memiliki batasan *NOT NULL* mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.7 Skema tabel 'orders'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	o_orderkey	INTEGER		NO	Primary key
2	o_custkey	INTEGER		NO	Foreign key ke 'c_custkey'
3	o_orderstatus	VARCHAR	1	NO	
4	o_totalprice	NUMERIC	12, 2	NO	
5	o_orderdate	DATETIME		NO	
6	o_orderpriority	VARCHAR	15	NO	
7	o_clerk	VARCHAR	15	NO	
8	o_shippriority	INTEGER		NO	
9	o_comment	VARCHAR	79	NO	

Tabel *orders* berisi 75.000.000 baris data yang terbagi menjadi 9 kolom. Kolom *o_orderkey* berperan sebagai *primary key*, kolom *o_custkey* berperan sebagai *foreign key* untuk menghubungkan tabel ini ke tabel *customer*. Seperti pada tabel lainnya, seluruh kolom dalam tabel *orders* ini memiliki batasan *NOT NULL* mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.8 Skema tabel 'lineitem'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	l_orderkey	INTEGER		NO	Composite Primary key dan Foreign key ke 'o_orderkey'
2	l_partkey	INTEGER		NO	Foreign key ke 'p_partkey', compound Foreign key ke (ps_partkey, ps_suppkey) dengan l_suppkey
3	l_suppkey	INTEGER		NO	Foreign key ke s_suppkey, compound Foreign key ke (ps_partkey, ps_suppkey) dengan l_partkey

4	<i>l_linenumber</i>	INTEGER		NO	Composite Primary key
5	<i>l_quantity</i>	INTEGER		NO	
6	<i>l_extendedprice</i>	NUMERIC	12, 2	NO	
7	<i>l_discount</i>	NUMERIC	12, 2	NO	
8	<i>l_tax</i>	NUMERIC	12, 2	NO	
9	<i>l_returnflag</i>	VARCHAR	1	NO	
10	<i>l_linestatus</i>	VARCHAR	1	NO	
11	<i>l_shipdate</i>	DATETIME		NO	
12	<i>l_commitdate</i>	DATETIME		NO	
13	<i>l_receiptdate</i>	DATETIME		NO	
14	<i>l_shipinstruct</i>	VARCHAR	25	NO	
15	<i>l_shipmode</i>	VARCHAR	10	NO	
16	<i>l_comment</i>	VARCHAR	44	NO	

Tabel *lineitem* berisi 300.060.750 baris data yang terbagi menjadi 16 kolom. Tabel ini merupakan tabel dengan ukuran terbesar dalam *dataset* ini. Tabel ini memiliki *composite primary key* (kunci utama gabungan) yang terdiri dari kolom *l_orderkey* dan *l_linenumber*. Kolom *l_orderkey* berperan sebagai bagian dari *primary key* sekaligus sebagai *foreign key* yang menghubungkan tabel ini dengan tabel *orders*. Kolom *l_partkey* berfungsi sebagai *foreign key* yang menghubungkannya ke tabel *part*. Selain itu, tabel ini memiliki *compound foreign key* yang mengaitkan kolom *l_partkey* dan *l_suppkey* secara bersamaan ke tabel *partsupp* (pada kolom *ps_partkey* dan *ps_suppkey*). Seperti pada tabel lainnya, seluruh kolom dalam tabel *lineitem* ini memiliki batasan *NOT NULL* mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.9 Skema tabel 'nation'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	<i>n_nationkey</i>	INTEGER			Primary key
2	<i>n_name</i>	VARCHAR	25		
3	<i>n_regionkey</i>	INTEGER			Foreign key ke <i>r_regionkey</i>
4	<i>n_comment</i>	VARCHAR	152		

Tabel *nation* berisi jumlah baris data yang konsisten, yaitu 25 baris data yang terbagi menjadi 4 kolom. Kolom *n_nationkey* berperan sebagai *primary key*, dan kolom *n_regionkey* berperan sebagai *foreign key* untuk menghubungkan tabel ini ke tabel *region*. Seperti pada tabel lainnya, seluruh kolom dalam tabel *nation* ini memiliki batasan *NOT NULL* mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Tabel 4.10 Skema tabel 'region'

No.	Column_Name	Data_Type	Max_Length	Is_Nullable	Note
1	<i>r_regionkey</i>	INTEGER			Primary key
2	<i>r_name</i>	VARCHAR	25		
3	<i>r_comment</i>	VARCHAR	152		

Seperti tabel *nation*, tabel *region* juga berisi jumlah baris data yang konsisten, yaitu 5 baris data yang terbagi menjadi 3 kolom. Terdapat kolom *r_regionkey* yang berperan sebagai *primary key*. Seperti pada tabel lainnya, seluruh kolom dalam tabel *region* ini memiliki batasan *NOT NULL* mengikuti standar TPC-H yang ada, untuk menjamin integritas data selama proses pengujian berlangsung.

Secara keseluruhan, *dataset* yang dibuat memiliki total baris data yang sangat masif guna mensimulasikan skenario beban kerja yang intensif. Seluruh data mentah yang dihasilkan dengan ukuran sekitar 50 GB tersebut kemudian disimpan di dalam direktori *dataset/tbl/* dengan format karakter pemisah *pipe* (“|”). Karena format ini tidak umum digunakan di beberapa tools *modern*, maka diperlukan tahap konversi ke format data yang lebih relevan, yang akan dibahas pada tahap-tahap selanjutnya.

4.1.3 Proses Konversi Format Data

Dalam penelitian tugas akhir ini, untuk mengevaluasi kinerja arsitektur sistem dalam menangani karakteristik format penyimpanan data yang berbeda (*variety*), digunakan data dengan format yang bervariasi. Data yang dibutuhkan penelitian ini memiliki tiga jenis format, yaitu CSV, JSON, dan Parquet. Sedangkan, data mentah yang dihasilkan oleh alat DBGEN secara bawaan masih berformat *.tbl*, sehingga perlu dilakukan konversi format data sebelum memasuki fase ekstraksi.

Skrip automasi dibuat menggunakan bahasa pemrograman Python (yang terletak pada *dataset/convert.py*), yang berfungsi untuk membaca file mentah berformat *.tbl* dan mengubahnya menjadi tiga format target utama:

1. Format CSV (*Comma-separated values*): Format ini dipilih sebagai representasi penyimpanan berbasis baris konvensional. CSV merupakan standar format pertukaran data yang paling umum, namun biasanya membutuhkan parsing I/O yang lebih lama untuk memuat data bervolume besar.
2. Format JSONL (JSON Lines): Format ini dipilih untuk merepresentasikan data semi-terstruktur. Format ini digunakan untuk menguji seberapa fleksibel *pipeline* Python (untuk arsitektur ETL) dan dbt (untuk arsitektur ELT) dalam melakukan *parsing* struktur data *key-value* berskala besar.
3. Format Parquet: Format ini merupakan standar penyimpanan data berorientasi kolom (*columnar storage*) yang sering digunakan dalam ekosistem *Big Data*. Parquet dipilih untuk menguji metrik *Velocity* secara spesifik, karena format ini memiliki tingkat kompresi data yang tinggi dan sangat optimal untuk operasi analitis (*query* agregasi) di *Data Warehouse*.

Secara teknis, skrip Python *convert.py* dirancang untuk memproses konversi ini secara efisien agar tidak membebani memori utama (RAM). Hasil dari proses konversi ini secara otomatis dikelompokkan ke dalam struktur folder yang sesuai, yaitu *dataset/csv/*, *dataset/parquet/*, dan *dataset/jsonl*. Seluruh *file* yang sudah masuk ke folder-folder ini sudah siap untuk ditarik dan diproses oleh skrip masing-masing arsitektur pada tahapan implementasi integrasi data selanjutnya.

4.2 Implementasi Arsitektur Integrasi Data

4.2.1 Arsitektur ETL

Proses *Extract, Transform, Load* (ETL) merupakan salah satu inti dari implementasi teknis dalam penelitian tugas akhir ini. Arsitektur ini dirancang sebagai *pipeline* yang bertujuan untuk menempatkan beban kerja transformasi di luar *Data Warehouse* utama, sehingga transformasi tersebut dilakukan di antara tahap ekstraksi data mentah dari sistem sumber dan tahap pemuatan data yang sudah terstandar ke dalam sistem *Data Warehouse* target. Dengan itu, beban kerja transformasi ditangani oleh Python yang bertindak sebagai server yang melakukan transformasi.

Secara konseptual, perancangan arsitektur ETL dalam penelitian tugas akhir ini digambarkan dengan skema berikut:

(skema penjelasan ETL, dari ekstraksi, misal kayak kalo CSV pakai function apa, kalo JSONL pakai function apa, kalo parquet pakai function apa, dll. jelasin juga bahwa akan pakai chunking, chunknya berapa, dll)

4.2.1.1 Fase Ekstraksi (*Extract*)

Fase pertama dalam *pipeline* ETL adalah mengekstrak data dari *file* sumber. Pada implementasi penelitian tugas akhir ini, proses ekstraksi dibuat menggunakan PySpark. Fase ekstraksi ini membutuhkan *table_name* (nama tabel) dan *file_format* (format file) sebagai parameter yang digunakan. Untuk memastikan metrik *Velocity* atau kecepatan dapat dengan mudah dianalisis melalui tampilan *Spark Web UI*, penelitian ini menerapkan pelabelan *Job* pada antarmuka *SparkContext* sebelum operasi baca (*read*) dieksekusi. Berikut merupakan potongan kode fungsi *extract_data()* yang bertanggung jawab untuk proses ini:

```
1. spark.sparkContext.setJobGroup("ETL_Extract", f"ETL Extract Phase:
   {table_name}")
2. spark.sparkContext.setJobDescription(f"[ETL] Extract '{table_name}'
   from {file_format}")
```

Kode 4.1 Konfigurasi Spark Job Group pada Arsitektur ETL

Setelah tahapan inisialisasi eksekusi dan pelabelan metrik dikonfigurasi, *pipeline* ETL ini dilanjutkan dengan tahap ekstraksi data mentah. Mengingat pengujian ini dirancang untuk mengevaluasi kinerja arsitektur terhadap karakteristik *Variety*, mekanisme ekstraksi harus dibuat secara modular agar mampu membaca berbagai format penyimpanan data tanpa perlu menuliskan ulang modul pembacaannya. Untuk mengakomodasi hal tersebut, implementasi ekstraksi data memanfaatkan pernyataan *if-else* yang menyesuaikan tipe ekstensi berkas yang sedang diproses. Implementasi logika ekstraksi tersebut ditunjukkan pada potongan kode berikut:

```
1. if file_format == "csv":
2.     df = spark.read \
3.         .format(file_format) \
4.         .option("header", "True") \
5.         .load(f"{DATASET_DIR}/{file_format}/{table_name}.{file_format}")
```

Potongan kode tersebut merupakan pemanfaatan *DataframeReader* bawaan Apache Spark untuk memuat data dari lapisan penyimpanan ke dalam struktur *in-memory DataFrame*. Metode *.load()* digunakan untuk menyatakan struktur lokasi file yang akan dibaca. Pendekatan kondisional *if* digunakan secara spesifik untuk mengontrol format yang sedang dibaca, seperti misalnya *csv*, *jsonl*, atau *parquet*. Karena format CSV berwujud teks datar (*flat-text*), format file tersebut membutuhkan penanganan yang berbeda dengan format file lainnya yang memiliki karakteristik *self-describing* seperti JSON dan Parquet yang secara otomatis mendeteksi struktur skema. Pembacaan format file CSV membutuhkan deklarasi *header* eksplisit, untuk memastikan bahwa PySpark membaca baris pertama pada data CSV sebagai *metadata* kolom skema, bukan menganggapnya sebagai baris data isi tabel.

Di akhir tahapan ekstraksi pada arsitektur ETL ini diberi penanganan error, untuk menangani kondisi jika format file yang masuk sebagai parameter di luar cakupan penelitian tugas akhir ini.

```
1.  else:
2.      raise ValueError(f"Format {file_format} tidak sesuai dengan
3.      pipeline ini.")
```

Kode 4.3 Implementasi Penanganan Kesalahan Ekstraksi File pada ETL

4.2.1.2 Fase Transformasi (*Transform*)

Data yang berhasil diekstrak kemudian dilakukan transformasi. Fase ini merupakan inti dari arsitektur ETL, di mana beban komputasi paling berat terjadi. Dalam arsitektur ini, transformasi dilakukan sepenuhnya di luar basis data target, yang dalam penelitian ini menggunakan *engine* pemrosesan *in-memory* terdistribusi. Pada fase ini, logika bisnis yang diterapkan merujuk pada spesifikasi TPC-H Query 9 (*Product Type Profit Measure*).

Implementasi transformasi data ini diawali dengan inisiasi enam tabel yang akan digunakan dalam transformasi ini.

```
1.  df_nation    = df["nation"]
2.  df_part      = df["part"]
3.  df_supplier  = df["supplier"]
4.  df_partsupp  = df["partsupp"]
5.  df_orders    = df["orders"]
6.  df_lineitem  = df["lineitem"]
```

Setelah tabel yang relevan sudah didefinisikan, dilakukan transformasi tabel-tabel tersebut sesuai dengan logika bisnis yang diterapkan sesuai standar spesifikasi *TPC-H Query 9*, yang potongan kodenya sebagai berikut:

```

1.  # FILTER: hanya yang contains 'green' di p_name
2.  df_part_filtered =
    df_part.filter(F.lower(F.col("p_name")).contains("green"))
3.
4.  # JOIN: supplier dan nation (s_nationkey = n_nationkey)
5.  df_supplier_nation = df_supplier.join(
6.      df_nation,
7.      df_supplier["s_nationkey"] == df_nation["n_nationkey"],
8.      "inner"
9.  )
10.
11. # JOIN: lineitem dan filtered part table (l_partkey = p_partkey)
12. df_joined = df_lineitem.join(
13.     df_part_filtered,
14.     df_lineitem["l_partkey"] == df_part_filtered["p_partkey"],
15.     "inner"
16. )
17.
18. # JOIN: df dan supplier+nation (l_suppkey = s_suppkey)
19. df_joined = df_joined.join(
20.     df_supplier_nation,
21.     df_joined["l_suppkey"] == df_supplier_nation["s_suppkey"],
22.     "inner"
23. )
24.
25. # JOIN: df dan partsupp (l_suppkey = ps_suppkey AND l_partkey =
    ps_partkey)
26. df_joined = df_joined.join(
27.     df_partsupp,
28.     (df_joined["l_suppkey"] == df_partsupp["ps_suppkey"]) &
29.     (df_joined["l_partkey"] == df_partsupp["ps_partkey"]),
30.     "inner"
31. )
32.
33. # JOIN: df dan orders (l_orderkey = o_orderkey)
34. df_joined = df_joined.join(
35.     df_orders,
36.     df_joined["l_orderkey"] == df_orders["o_orderkey"],
37.     "inner"
38. )
39.
40. # Calculate o_year dan amount
41. df_profit = df_joined.select(

```

```

42.     F.col("n_name").alias("nation"),
43.     F.year(F.col("o_orderdate").cast("date")).alias("o_year"),
44.     (
45.         (F.col("l_extendedprice").cast("double") * (1 -
46.             F.col("l_discount").cast("double")))
47.         - (F.col("ps_supplycost").cast("double") *
48.             F.col("l_quantity").cast("double"))
49.     ).alias("amount")
50. )

# GROUP BY: nation, o_year dan hitung sum(amount) as sum_profit
df_result = df_profit.groupBy("nation", "o_year") \
    .agg(F.sum("amount").alias("sum_profit"))

# ORDER BY: nation ASC, o_year DESC
df_result = df_result.orderBy(
    F.col("nation").asc(),
    F.col("o_year").desc()
)

```

Kode 4.5 Implementasi Transformasi TPC-H Query 9 pada Arsitektur ETL

Secara arsitektural, proses transformasi ini diawali dengan penerapan *filter* pada tabel *part* menggunakan fungsi *lower()* dan *contains()* untuk menyaring baris-baris yang memiliki *substring* “green” pada kolom nama produk. Langkah ini ditempatkan di awal fase transformasi untuk mengikuti prinsip *predicate pushdown*, yang merupakan teknik optimisasi kueri yang menempatkan operasi *filter* dekat dengan sumber data nya, yang pada konteks penelitian tugas akhir ini berada di awal fase. Dengan itu, volume data dapat direduksi sedini mungkin sebelum memasuki operasi penggabungan (*join*) yang berat, sehingga meningkatkan efisiensi memori dan kecepatan pemrosesan. Selanjutnya, dilakukan operasi *INNER JOIN* yang secara sistematis menggabungkan enam tabel fakta dan dimensi. Urutan penggabungan ini juga dipertimbangkan, yaitu dimulai dengan tabel-tabel yang berukuran lebih kecil terlebih dahulu (seperti *supplier* dan *nation*) sebelum kemudian disatukan dengan tabel fakta utama *lineitem*, agar ukuran partisi data selama *shuffling* dalam memori dapat dijaga agar lebih stabil.

Penerjemahan kueri SQL pada spesifikasi TPC-H ke dalam sintaks Python (PySpark) dilakukan dengan menerjemahkan blok *SELECT* dan *WHERE* menjadi metode *.filter()* dan *.select()*, sementara relasi antartabel dipetakan menggunakan metode *.join()*. Pada tahap kalkulasi, dilakukan proyeksi data menggunakan aritmetika untuk menghitung nilai keuntungan bersih setelah sebelumnya dilakukan penyesuaian tipe data. Proses tersebut diakhiri dengan agregasi profit per negara dan tahun menggunakan *.groupBy()* dan *.agg()*, serta diurutkan menggunakan *.orderBy()* untuk menghasilkan dataset akhir yang terstruktur dan siap untuk dimuat ke dalam basis data target.

Namun, karena Apache Spark menerapkan prinsip *Lazy Evaluation*, maka pada tahap ini Spark hanya membangun perencanaan atau grafik instruksi, dan baru akan mengeksekusi beberapa operasi transformasi ketika *action* seperti penulisan JDBC dijalankan. Oleh karena itu, penelitian ini menggunakan waktu pemrosesan total pipeline sebagai metrik utama agar

perbandingan antar arsitektur tetap konsisten. Pengamatan per fase melalui Spark Web UI digunakan sebagai informasi pendukung, bukan sebagai dasar utama perbandingan.

4.2.1.3 Fase Pemuatan (*Load*)

Setelah seluruh rangkaian transformasi selesai dibuat perencanaannya di dalam lingkungan *in-memory* Apache Spark, tahap akhir dari alur kerja ETL adalah pemuatan (*load*) yang akan memindahkan hasil kalkulasi tersebut ke dalam basis data target. Berbeda dengan fase transformasi yang membebani pada CPU dan RAM, fase pemuatan ini lebih bergantung pada kecepatan konektivitas jaringan dan *throughput* penulisan pada *disk* basis data target. Pada eksperimen ini, pemuatan data dilakukan menggunakan konektor JDBC (*Java Database Connectivity*) yang mengintegrasikan ekosistem Spark dengan PostgreSQL secara langsung. Penelitian ini menggunakan mode penulisan *overwrite*, untuk menjaga konsistensi dan mencegah redundansi data. Hal ini memastikan bahwa setiap iterasi pengujian dimulai dengan kondisi tabel yang bersih, sehingga metrik yang dicatat bersifat murni dan tidak terpengaruh oleh data dari iterasi sebelumnya. Implementasi teknis dari proses pemuatan ini dapat dijabarkan pada kode berikut:

```
1. spark.sparkContext.setJobGroup("ETL_Load", "ETL Load Phase:
   result_etl.etl_query9")
2. spark.sparkContext.setJobDescription("[ETL] Load transformed data
   into result_etl.etl_query9")
3. print("\n=== LOADING ===")
4. print("Starting the loading phase...\nLoading into
   'result_etl.etl_query9' table...")
5. try:
6.     df.write \
7.         .jdbc(
8.             url=DB_URL,
9.             table="result_etl.etl_query9",
10.            mode="overwrite",
11.            properties=DB_PROPERTIES
12.        )
13.     print("Table 'result_etl.etl_query9' loaded successfully.")
14. except Exception as e:
15.     print(f"Error loading data: {e}")
```

Kode 4.6 Implementasi Pemuatan Data ke PostgreSQL pada Arsitektur ETL

Kode tersebut menunjukkan bahwa proses pemuatan data ke dalam basis data target dilakukan dengan memanfaatkan konektor JDBC. Namun, arsitektur Apache Spark menerapkan prinsip *Lazy Evaluation*, yang menyebabkan proses transformasi yang telah didefinisikan pada tahap sebelumnya tidak langsung dieksekusi secara instan, tetapi saat sebuah *Action* dipanggil, yang dalam hal ini adalah *df.write.jdbc()*. Oleh karena itu, metrik yang tercatat pada halaman *Stages* di tampilan antarmuka *Spark Web UI* akan menampilkan bahwa fase pemuatan di arsitektur ETL ini mencakup akumulasi waktu dari eksekusi transformasi serta waktu perpindahan data ke PostgreSQL. Sehingga, pada titik inilah seluruh beban kerja komputasi data berskala 50 GB dieksekusi dan dipindahkan ke penyimpanan permanen.

4.2.1.4 Fungsi Utama Pipeline ETL

Fungsi utama pada arsitektur ETL digunakan untuk mengatur urutan eksekusi seluruh *pipeline*, mulai dari ekstraksi data dari sumber, transformasi data, hingga pemuatan seluruh data hasil akhir ke *data warehouse* PostgreSQL. Kode implementasi fungsi utama arsitektur pipeline ETL ditunjukkan pada potongan Kode 4.7 berikut:

```
1.  if __name__ == "__main__":
2.      spark.sparkContext.setJobGroup("ETL_Pipeline", "TPC-H Query 9 - ETL Pipeline")
3.      extracted_dfs = {}
4.      for table in TABLE_NAME:
5.          extracted_dfs[table] = extract_data(table, "jsonl")
6.      transformed_data = transform_data(extracted_dfs)
7.      load_data(transformed_data)
8.      spark.sparkContext.setJobDescription("[ETL] Pipeline Complete")
9.
10.     print("ETL Process done. You can access the http://localhost:4040
        for detailed monitoring.")
11.     print("Press CTRL+C to terminate the process.")
12.     time.sleep(3600)
13.     spark.stop()
```

Kode 4.7 Implementasi Fungsi Utama Pipeline ETL

Selain berfungsi sebagai pengendali alur eksperimen agar setiap skenario dijalankan dengan urutan yang konsisten, fungsi utama juga digunakan untuk menentukan format data yang diuji, yaitu CSV, JSON, atau Parquet. Dengan demikian, hasil pengukuran waktu pemrosesan dan penggunaan sumber daya dapat dibandingkan dengan lebih adil antar format data dan arsitektur.

4.2.2 Arsitektur ELT

Pada arsitektur *Extract, Load, Transform* (ELT), proses ekstraksi dan pemuatan data dilakukan terlebih dahulu tanpa melakukan transformasi di luar *data warehouse* tujuan. Data sumber TPC-H dalam format CSV, JSON, dan Parquet dibaca menggunakan PySpark, lalu dimuat ke dalam skema *raw* dalam PostgreSQL. Setelah seluruh data berhasil dimuat, transformasi data dilakukan di dalam *data warehouse* menggunakan dbt. Dengan ini, seluruh transformasi utama, seperti penggabungan beberapa tabel, agregasi, filter, dan perhitungan analitis pada TPC-H dilakukan sepenuhnya di dalam lingkungan *data warehouse* tujuan, sehingga beban kerja yang besar dilakukan sepenuhnya oleh PostgreSQL.

Struktur proyek dbt pada arsitektur ELT dibagi menjadi dua lapisan utama, yaitu lapisan *staging* dan lapisan *marts*. Lapisan *staging* digunakan untuk mendefinisikan model awal dari delapan tabel data mentah yang telah dimuat ke skema *raw*, seperti *lineitem*, *orders*, *part*, *partsupp*, *customer*, *supplier*, *nation*, dan *region*. Pada lapisan ini dilakukan penyesuaian struktur awal seperti pemilihan kolom, penyesuaian nama kolom, dan penyesuaian tipe data jika diperlukan. Selanjutnya, lapisan data *marts* digunakan untuk membentuk tabel akhir dngan melakukan transformasi analitis berdasarkan logika TPC-H Query 9. Model akhir disimpan dalam file *mart_elt_query9.sql*.

Pemisahan antara model *staging* dan *marts* bertujuan untuk menjaga modularitas transformasi. Model *staging* berperan sebagai representasi bersih dari data mentah, sedangkan model *marts* berisi hasil transformasi analitis utama yang menghasilkan keluaran akhir penelitian tugas akhir ini.

4.2.2.1 Fase Ekstraksi (*Extract*)

Fase pertama dalam arsitektur pipeline ELT ini adalah ekstraksi data dari sumber. Fase ini dilakukan dengan menggunakan fungsi `extract_data()`. Fungsi ini menerima parameter nama tabel dan format file, kemudian membaca data sumber dari direktori `dataset/` menggunakan PySpark Dataframe. Seperti pada arsitektur ETL dan hibrida, penanganan format file CSV membutuhkan konfigurasi tambahan berupa `header=True`, sedangkan format JSON dan Parquet dapat dibaca menggunakan pembaca Spark sesuai format masing-masing. Pada tahap ini belum dilakukan transformasi analitis karena data yang dibaca akan langsung dimuat ke PostgreSQL sebagai data mentah.

Untuk memastikan fase ekstraksi dapat dianalisis dengan mudah melalui tampilan *Spark Web UI*, diterapkan pelabelan *Job* antarmuka *SparkContext* sebelum operasi baca (*read*) dieksekusi. Berikut merupakan bagian kode yang berperan dalam pelabelan tersebut:

```
1. spark.sparkContext.setJobGroup("ELT_Extract", f"ELT Extract Phase:
   {table_name}")
2. spark.sparkContext.setJobDescription(f"[ELT] Extract '{table_name}'
   from {file_format}")
```

Kode 4.8 Konfigurasi Spark Job Group pada Arsitektur ELT

Setelah dilakukan inisiasi pelabelan *Job* antarmuka *SparkContext*, dilakukan ekstraksi data mentah. Ekstraksi ini dilakukan dengan cara yang sama seperti ekstraksi pada arsitektur ETL dan hibrida, yaitu dengan memanfaatkan pernyataan *if-else* untuk menangani format data yang bervariasi. Implementasi ekstraksi pada arsitektur ELT tersebut ditunjukkan pada potongan kode berikut:

```
1. if file_format == "csv":
2.     df = spark.read \
3.         .format(file_format) \
4.         .option("header", "True") \
5.         .load(f"{DATASET_DIR}/{file_format}/{table_name}.{file_format}")
```

Kode 4.9 Implementasi Ekstraksi Data Multi-Format pada Arsitektur ELT

Di akhir tahapan ekstraksi pada arsitektur ELT ini diberi penanganan error, untuk menangani kondisi jika format file yang masuk sebagai parameter di luar cakupan penelitian tugas akhir ini.

```
1. else:
2.     raise ValueError(f"Format {file_format} tidak sesuai dengan
3.     pipeline ini.")
```

4.2.2.2 Fase Pemuatan Data Mentah (Load)

Setelah data berhasil diekstrak ke dalam Spark DataFrame, tahap selanjutnya adalah pemuatan data mentah ke dalam *data warehouse* PostgreSQL menggunakan koneksi JDBC. Berbeda dengan arsitektur ETL, pada tahap ini data tidak melalui proses transformasi analitis terlebih dahulu. Data yang telah dibaca dari sumber langsung dimuat ke PostgreSQL dalam bentuk mentah agar dapat dilakukan transformasi setelah data berada di dalam *data warehouse*.

Pada penelitian ini, proses pemuatan dilakukan dengan menggunakan koneksi JDBC dari Apache Spark ke PostgreSQL. Seluruh tabel TPC-H yang digunakan dalam proses transformasi Query 9 dimuat ke dalam skema *raw* dengan prefiks *elt_*, seperti *raw.elt_lineitem*, *raw.elt_orders*, *raw.elt_part*, *raw.elt_partsupp*, *raw.elt_supplier*, dan *raw.elt_nation*. Penggunaan schema *raw* bertujuan untuk memisahkan data mentah hasil pemuatan dari tabel hasil transformasi yang dibentuk oleh dbt. Dengan pemisahan ini, struktur data pada arsitektur ELT menjadi lebih jelas, yaitu data mentah disimpan terlebih dahulu pada lapisan *raw*, sedangkan data hasil transformasi disimpan pada model *staging* dan *marts* yang dikelola oleh dbt.

Proses pemuatan data mentah dilakukan melalui fungsi *load_data()*. Fungsi ini menerima dua parameter, yaitu *table_name* sebagai nama tabel sumber dan *df* sebagai Spark DataFrame hasil ekstraksi. Di dalam fungsi tersebut, *Spark Job Group* dan *Job Description* juga diatur agar proses pemuatan setiap tabel dapat dipantau secara lebih jelas melalui Spark Web UI. Hal ini memudahkan proses monitor ketika *pipeline* dijalankan, terutama untuk melihat tahapan pemuatan data pada masing-masing tabel.

```

1.  # LOADING PHASE
2.  def load_data(table_name, df):
3.      spark.sparkContext.setJobGroup("ELT_Load", f"ELT Load Phase:
      {table_name}")
4.      spark.sparkContext.setJobDescription(f"[ELT] Load '{table_name}'
      into raw.elt_{table_name}")
5.      print(f"Starting to Load table '{table_name}'...")
6.      try:
7.          df.write.jdbc(
8.              url=DB_URL,
9.              table=f"raw.elt_{table_name}",
10.             mode="overwrite",
11.             properties=DB_PROPERTIES
12.          )
13.     except Exception as e:
14.         print(f"Error loading data: {e}")

```

Kode 4.11 Implementasi Pemuatan Data Mentah pada Arsitektur ELT

Pada kode tersebut, proses pemuatan data dilakukan dengan menggunakan *write.jdbc()* yang disediakan oleh Spark DataFrame. Beberapa parameter yang digunakan adalah *url* yang berfungsi untuk mendefinisikan alamat koneksi menuju basis data PostgreSQL, sedangkan

parameter *table* menentukan nama tabel tujuan di dalam *data warehouse*. Parameter *mode="overwrite"* digunakan agar tabel tujuan ditulis ulang setiap kali pengujian dijalankan. Penggunaan mode ini bertujuan untuk memastikan bahwa setiap skenario pengujian dimulai dari kondisi data yang bersih dan konsisten. Dengan demikian, hasil pengujian tidak dipengaruhi oleh data dari eksekusi sebelumnya. Sementara itu, parameter *properties* berisi informasi autentikasi dan *driver* JDBC PostgreSQL yang digunakan untuk melakukan koneksi ke database.

Konfigurasi koneksi PostgreSQL pada pipeline ELT didefinisikan dengan memanfaatkan *environment variable* (pada file *.env*) yang dimuat menggunakan library *dotenv*. Pendekatan ini digunakan agar informasi sensitif seperti nama pengguna, kata sandi, *host*, dan nama basis data tidak ditulis secara langsung di dalam kode program. Dengan demikian, konfigurasi koneksi ke PostgreSQL menjadi lebih fleksibel dan lebih aman untuk digunakan dalam proses pengembangan maupun pengujian.

```
1.  # DB CONNECTION PROPERTIES
2.  load_dotenv()

3.  DB_HOST = os.getenv("DB_HOST")
4.  DB_PORT = os.getenv("DB_PORT")
5.  DB_NAME = os.getenv("DB_NAME")
6.  DB_USER = os.getenv("DB_USER")
7.  DB_PASS = os.getenv("DB_PASS")
8.  DB_RAW_SCHEMA = os.getenv("DB_RAW_SCHEMA")

9.  DB_URL = f"jdbc:postgresql://{DB_HOST}:{DB_PORT}/{DB_NAME}"

10. DB_PROPERTIES = {
11.     "user": DB_USER,
12.     "password": DB_PASS,
13.     "driver": "org.postgresql.Driver"
14. }
```

Kode 4.12 Konfigurasi Koneksi PostgreSQL pada Arsitektur ELT

Dalam konteks arsitektur ELT, tahap pemuatan ini dianggap krusial karena seluruh proses transformasi bergantung pada keberhasilan data mentah dimuat ke dalam PostgreSQL tujuan. Setelah tabel-tabel tersedia di dalam skema *raw*, dbt akan menggunakan tabel tersebut sebagai sumber data melalui konfigurasi *sources.yml*. Selanjutnya, model *staging* dan *marts* pada dbt akan melakukan transformasi analitis berdasarkan logika pada standar TPC-H Query 9. Dengan demikian, tahap pemuatan pada arsitektur ELT tidak sekedar untuk menyimpan data ke *data warehouse*, tetapi juga menjadi prasyarat utama bagi tahap transformasi data yang dilakukan setelahnya.

Perlu diperhatikan bahwa pada tahap pemuatan ini belum dilakukan proses penggabungan, agregasi, filter, maupun perhitungan metrik analitis. Seluruh operasi tersebut akan dilakukan pada tahap transformasi menggunakan dbt. Hal ini sesuai dengan prinsip utama arsitektur ELT, yaitu memuat data mentah terlebih dahulu ke dalam *data warehouse*, kemudian memanfaatkan kemampuan mesin basis data target untuk melakukan transformasi data. Oleh karena itu, proses

pemuatan data mentah pada arsitektur ELT menjadi pembeda utama dibandingkan arsitektur ETL, di mana transformasi dilakukan sebelum data dimuat ke *data warehouse*.

4.2.2.3 Fase Transformasi dengan dbt (*Transform*)

Data yang sudah berhasil dimuat ke dalam PostgreSQL kemudian dilakukan transformasi dengan standar kueri 9 TPC-H. Berbeda dengan arsitektur ETL yang menjalankan transformasi sebelum data mencapai lingkungan basis data tujuan, pada arsitektur ELT transformasi dijalankan setelah data berada di dalam sistem PostgreSQL target. Pada penelitian tugas akhir ini, transformasi data dilakukan dengan menggunakan alat dbt (*data build tool*) dan mesin pemrosesan data PostgreSQL.

Penggunaan dbt pada tahap transformasi ELT bertujuan untuk memisahkan proses transformasi dengan proses ekstraksi dan pemuatan yang dilakukan sepenuhnya dengan PySpark. Dengan pendekatan ini, PySpark hanya digunakan untuk membaca data dari sumber dan memuat seluruh data mentah ke sistem basis data target, sedangkan seluruh transformasi dilakukan menggunakan alat berbasis SQL di dalam *data warehouse*. Transformasi yang dilakukan mengacu pada logika TPC-H Query 9, yang mencakup penggabungan beberapa tabel, agregasi berdasarkan negara dan tahun, filter data, perhitungan nilai profit, serta pengurutan hasil data akhir.

Pada implementasinya, model dbt untuk arsitektur ELT ditempatkan pada direktori *models/elt*. Struktur direktori tersebut dibagi menjadi dua lapisan utama, yaitu *staging* dan *marts*. Lapisan *staging* digunakan sebagai wadah untuk menampung data mentah yang dimuat dari sumber oleh PySpark ke skema *raw*, sedangkan lapisan *marts* digunakan untuk membentuk tabel hasil transformasi akhir berdasarkan kebutuhan analitis kueri 9. Struktur direktori model dbt pada arsitektur ELT dapat ditunjukkan sebagai berikut:

```
dbt_transform/
├── models/
│   ├── elt/
│   │   ├── staging/
│   │   │   ├── stg_elt_lineitem.sql
│   │   │   ├── stg_elt_nation.sql
│   │   │   ├── stg_elt_orders.sql
│   │   │   ├── stg_elt_part.sql
│   │   │   ├── stg_elt_partsupp.sql
│   │   │   └── stg_elt_supplier.sql
│   │   └── marts/
│   │       └── mart_elt_query9.sql
│   └── sources.yml
```

Kode 4.13 Struktur Direktori dbt pada Arsitektur ELT

File *sources.yml* digunakan untuk mendefinisikan tabel-tabel mentah yang dimuat ke dalam sistem *data warehouse* PostgreSQL, sebagai sumber data bagi model dbt. Pada arsitektur ELT, tabel mentah disimpan di skema *raw* dengan prefiks *elt_*, seperti *raw.elt_lineitem*, *raw.elt_orders*, *raw.elt_part*, *raw.elt_partsupp*, *raw.elt_supplier*, dan *raw.elt_nation*.

Pendefinisian tabel sumber data pada *sources.yml* membuat hubungan antara data mentah dan model transformasi menjadi lebih eksplisit. Berikut merupakan isi file *sources.yml*:

```
1.  version: 2
2.
3.  sources:
4.    - name: raw_data
5.      schema: raw
6.      tables:
7.        # ELT
8.        - name: elt_nation
9.        - name: elt_region
10.       - name: elt_customer
11.       - name: elt_part
12.       - name: elt_supplier
13.       - name: elt_partsupp
14.       - name: elt_orders
15.       - name: elt_lineitem
```

Kode 4.14 Konfigurasi *sources.yml* pada Arsitektur ELT

Dengan konfigurasi tersebut, model dbt dapat mengakses tabel mentah menggunakan fungsi *source()*. Sebagai contoh, tabel mentah *raw.elt_lineitem* dapat direferensikan dengan sintaks `{{ source('raw', 'elt_lineitem') }}`. Pendekatan file konfigurasi *source.yml* ini lebih jelas dan terstruktur jika dibandingkan dengan menuliskan nama skema dan tabel secara langsung di setiap model dbt, karena sumber data didefinisikan secara terpusat pada satu file.

Tahap pertama dalam transformasi dbt adalah pembentukan model *staging*. Model *staging* berfungsi sebagai lapisan awal yang mengambil data dari tabel mentah, memilih kolom yang relevan, serta melakukan penyesuaian tipe data apabila diperlukan. Pada arsitektur ELT, proses *staging* menjadi penting karena data yang dimuat ke schema *raw* belum mengalami transformasi analitis. Oleh karena itu, model *staging* digunakan untuk memastikan data memiliki struktur yang sesuai sebelum digunakan pada model *marts*.

Model *staging* dibuat untuk semua tabel yang digunakan, mulai dari *lineitem*, *orders*, *part*, *partsupp*, *supplier*, dan *nation*. Salah satu contohnya adalah *stg_elt_lineitem.sql*, yang digunakan untuk mengambil kolom-kolom dari tabel *raw.elt_lineitem* yang dibutuhkan dalam transformasi dengan logika kueri 9. Berikut merupakan model tersebut:

```
1.  WITH source_data AS (
2.    SELECT * FROM {{ source('raw_data', 'elt_lineitem') }}
3.  ),
4.
5.  casted_data AS (
6.    SELECT
7.      CAST(l_orderkey AS INTEGER) AS l_orderkey,
8.      CAST(l_partkey AS INTEGER) AS l_partkey,
9.      CAST(l_suppkey AS INTEGER) AS l_suppkey,
10.     CAST(l_quantity AS DOUBLE PRECISION) AS l_quantity,
```

```

11.         CAST(l_extendedprice AS DOUBLE PRECISION) AS l_extendedprice,
12.         CAST(l_discount AS DOUBLE PRECISION) AS l_discount
13.
14.     FROM source_data
15. )
16.
17. SELECT * FROM casted_data

```

Kode 4.15 Model Staging stg_elt_lineitem.sql pada Arsitektur ELT

Pada kode tersebut, didefinisikan data sumber sebagai *source_data*, kemudian *casted_data* mewakili kolom-kolom yang dibutuhkan dari tabel *lineitem*, yaitu *l_orderkey*, *l_partkey*, *l_suppkey*, *l_quantity*, *l_extendedprice*, dan *l_discount*, sekaligus melakukan *casting* tipe data masing-masing. Proses *casting* dilakukan untuk memastikan tipe data numerik dapat digunakan dalam operasi perhitungan. Hal ini penting terutama ketika data dimuat dari format berbasis teks seperti CSV atau JSON, yang berpotensi terbaca sebagai tipe data *string*.

Berikut merupakan model *staging* untuk tabel-tabel lainnya:

```

1.  WITH source_data AS (
2.      SELECT * FROM {{ source('raw_data', 'elt_orders') }}
3.  ),
4.  casted_data AS (
5.      SELECT
6.          CAST(o_orderkey AS INTEGER) AS o_orderkey,
7.          CAST(o_orderdate AS DATE) AS o_orderdate
8.      FROM source_data
9.  )
10. SELECT * FROM casted_data

```

Kode 4.16 Model Staging stg_elt_orders.sql pada Arsitektur ELT

```

1.  WITH source_data AS (
2.      SELECT * FROM {{ source('raw_data', 'elt_part') }}
3.  ),
4.  casted_data AS (
5.      SELECT
6.          CAST(p_partkey AS INTEGER) AS p_partkey,
7.          CAST(p_name AS VARCHAR) AS p_name
8.      FROM source_data
9.  )
10. SELECT * FROM casted_data

```

Kode 4.17 Model Staging stg_elt_part.sql pada Arsitektur ELT

```

1.  WITH source_data AS (
2.      SELECT * FROM {{ source('raw_data', 'elt_partsupp') }}
3.  ),

```

```

4.  casted_data AS (
5.      SELECT
6.          CAST(ps_partkey AS INTEGER) AS ps_partkey,
7.          CAST(ps_suppkey AS INTEGER) AS ps_suppkey,
8.          CAST(ps_supplycost AS DOUBLE PRECISION) AS ps_supplycost
9.
10.     FROM source_data
11. )
12. SELECT * FROM casted_data

```

Kode 4.18 Model Staging *stg_elt_partsupp.sql* pada Arsitektur ELT

```

1.  WITH source_data AS (
2.      SELECT * FROM {{ source('raw_data', 'elt_supplier') }}
3.  ),
4.  casted_data AS (
5.      SELECT
6.          CAST(s_suppkey AS INTEGER) AS s_suppkey,
7.          CAST(s_nationkey AS INTEGER) AS s_nationkey
8.
9.     FROM source_data
10. )
11. SELECT * FROM casted_data

```

Kode 4.19 Model Staging *stg_elt_supplier.sql* pada Arsitektur ELT

```

1.  WITH source_data AS (
2.      SELECT * FROM {{ source('raw_data', 'elt_nation') }}
3.  ),
4.  casted_data AS (
5.      SELECT
6.          CAST(n_nationkey AS INTEGER) AS n_nationkey,
7.          CAST(n_name AS VARCHAR) AS n_name
8.
9.     FROM source_data
10. )
11. SELECT * FROM casted_data

```

Kode 4.20 Model Staging *stg_elt_nation.sql* pada Arsitektur ELT

Setelah seluruh model *staging* terbentuk, tahap berikutnya adalah pembentukan model *marts*. Model ini merupakan lapisan akhir yang menjalankan kueri transformasi analitis utama berdasarkan standar TPC-H Query 9. Pada penelitian tugas akhir ini, model ini diberi nama *mart_elt_query9.sql*. Model ini menggabungkan beberapa operasi penggabungan beberapa model *staging*, kemudian menghitung nilai profit berdasarkan harga penjualan, diskon, jumlah barang, dan biaya pasokan. Hasil akhir dari transformasi tersebut kemudian diagregasi berdasarkan negara dan tahun.

```

1.  select
2.      nation,
3.      o_year,
4.      sum(amount) as sum_profit
5.  from
6.      (
7.          select
8.              n.n_name as nation,
9.              extract(year from o.o_orderdate) as o_year,
10.             l.l_extendedprice * (1 - l.l_discount) - ps.ps_supplycost *
11.             l.l_quantity as amount
12.          from
13.              {{ ref('stg_elt_part') }} p,
14.              {{ ref('stg_elt_supplier') }} s,
15.              {{ ref('stg_elt_lineitem') }} l,
16.              {{ ref('stg_elt_partsupp') }} ps,
17.              {{ ref('stg_elt_orders') }} o,
18.              {{ ref('stg_elt_nation') }} n
19.          where
20.              s.s_suppkey = l.l_suppkey
21.              and ps.ps_suppkey = l.l_suppkey
22.              and ps.ps_partkey = l.l_partkey
23.              and p.p_partkey = l.l_partkey
24.              and o.o_orderkey = l.l_orderkey
25.              and s.s_nationkey = n.n_nationkey
26.              and p.p_name like '%green%'
27.          ) as profit
28.  group by
29.      nation,
30.      o_year
31.  order by
32.      nation,
33.      o_year desc

```

Kode 4.21 Model Mart *mart_elt_query9.sql* pada Arsitektur ELT

Pada model *mart_elt_query9*, fungsi *ref()* digunakan untuk mereferensikan model staging yang telah didefinisikan sebelumnya. Penggunaan *ref()* memungkinkan dbt untuk membangun *dependencies* antar model secara otomatis, sehingga model *marts* hanya akan dijalankan setelah model *staging* yang dibutuhkan tersedia. Hal ini membantu menjaga konsistensi urutan eksekusi transformasi.

Transformasi pada model *marts* terdiri dari beberapa operasi utama. Pertama, model melakukan penggabungan antara tabel *part*, *lineitem*, *supplier*, *partsupp*, *orders*, dan *nation*. Operasi penggabungan tersebut digunakan untuk menghubungkan informasi produk, transaksi, pemasok, biaya pasokan, tanggal pesanan, dan negara asal pemasok. Kedua, model melakukan filter terhadap nama *part* yang mengandung kata “green”, sesuai dengan logika TPC-H Query 9. Ketiga, model menghitung nilai profit menggunakan selisih antara pendapatan setelah diskon

dan biaya pasokan. Keempat, hasil perhitungan diagregasi berdasarkan nama negara dan tahun pesanan. Terakhir, hasil akhir diurutkan berdasarkan negara dan tahun secara menurun.

Secara konseptual, tahap transformasi ini menunjukkan karakteristik utama arsitektur ELT. Data mentah dimuat terlebih dahulu ke dalam PostgreSQL, kemudian transformasi dilakukan menggunakan dbt berbasis SQL di dalam *data warehouse*. Dengan demikian, beban komputasi untuk operasi *join*, agregasi, filter, dan perhitungan numerik tidak dilakukan oleh PySpark sebelum pemuatan, melainkan oleh PostgreSQL setelah data tersedia di dalam *data warehouse*.

Untuk menjalankan model dbt pada arsitektur ELT secara spesifik, dapat digunakan perintah berikut.

```
dbt run --select elt
```

Kode 4.22 Perintah Eksekusi Transformasi dbt pada Arsitektur ELT

Dalam konteks pengujian performa, durasi transformasi pada arsitektur ELT dihitung dari waktu eksekusi model dbt. Waktu eksekusi tersebut mencakup durasi yang dibutuhkan PostgreSQL untuk menjalankan transformasi setelah data mentah berhasil dimuat. Dengan demikian, waktu total proses ELT dapat diperoleh dari gabungan durasi proses ekstraksi dan pemuatan menggunakan PySpark serta durasi transformasi menggunakan dbt.

4.2.3 Arsitektur Hibrida

Arsitektur hibrida pada penelitian tugas akhir ini dirancang sebagai kombinasi antara ETL dan ELT. Pada pendekatan ini, sebagian proses transformasi dilakukan sebelum data dimuat ke dalam *data warehouse* tujuan, dan transformasi analitis utama dilakukan setelah data berada di dalam PostgreSQL. Dengan demikian, arsitektur hibrida menggabungkan konsep ETL yang melakukan pembersihan awal data sebelum data dimuat ke sistem target, dan konsep ELT yang memanfaatkan sistem target untuk mengeksekusi transformasi analitis.

Pada implementasi penelitian ini, PySpark digunakan untuk menjalankan tahap ekstraksi, pembersihan awal, dan pemuatan data ke PostgreSQL. Sementara itu, transformasi akhir dijalankan berdasarkan logika TPC-H *Query 9* dengan dbt. Transformasi yang dilakukan oleh PySpark berfokus pada pembersihan data yang bersifat ringan, seperti pemilihan kolom yang relevan dan *casting* tipe data yang sesuai. Setelah data yang telah dilakukan pembersihan tersebut dimuat ke dalam sistem *data warehouse* PostgreSQL, dbt digunakan untuk menjalankan transformasi analitis yang lebih kompleks, seperti penggabungan beberapa tabel, filter, perhitungan profit, agregasi, dan pengurutan hasil akhir.

Perbedaan utama antara arsitektur ETL dan ELT dengan arsitektur hibrida adalah pada pembagian beban kerja transformasi. Pada arsitektur ETL, seluruh beban transformasi dilakukan dengan PySpark sebelum data dimuat ke dalam sistem basis data tujuan. Pada arsitektur ELT, data mentah dimuat langsung ke dalam sistem tujuan sebelum kemudian dilakukan transformasi analitis menggunakan dbt. Sedangkan, pada arsitektur hibrida, proses transformasi dibagi menjadi dua tahap, yaitu pembersihan awal yang dilakukan oleh PySpark dan transformasi analitis yang dilakukan oleh dbt setelah data berada di PostgreSQL.

4.2.3.1 Fase Ekstraksi (*Extract*)

Arsitektur hibrida diawali dengan fase ekstraksi data. Tahap ini bertujuan untuk membaca data sumber TPC-H dari direktori dataset ke dalam bentuk Spark DataFrame. Tahap ini bertujuan untuk mengambil data berdasarkan nama tabel dan format file yang digunakan dalam skenario pengujian. Pada implementasi arsitektur hibrida, format data yang didukung adalah CSV, JSON, dan Parquet. Pada arsitektur hibrida, data hasil ekstraksi tidak langsung dimuat ke PostgreSQL seperti pada arsitektur ELT, namun akan diproses terlebih dahulu pada fase pembersihan awal sebelum kemudian dimuat ke *data warehouse* tujuan. Oleh karena itu, fase ekstraksi hanya berfokus pada pembacaan data sumber tanpa melakukan operasi transformasi analitis seperti penggabungan, agregasi, filter, maupun perhitungan profit. Transformasi analitis utama tetap dilakukan pada tahap akhir menggunakan dbt setelah data berada di PostgreSQL.

Proses ekstraksi diimplementasikan menggunakan fungsi `extract_data()`. Fungsi ini menerima dua parameter, yaitu `table_name` dan `file_format`. Parameter `table_name` digunakan untuk menentukan tabel TPC-H yang akan dibaca, sedangkan parameter `file_format` digunakan untuk menentukan format file sumber. Dengan pendekatan ini, fungsi ekstraksi dapat digunakan secara modular untuk membaca beberapa tabel TPC-H dengan format data yang berbeda tanpa perlu membuat fungsi terpisah untuk setiap format.

```
1. # EXTRACTION PHASE
2. def extract_data(table_name, file_format):
3.     spark.sparkContext.setJobGroup("Hybrid_Extract", f"Hybrid Extract
   Phase: {table_name}")
4.     spark.sparkContext.setJobDescription(f"[Hybrid] Extract
   '{table_name}' from {file_format}")
```

Kode 4.23 Konfigurasi Spark Job Group pada Tahap Ekstraksi Arsitektur Hibrida

```
1. if file_format == "csv": # butuh config header=True khusus untuk CSV.
2.     df = spark.read \
3.         .format(file_format) \
4.         .option("header", "True") \
5.         .load(f"{DATASET_DIR}/{file_format}/{table_name}.{file_format}
   ")
6. elif file_format == "jsonl": # format "jsonl" dihandle oleh
   .format("json") di pyspark, jadi butuh special handling
7.     df = spark.read \
8.         .format("json") \
9.         .load(f"{DATASET_DIR}/{file_format}/{table_name}.{file_format}
   ")
10. elif file_format == "parquet": # normal case handling
11.     df = spark.read \
12.         .format(file_format) \
13.         .load(f"{DATASET_DIR}/{file_format}/{table_name}.{file_format}
   ")
14. else:
15.     raise ValueError(f"Format {file_format} tidak sesuai dengan
   pipeline ini.")
```

```
16.     print(f"Table '{table_name}' extracted successfully.")
17.     return df
```

Kode 4.24 Implementasi Ekstraksi Data Multi-Format pada Arsitektur Hibrida

Pada kode tersebut, fungsi *extract_data()* terlebih dahulu mengatur Spark Job Group dengan nama *Hybrid_Extract*. Pengaturan ini digunakan agar proses ekstraksi setiap tabel dapat dipantau secara lebih jelas melalui Spark Web UI. Selain itu, *setJobDescription()* digunakan untuk memberikan deskripsi terhadap *job* Spark yang sedang berjalan, sehingga proses ekstraksi dapat diidentifikasi berdasarkan nama tabel dan format file yang sedang diproses.

Bagian utama dari fungsi ekstraksi adalah percabangan *if-else* berdasarkan format data. Untuk format CSV, konfigurasi *.option("header", "True")* digunakan agar baris pertama pada file dikenali sebagai nama kolom. Hal ini diperlukan karena CSV merupakan format berbasis teks yang tidak menyimpan metadata skema secara eksplisit. Untuk format JSON Lines (jsonl), Spark tetap menggunakan *.format("json")* karena setiap baris pada file merepresentasikan satu objek JSON yang dapat dibaca oleh *JSON reader* milik Spark. Sementara itu, format Parquet dapat dibaca langsung menggunakan *.format("parquet")* karena Parquet telah menyimpan informasi skema dan metadata di dalam file. Fungsi ini juga dilengkapi dengan mekanisme validasi format. Apabila nilai *file_format* yang diberikan tidak termasuk dalam format yang didukung, yaitu CSV, JSON Lines, atau Parquet, maka fungsi akan menghasilkan *ValueError*. Mekanisme ini digunakan untuk mencegah proses integrasi tetap berjalan ketika konfigurasi format data tidak sesuai dengan format yang telah ditentukan dalam eksperimen.

Hasil dari fase ekstraksi adalah Spark DataFrame yang merepresentasikan data dari masing-masing tabel TPC-H. DataFrame tersebut kemudian digunakan sebagai masukan data pada fase berikutnya, yaitu fase pembersihan awal. Dengan demikian, fase ekstraksi pada arsitektur hibrida hanya berperan sebagai tahap pembacaan data sumber, sedangkan proses pemilihan kolom, penyesuaian tipe data, dan pembersihan struktur awal dilakukan pada fase pembersihan.

4.2.3.2 Fase Pembersihan (*Clean*)

Setelah data berhasil diekstraksi ke dalam bentuk Spark DataFrame, tahap berikutnya pada arsitektur hibrida adalah melakukan pembersihan awal, sebelum data dimuat ke PostgreSQL. Tahap ini menjadi pembeda utama arsitektur hibrida dibandingkan arsitektur ELT. Pada ELT, data mentah langsung dimuat ke data warehouse tanpa transformasi awal. Sementara itu, pada arsitektur hibrida, data mengalami pembersihan ringan terlebih dahulu agar data yang masuk ke PostgreSQL sudah memiliki struktur dasar yang lebih siap untuk diproses lebih lanjut oleh dbt. Pembersihan awal yang dilakukan pada tahap ini tidak mencakup transformasi analitik kompleks seperti join antar tabel, agregasi, atau perhitungan profit. Transformasi analitik tersebut tetap dilakukan di lingkungan PostgreSQL dengan dbt. Tahap pembersihan awal hanya difokuskan pada proses yang bersifat dasar, seperti memilih kolom yang relevan dengan TPC-H Query 9 serta melakukan *casting* tipe data untuk masing-masing kolom.

Fase pembersihan data diimplementasikan menggunakan fungsi *clean_data()*. Fungsi ini menerima dua parameter, yaitu *table_name* sebagai nama tabel yang sedang diproses dan *df* sebagai Spark DataFrame hasil ekstraksi. Berdasarkan nilai *table_name*, fungsi akan memilih kolom yang relevan dan melakukan *casting* tipe data pada masing-masing tabel.

```

1. def clean_data(table_name, df):
2.     spark.sparkContext.setJobGroup("Hybrid_Clean", f"Hybrid Clean
   Phase: {table_name}")
3.     spark.sparkContext.setJobDescription(f"[Hybrid] Clean
   '{table_name}' - select & cast columns")

```

Kode 4.25 Konfigurasi Spark Job Group pada Tahap Pembersihan Arsitektur Hibrida

Pada kode tersebut, fungsi `clean_data()` terlebih dahulu mengatur *Spark Job Group* dengan nama “Hybrid_Clean”. Pengaturan ini bertujuan agar proses pembersihan setiap tabel dapat dipantau secara lebih jelas melalui *Spark Web UI*. Selain itu, `setJobDescription()` digunakan untuk memberikan deskripsi bahwa proses yang sedang berjalan adalah proses pemilihan kolom dan *casting* tipe data. Proses pembersihan dilakukan dengan menggunakan `select()` dari Spark DataFrame. Metode ini digunakan untuk memilih kolom-kolom tertentu dari setiap tabel yang relevan dengan kebutuhan TPC-H Query 9. Pada saat yang sama, setiap kolom juga dikonversi ke tipe data yang sesuai menggunakan metode `cast()`.

```

1. if table_name == "lineitem":
2.     df = df.select(
3.         col("l_orderkey").cast("integer"),
4.         col("l_partkey").cast("integer"),
5.         col("l_suppkey").cast("integer"),
6.         col("l_quantity").cast("double"),
7.         col("l_extendedprice").cast("double"),
8.         col("l_discount").cast("double")
9.     )
10.
11. elif table_name == "orders":
12.     df = df.select(
13.         col("o_orderkey").cast("integer"),
14.         col("o_orderdate").cast("date")
15.     )
16.
17. elif table_name == "part":
18.     df = df.select(
19.         col("p_partkey").cast("integer"),
20.         col("p_name").cast("string")
21.     )
22.
23. elif table_name == "supplier":
24.     df = df.select(
25.         col("s_suppkey").cast("integer"),
26.         col("s_nationkey").cast("integer")
27.     )
28. elif table_name == "nation":
29.     df = df.select(
30.         col("n_nationkey").cast("integer"),
31.         col("n_name").cast("string")

```

```

32.     )
33.     elif table_name == "partsupp":
34.         df = df.select(
35.             col("ps_partkey").cast("integer"),
36.             col("ps_suppkey").cast("integer"),
37.             col("ps_supplycost").cast("double")
38.         )

```

Kode 4.26 Implementasi Pembersihan Data pada Arsitektur Hibrida

Pada tabel *lineitem*, kolom yang dipilih adalah *l_orderkey*, *l_partkey*, *l_suppkey*, *l_quantity*, *l_extendedprice*, dan *l_discount*. Kolom-kolom tersebut digunakan dalam proses transformasi penggabungan dengan tabel lain serta untuk perhitungan nilai profit. Kolom *l_orderkey*, *l_partkey*, dan *l_suppkey* dikonversi ke tipe data *integer* karena digunakan sebagai kunci relasi penggabungan antartabel (*join key*). Sementara itu, kolom *l_quantity*, *l_extendedprice*, dan *l_discount* dikonversi ke tipe data *double* karena digunakan dalam operasi perhitungan numerik.

Pada tabel *orders*, kolom yang dipilih adalah *o_orderkey* dan *o_orderdate*. Kolom *o_orderkey* digunakan sebagai kunci untuk menghubungkan tabel *orders* dengan tabel *lineitem*, sedangkan kolom *o_orderdate* berguna untuk mendapatkan informasi tahun pesanan untuk keperluan transformasi dengan logika Query 9. Oleh karena itu, kolom *o_orderkey* dinyatakan sebagai tipe data *integer* dan *o_orderdate* sebagai tipe data *date*.

Pada tabel *part*, kolom yang dipilih adalah *p_partkey* dan *p_name*. Kolom *p_partkey* digunakan sebagai kunci untuk menghubungkan tabel *part* dengan tabel *lineitem*, sedangkan *p_name* digunakan dalam proses filter nama *part* pada Query 9. Sehingga kolom *p_partkey* cocok dikonversi ke tipe data *integer*, sedangkan *p_name* dikonversi sebagai tipe data *string*.

Pada tabel *supplier*, kolom yang dipilih adalah *s_suppkey* dan *s_nationkey*. Kolom *s_suppkey* berguna sebagai penghubung data *supplier* dengan tabel *lineitem* dan *partsupp*, sedangkan untuk kolom *s_nationkey* digunakan untuk menghubungkan data tabel *supplier* dengan tabel *nation*. Karena keduanya berfungsi sebagai kunci relasi, kolom-kolom tersebut dikonversi ke tipe data *integer*.

Pada tabel *nation*, kolom yang dipilih adalah *n_nationkey* dan *n_name*. Kolom *n_nationkey* digunakan sebagai kunci untuk menghubungkan tabel *nation* dengan tabel *supplier*, sedangkan *n_name* digunakan sebagai atribut negara untuk hasil akhir logika transformasi Query 9. Kolom *n_nationkey* dikonversi ke tipe data *integer*, dan *n_name* dikonversi ke tipe data *string*.

Pada tabel *partsupp*, kolom yang dipilih adalah kolom *ps_partkey*, *ps_suppkey*, dan *ps_supplycost*. Kolom *ps_partkey* dan *ps_suppkey* digunakan untuk menghubungkan tabel *partsupp* dengan tabel *lineitem*, sedangkan kolom *ps_supplycost* digunakan dalam perhitungan nilai profit. Oleh karena itu, *ps_partkey* dan *ps_suppkey* dikonversi ke tipe *integer*, sementara *ps_supplycost* dikonversi ke tipe data *double*.

Dengan adanya fase pembersihan data ini, arsitektur hibrida tidak memuat seluruh kolom dari data mentah ke PostgreSQL. Kolom-kolom yang dimuat hanya yang dibutuhkan dalam transformasi TPC-H Query 9. Hal ini membuat data yang dimuat ke dalam *data warehouse* menjadi lebih ringan dan terfokus pada kebutuhan jika dibandingkan dengan pendekatan ELT murni. Namun, karakteristik pendekatan ELT masih dipertahankan dari segi transformasi

analitis utama yang dilakukan di dalam sistem target, seperti proses penggabungan, filter, agregasi, dan perhitungan profit.

Perlu diperhatikan juga bahwa operasi *select()* dan *cast()* pada Spark bersifat “lazy”. Dengan itu, operasi tersebut belum benar-benar dieksekusi sampai terdapat *action*, seperti proses pemuatan data menggunakan koneksi JDBC pada tahap berikutnya. Oleh karena itu, fase pembersihan ini didefinisikan pada tahap ini, namun eksekusi fisiknya dapat terjadi bersamaan dengan proses pemuatan ketika Spark menjalankan *action* untuk menulis data ke PostgreSQL.

4.2.3.3 Fase Pemuatan (Load)

Setelah data melalui fase pembersihan (*clean*), tahap berikutnya dalam arsitektur hibrida ini adalah fase pemuatan data (*load*). Pada tahap ini, data yang telah diseleksi kolomnya dan disesuaikan tipe datanya dimuat ke dalam *data warehouse* PostgreSQL. Berbeda dengan arsitektur ELT yang memuat data mentah secara langsung, pada arsitektur hibrida data yang dimuat sudah melalui proses pembersihan awal menggunakan PySpark. Proses pemuatan dilakukan menggunakan koneksi JDBC dari Spark DataFrame ke PostgreSQL. Setiap tabel hasil pembersihan dimuat ke dalam skema *raw* dengan prefiks *hybrid_*, seperti *raw.hybrid_lineitem*, *raw.hybrid_orders*, *raw.hybrid_part*, *raw.hybrid_supplier*, *raw.hybrid_partsupp*, dan *raw.hybrid_nation*. Penggunaan prefiks tersebut bertujuan untuk membedakan tabel hasil pemuatan arsitektur hibrida dengan tabel pada arsitektur lain.

Fase pemuatan diimplementasikan menggunakan fungsi *load_data()*. Fungsi ini menerima dua parameter, yaitu *table_name* yang mewakili nama tabel yang sedang diproses, dan *df* sebagai Spark DataFrame hasil pembersihan. Fungsi ini kemudian menulis DataFrame tersebut ke PostgreSQL menggunakan metode *write.jdbc()*.

```
1. def load_data(table_name, df):
2.     spark.sparkContext.setJobGroup("Hybrid_Load", f"Hybrid Load Phase:
3.     {table_name}")
4.     spark.sparkContext.setJobDescription(f"[Hybrid] Load
5.     '{table_name}' into raw.hybrid_{table_name}")
6.     print(f"Starting to load table '{table_name}'...")
7.     try:
8.         df.write.jdbc(
9.             url=DB_URL,
10.            table=f"raw.hybrid_{table_name}",
11.            mode="overwrite",
12.            properties=DB_PROPERTIES
13.        )
14.     except Exception as e:
15.         print(f"Error loading data: {e}")
```

Kode 4.27 Implementasi Pemuatan Data pada Arsitektur Hibrida

Pada kode tersebut, proses pemuatan diawali dengan pengaturan Spark Job Group menggunakan *setJobGroup()* dan memberinya nama “Hybrid_Load”. Dengan dilakukannya pengaturan ini, proses pemuatan setiap tabel dapat diamati dengan lebih jelas pada Spark Web UI. Selain itu, diatur juga *setJobDescription()* untuk memberikan deskripsi dari *job* yang

sedang berjalan, yaitu proses pemuatan tabel tertentu dari Spark DataFrame ke dalam PostgreSQL dengan skema *raw* dengan prefiks *hybrid_*.

Proses utama tahap pemuatan menggunakan metode *df.write.jdbc()*, yang memanfaatkan beberapa parameter, yaitu *url*, *table*, *mode*, dan *properties*. Parameter *url* berguna untuk menentukan alamat koneksi JDBC melalui basis data PostgreSQL. Parameter *table* digunakan untuk menentukan tabel tujuan, yang pada arsitektur ini diberi prefiks “*hybrid_*” dan ditempatkan di skema *raw*, dan dilanjutkan dengan nama tabel yang diatur secara dinamis menggunakan variabel *{table_name}* agar setiap tabel dimuat ke tabel tujuan yang sesuai. Parameter *mode* menggunakan *mode=“overwrite”*, untuk memastikan bahwa tabel tujuan selalu ditulis ulang setiap kali program dijalankan. Hal ini bertujuan untuk memastikan bahwa setiap skenario eksperimen dijalankan dengan data yang bersih dan konsisten. Dengan kata lain, data yang sebelumnya telah dimuat tidak akan tercampur dengan data pada eksekusi terbaru. Hal ini sangat penting agar hasil pengujian tidak dipengaruhi oleh sisa data dari percobaan sebelumnya. Sementara itu, parameter *properties* berisi konfigurasi koneksi menuju PostgreSQL, seperti nama pengguna, kata sandi, dan *driver* JDBC yang digunakan. Konfigurasi ini telah didefinisikan sebelumnya pada bagian koneksi ke basis data.

Pada arsitektur hibrida, fase pemuatan ini menjadi penghubung antara proses pembersihan awal di PySpark dan proses transformasi lanjutan menggunakan dbt. Setelah tabel-tabel hasil pembersihan berhasil dimuat ke PostgreSQL, tabel tersebut akan digunakan sebagai sumber data oleh model dbt pada tahap transformasi analitis. Dengan demikian, proses penggabungan beberapa tabel, filter, agregasi, dan perhitungan profit akan dilakukan berdasarkan tabel-tabel yang sudah dimuat ke *data warehouse* ini. Fase pemuatan pada arsitektur hibrida berbeda dengan fase pemuatan pada arsitektur ETL maupun ELT. Pada ETL, data yang dimuat ke PostgreSQL sudah merupakan hasil transformasi akhir. Pada ELT, data yang dimuat masih berupa data mentah. Sementara itu, pada arsitektur hibrida, data yang dimuat berada di antara keduanya, yaitu data yang telah melalui pembersihan awal tetapi belum melalui transformasi analitik utama. Transformasi akhir tetap dilakukan menggunakan dbt berdasarkan logika TPC-H Query 9.

4.2.3.4 Fase Transformasi (*Transform*)

Data hasil pembersihan yang sudah dimuat ke dalam PostgreSQL kemudian dilakukan transformasi. Pada tahap ini, transformasi analitik utama dilakukan menggunakan dbt. Berbeda dengan tahap sebelumnya yang hanya membersihkan dan berfokus pada pemilihan kolom dan penyesuaian tipe data, fase transformasi ini menggunakan dbt untuk menjalankan operasi analitik yang lebih kompleks, seperti penggabungan antartabel, filter data, perhitungan nilai profit, agregasi, dan penyesuaian urutan baris hasil akhir. Pada arsitektur ini, dbt memanfaatkan tabel hasil pemuatan PySpark yang berada pada skema *raw*. Tabel-tabel tersebut mencakup *raw.hybrid_lineitem*, *raw.hybrid_orders*, *raw.hybrid_part*, *raw.hybrid_supplier*, *raw.hybrid_partsupp*, dan *raw.hybrid_nation*. Seluruh tabel tersebut sudah melewati fase pembersihan awal, sehingga hanya tersisa kolom-kolom yang relevan dengan TPC-H Query 9 yang sudah dimuat ke dalam PostgreSQL.

Model dbt pada arsitektur hibrida ini dibagi menjadi dua lapisan, yaitu *staging* dan *marts*. Lapisan *staging* digunakan sebagai representasi tabel yang telah melewati fase pembersihan dan sudah dimuat ke dalam PostgreSQL. Sementara itu, lapisan *marts* digunakan untuk

membuat tabel akhir yang memenuhi logika TPC-H *Query 9*. Pemisahan ini digunakan untuk menjaga kerapian dan modularitas, dan mempermudah pengelolaan dependensi antarmodel.

```
dbt_transform/
├── models/
│   ├── hybrid/
│   │   ├── staging/
│   │   │   ├── stg_hybrid_lineitem.sql
│   │   │   ├── stg_hybrid_nation.sql
│   │   │   ├── stg_hybrid_orders.sql
│   │   │   ├── stg_hybrid_part.sql
│   │   │   ├── stg_hybrid_partsupp.sql
│   │   │   └── stg_hybrid_supplier.sql
│   │   └── marts/
│   │       └── mart_hybrid_query9.sql
│   └── sources.yml
```

Kode 4.28 Struktur Direktori dbt pada Arsitektur Hibrida

Seperti yang sudah dijelaskan pada arsitektur ELT, file *sources.yml* digunakan untuk mendefinisikan tabel sumber yang telah dimuat ke PostgreSQL. Dengan mendefinisikan tabel sumber melalui file ini, model dbt dapat memanggil tabel pada skema *raw* menggunakan fungsi *source()*. Hal ini membuat hubungan antara tabel hasil pemuatan dan model transformasi menjadi lebih jelas dan eksplisit.

Tahap pertama dalam transformasi dbt adalah pembentukan model *staging*. Pada arsitektur hibrida ini, model *staging* relatif lebih sederhana dibandingkan yang ada pada arsitektur ELT, karena sebagian besar proses pembersihan sudah dilakukan di fase sebelumnya, menggunakan PySpark. Meski demikian, model *staging* tetap digunakan untuk menjaga konsistensi struktur data dan memastikan data masuk terlebih dahulu ke dalam sistem basis data tujuan sebelum dilakukannya transformasi analitik.

```
1.  select
2.      cast(l_orderkey as integer) as l_orderkey,
3.      cast(l_partkey as integer) as l_partkey,
4.      cast(l_suppkey as integer) as l_suppkey,
5.      cast(l_quantity as double precision) as l_quantity,
6.      cast(l_extendedprice as double precision) as l_extendedprice,
7.      cast(l_discount as double precision) as l_discount
8.  from {{ source('raw_data', 'hybrid_lineitem') }}
```

Kode 4.29 Model Staging *stg_hybrid_lineitem.sql* pada Arsitektur Hibrida

```
1.  select
2.      cast(o_orderkey as integer) as o_orderkey,
3.      cast(o_orderdate as date) as o_orderdate
4.  from {{ source('raw_data', 'hybrid_orders') }}
```


Kode 4.30 Model Staging *stg_hybrid_orders* pada Arsitektur Hibrida

```
1.  select
2.      cast(p_partkey as integer) as p_partkey,
3.      cast(p_name as text) as p_name
4.  from {{ source('raw_data', 'hybrid_part') }}
```

Kode 4.31 Model Staging *stg_hybrid_part* pada Arsitektur Hibrida

```
1.  select
2.      cast(ps_partkey as integer) as ps_partkey,
3.      cast(ps_suppkey as integer) as ps_suppkey,
4.      cast(ps_supplycost as double precision) as ps_supplycost
from {{ source('raw_data', 'hybrid_partsupp') }}
```

Kode 4.32 Model Staging *stg_hybrid_partsupp* pada Arsitektur Hibrida

```
1.  select
2.      cast(s_suppkey as integer) as s_suppkey,
3.      cast(s_nationkey as integer) as s_nationkey
4.  from {{ source('raw_data', 'hybrid_supplier') }}
```

Kode 4.33 Model Staging *stg_hybrid_supplier* pada Arsitektur Hibrida

```
1.  Select
2.      cast(n_nationkey as integer) as n_nationkey,
3.      cast(n_name as text) as n_name
4.  from {{ source('raw_data', 'hybrid_nation') }}
```

Kode 4.34 Model Staging *stg_hybrid_nation* pada Arsitektur Hibrida

Pada model staging tersebut, setiap tabel hanya mengambil kolom yang telah dibersihkan saat fase pembersihan. Sebagai contoh, model *stg_hybrid_lineitem* hanya memuat kolom yang diperlukan untuk proses penggabungan dan perhitungan profit, yaitu *l_orderkey*, *l_partkey*, *l_suppkey*, *l_quantity*, *l_extendedprice*, dan *l_discount*. Casting tipe data tetap dilakukan pada tahap *staging* ini, sebagai bentuk validasi struktur data sebelum model digunakan untuk melakukan transformasi analitik akhir.

Setelah model *staging* terbentuk, transformasi utama dilakukan pada model *marts*. Model *marts* pada arsitektur hibrida diberi nama *mart_hybrid_query9.sql*. Model ini berisi tabel yang sesuai logika TPC-H Query 9, yaitu menghitung profit berdasarkan negara dan tahun pesanan. Proses ini melibatkan penggabungan antara tabel *part*, *lineitem*, *supplier*, *partsupp*, *orders*, dan *nation*.

```
1.  Select
2.      nation,
3.      o_year,
4.      sum(amount) as sum_profit
```

```

5.      from
6.      (
7.          select
8.              n.n_name as nation,
9.              extract(year from o.o_orderdate) as o_year,
10.             l.l_extendedprice * (1 - l.l_discount) - ps.ps_supplycost
11.         * l.l_quantity as amount
12.         from
13.             {{ ref('stg_hybrid_part') }} p,
14.             {{ ref('stg_hybrid_supplier') }} s,
15.             {{ ref('stg_hybrid_lineitem') }} l,
16.             {{ ref('stg_hybrid_partsupp') }} ps,
17.             {{ ref('stg_hybrid_orders') }} o,
18.             {{ ref('stg_hybrid_nation') }} n
19.         where
20.             s.s_suppkey = l.l_suppkey
21.             and ps.ps_suppkey = l.l_suppkey
22.             and ps.ps_partkey = l.l_partkey
23.             and p.p_partkey = l.l_partkey
24.             and o.o_orderkey = l.l_orderkey
25.             and s.s_nationkey = n.n_nationkey
26.             and p.p_name like '%green%'
27.         ) as profit
28.     group by
29.         nation,
30.         o_year
31.     order by
32.         nation asc,
33.         o_year desc

```

Kode 4.35 Model Mart *mart_hybrid_query9.sql* pada Arsitektur Hibrida

Pada kode tersebut, fungsi *ref()* digunakan untuk memanggil model *staging* yang telah dibuat sebelumnya. Transformasi pada model *mart_hybrid_query9* ini terdiri dari beberapa proses utama. Pertama, model melakukan penggabungan antara tabel *part*, *supplier*, *lineitem*, *partsupp*, *orders*, dan *nation*. Penggabungan semua tabel tersebut bertujuan untuk mengambil informasi mengenai produk, pemasok, biaya pasokan, transaksi, tanggal pesanan, dan asal negara pemasok. Kedua, model melakukan filter pada kolom *p_name* yang berisi nama produk yang mengandung kata “green”. Ketiga, model *mart* ini akan menghitung nilai profit menggunakan hasil selisih antara pendapatan setelah diskon dan biaya pasokan total. Hasil dari nilai profit ini kemudian dilakukan *GROUP BY* berdasarkan negara dan tahun pesanan, lalu diurutkan berdasarkan kolom yang sama.

Untuk menjalankan proses transformasi dbt pada arsitektur hibrida, dapat digunakan perintah berikut.

```
dbt run --select hybrid
```

Perintah tersebut digunakan untuk menjalankan seluruh model yang berada pada direktori *models/hybrid*, mulai dari model *staging* hingga model *marts*. Dapat dilakukan juga eksekusi yang berfokus hanya pada model akhir beserta dependensinya, dengan perintah berikut.

```
dbt run --select mart_hybrid_query9+
```

Dalam konteks *pipeline* hibrida, fase transformasi ini menjadi tahap yang menghasilkan output akhir penelitian. Hasil dari model *mart_hybrid_query9* ini digunakan sebagai keluaran dari arsitektur hibrida yang nantinya akan dibandingkan dengan tabel hasil dari arsitektur ETL dan ELT. Dengan demikian, ini dapat menjadikan bukti bahwa meskipun arsitektur hibrida melakukan pembersihan awal menggunakan PySpark, transformasi analitik utama masih tetap dijalankan di dalam PostgreSQL melalui dbt.

4.2.3.5 Fungsi Utama Pipeline Hibrida

Fungsi utama pipeline arsitektur hibrida ini diawali dengan pengaturan Spark Job Group menggunakan nama *Hybrid_Pipeline*. Pengaturan ini digunakan agar keseluruhan proses pipeline hibrida dapat dipantau melalui Spark Web UI. Setelah itu, program melakukan iterasi terhadap setiap tabel yang terdapat pada variabel *TABLE_NAME*. Di dalam proses iterasi tersebut, setiap tabel terlebih dahulu dibaca menggunakan fungsi *extract_data()*. Data hasil ekstraksi tersebut kemudian diproses menggunakan fungsi *clean_data()* untuk memilih kolom yang relevan dan menyesuaikan tipe data. Setelah proses pembersihan selesai, data hasil pembersihan tersebut kemudian dimuat ke PostgreSQL menggunakan fungsi *load_data()*. Implementasi fungsi utama pada *pipeline* hibrida ini dapat dilihat sebagai berikut.

```
1.  if __name__ == "__main__":
2.      spark.sparkContext.setJobGroup("Hybrid_Pipeline", "TPC-H Query 9
   - Hybrid Pipeline")
3.      for table in TABLE_NAME:
4.          print(f"=== Starting ECL Process for table '{table}' ===")
5.          extracted_data = extract_data(table, FILE_FORMAT)
6.          df_cleaned = clean_data(table, extracted_data)
7.          load_data(table, df_cleaned)
8.          print(f"Table '{table}' loaded successfully.\n")
9.      spark.sparkContext.setJobDescription("[Hybrid] Pipeline
   Complete")
10.
11.     print("E, C, L Process done. You can access the
12. http://localhost:4040 for detailed monitoring.")
13.     print("Press CTRL+C to terminate the process.")
14.
15.     time.sleep(3600)
16.     spark.stop()
```

Dalam skenario pengujian ini, format yang digunakan dapat disesuaikan dengan format data yang sedang diuji. Dengan demikian, satu kali eksekusi merepresentasikan satu skenario pengujian berdasarkan format data tertentu. Fungsi *time.sleep(3600)* digunakan untuk mempertahankan Spark Web UI agar hasil eksekusi dapat ditinjau setelah proses selesai. Instruksi ini tidak termasuk dalam proses utama pipeline dan tidak dihitung sebagai bagian dari waktu pemrosesan. Setelah proses monitoring selesai, SparkSession dihentikan menggunakan *spark.stop()*.

4.3 Hasil Pengujian

Bagian ini menyajikan hasil pengujian dari implementasi arsitektur ETL, ELT, dan hibrida yang telah dijelaskan pada bagian sebelumnya. Pengujian ini dilakukan untuk membandingkan kinerja antara ketiga arsitektur berdasarkan beberapa metrik utama, yaitu waktu pemrosesan total, penggunaan CPU, penggunaan memori, penggunaan ruang penyimpanan, dan skalabilitas terhadap peningkatan volume data. Setiap arsitektur diuji menggunakan dataset TPC-H dengan format data yang telah ditentukan, yaitu CSV, JSON, dan Parquet. Selain itu, pengujian juga dilakukan pada beberapa variasi ukuran data berdasarkan skala yang digunakan. Dengan demikian, hasil pengujian tidak hanya menunjukkan perbandingan antara arsitektur yang digunakan, namun juga memperlihatkan pengaruh format data dan ukuran dataset terhadap performa *pipeline* integrasi data.

Sebelum hasil performa dianalisis, keluaran terakhir dari setiap arsitektur telah divalidasi untuk memastikan bahwa ETL, ELT, dan hibrida menghasilkan keluaran yang sama. Validasi tersebut dilakukan agar perbandingan performa dapat dianggap lebih adil, karena setiap arsitektur menjalani proses logika transformasi yang sama, yaitu TPC-H Query 9, meskipun menggunakan lokasi struktur dan urutan tahapan transformasi yang berbeda.

4.3.1 Performa Durasi Pemrosesan

4.3.2 Penggunaan Sumber Daya Komputasi

4.4 Analisis dan Pembahasan

4.4.1 Analisis Kecepatan Pemrosesan

4.4.2 Analisis Penggunaan Sumber Daya

DAFTAR PUSTAKA

- Al Khozi, I. K. (2025). *Perbandingan Performa Teknologi Data Lake dan Data Warehouse dalam Pengelolaan Data Stack Overflow Menggunakan Arsitektur Hybrid Hub-and-Spoke dan Kimball Method*. Institut Teknologi Sepuluh Nopember.
- Ali, S. M. F., Mey, J., & Thiele, M. (2019). Parallelizing user-defined functions in the ETL workflow using orchestration style sheets. *International Journal of Applied Mathematics and Computer Science*, 29(1), 69–79. <https://doi.org/10.2478/amcs-2019-0005>
- Apache Parquet: Documentation. (2024). <https://parquet.apache.org/docs/>
- Beneventano, D., Bergamaschi, S., Gagliardelli, L., & Simonini, G. (2019). Entity Resolution and Data Fusion: an integrated approach. *Proceedings of the 27th Italian Symposium on Advanced Database Systems, Castiglione Della Pescaia (Grosseto)*, 2400, 1–7.
- Bourhis, P., Reutter, J. L., & Vrgoč, D. (2020). JSON: Data model and query languages. *Information Systems*, 89. <https://doi.org/10.1016/j.is.2019.101478>
- Brackenbury, W., Liu, R., Mondal, M., Elmore, A. J., Ur, B., Chard, K., & Franklin, M. J. (2018, June 10). Draining the data swamp: A similarity-based approach. *Proceedings of the Workshop on Human-In-the-Loop Data Analytics, HILDA 2018*. <https://doi.org/10.1145/3209900.3209911>
- DataForge. (2023). *ETL vs ELT: Key Differences*. <https://www.dataforlabs.com/data-transformation-tools/etl-vs-elt>
- dbt Labs. (n.d.). *What is dbt?* Retrieved November 7, 2025, from <https://docs.getdbt.com/docs/introduction>
- dbt Labs. (2025, June 24). *Understanding ELT: Extract, Load, Transform*. <https://www.getdbt.com/blog/extract-load-transform>
- Dhaouadi, A., Bousselmi, K., Gammoudi, M. M., Monnet, S., & Hammoudi, S. (2022). Data Warehousing Process Modeling from Classical Approaches to New Trends: Main Features and Comparisons. *Data*, 7(8). <https://doi.org/10.3390/data7080113>
- Farhan, M. S., Youssef, A., & Abdelhamid, L. (2024). A Model for Enhancing Unstructured Big Data Warehouse Execution Time. *Big Data and Cognitive Computing*, 8(2). <https://doi.org/10.3390/bdcc8020017>
- Google Cloud. (n.d.). *What is ELT (extract, load, and transform)?* Retrieved October 28, 2025, from <https://cloud.google.com/discover/what-is-elt?>
- Harby, A. A., & Zulkernine, F. (2025). Data Lakehouse: A survey and experimental study. *Information Systems*, 127. <https://doi.org/10.1016/j.is.2024.102460>
- Haryono, E. M., Fahmi, Tri W, A. S., Gunawan, I., Nizar Hidayanto, A., & Rahardja, U. (2020). Comparison of the E-LT vs ETL Method in Data Warehouse Implementation: A Qualitative Study. *Proceedings - 2nd International Conference on Informatics, Multimedia, Cyber, and Information System, ICIMCIS 2020*, 115–120. <https://doi.org/10.1109/ICIMCIS51567.2020.9354284>

- IBM. (n.d.). *What is ETL (extract, transform, load)?* Retrieved October 28, 2025, from <https://www.ibm.com/think/topics/etl>
- IBM. (2025). *What is Apache Parquet?*
- Integrate.io. (2025, August 18). *ELT vs ETL Comparison Statistics – 40+ Key Data Points Every Data Leader Should Know in 2025*. <https://www.integrate.io/blog/elt-vs-etl-comparison-statistics/>.
- json.org. (n.d.). *Introducing JSON*. Retrieved November 11, 2025, from <https://www.json.org/json-en.html>
- Khine, P. P., & Wang, Z. S. (2018). Data lake: a new ideology in big data era. *ITM Web of Conferences*, 17, 03025. <https://doi.org/10.1051/itmconf/20181703025>
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit* (Robert Elliott, Ed.; 3rd ed.). John Wiley & Sons, Inc.
- Lau, S., Kross, S., Wu, E., & Guo, P. J. (2023). Teaching Data Science by Visualizing Data Table Transformations: Pandas Tutor for Python, Tidy Data Tutor for R, and SQL Tutor. *Proceedings of the 2nd ACM SIGMOD International Workshop on Data Systems Education: Bridging Education Practice with Education Research, DataEd 2023*, 50–55. <https://doi.org/10.1145/3596673.3596972>
- Mehmood, E., & Anees, T. (2020). Challenges and Solutions for Processing Real-Time Big Data Stream: A Systematic Literature Review. In *IEEE Access* (Vol. 8, pp. 119123–119143). Institute of Electrical and Electronics Engineers Inc. <https://doi.org/10.1109/ACCESS.2020.3005268>
- Microsoft. (2025). *Get data from comma separated value (CSV) files*. <https://learn.microsoft.com/en-us/power-bi/connect-data/service-comma-separated-value-files>
- Momjian, Bruce. (2001). *PostgreSQL: Introduction and Concepts*. Addison-Wesley.
- Mueller, J. P. (2014). *Beginning Programming with Python for Dummies* (3rd ed., Vol. 3). John Wiley & Sons Inc.
- Naeem, M., Jamal, T., Diaz-Martinez, J., Butt, S. A., Montesano, N., Tariq, M. I., De-la-Hoz-Franco, E., & De-La-Hoz-Valdiris, E. (2022). *Trends and Future Perspective Challenges in Big Data* (pp. 309–325). https://doi.org/10.1007/978-981-16-5036-9_30
- Nambiar, A., & Mundra, D. (2022). An Overview of Data Warehouse and Data Lake in Modern Enterprise Data Management. In *Big Data and Cognitive Computing* (Vol. 6, Issue 4). MDPI. <https://doi.org/10.3390/bdcc6040132>
- Neang, A. B., Sutherland, W., Beach, M. W., & Lee, C. P. (2021). Data Integration as Coordination: The Articulation of Data Work in an Ocean Science Collaboration. *Proceedings of the ACM on Human-Computer Interaction*, 4(CSCW3). <https://doi.org/10.1145/3432955>
- Ong, T. C., Kahn, M. G., Kwan, B. M., Yamashita, T., Brandt, E., Hosokawa, P., Uhrich, C., & Schilling, L. M. (2017). Dynamic-ETL: A hybrid approach for health data extraction,

- transformation and loading. *BMC Medical Informatics and Decision Making*, 17(1). <https://doi.org/10.1186/s12911-017-0532-3>
- Rambabu, V. P., & Althati, C. (2023). ETL vs. ELT: Optimizing Data Integration for Retail and Insurance Analytics. *Journal of Computational Intelligence and Robotics*, 3(1).
- Reinsel, D., Gantz, J., & Rydning, J. (2018). *The Digitization of the World From Edge to Core*.
- Rongala, S. (2025). Optimizing ETL Processes for High-Volume Data Warehousing in Financial Applications. *Journal of Information Systems Engineering and Management*, 10(8s), 700–708. <https://doi.org/10.52783/jisem.v10i8s.1130>
- Shang, X., Subenderan, P., Islam, M., Xu, J., Zhang, J., Gupta, N., & Panda, A. (2022). One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale. *Proceedings - 2022 IEEE International Conference on Big Data, Big Data 2022*, 5802–5811. <https://doi.org/10.1109/BigData55660.2022.10020987>
- Singh, S. S., Singh, S., Singh, K., Srivastava, V., & Shakya, H. K. (2025). Big Data Meets Social Networks: A Survey of Analytical Strategies and Research Challenges. *IEEE Access*, 13, 98668–98698. <https://doi.org/10.1109/ACCESS.2025.3576143>
- Singhal, B., & Aggarwal, A. (2022). ETL, ELT and Reverse ETL: A business case Study. *2nd IEEE International Conference on Advanced Technologies in Intelligent Control, Environment, Computing and Communication Engineering, ICATIECE 2022*. <https://doi.org/10.1109/ICATIECE56365.2022.10046997>
- Sivabalan, S., & Minu, R. I. (2021). Heterogeneous Data Integration with ELT and Analytical MPP Database for Data Analysis Application. *3rd IEEE International Virtual Conference on Innovations in Power and Advanced Computing Technologies, i-PACT 2021*. <https://doi.org/10.1109/i-PACT52855.2021.9696841>
- Sivarajah, U., Kamal, M. M., Irani, Z., & Weerakkody, V. (2017). Critical analysis of Big Data challenges and analytical methods. *Journal of Business Research*, 70, 263–286. <https://doi.org/10.1016/j.jbusres.2016.08.001>
- TPC Benchmark H Standard Specification Book*. (1993).
- TPC-H Official Homepage*. (n.d.). Retrieved November 17, 2025, from <http://www.tpc.org/tpch/>
- Vallejos, C., Caniupan, M., & Gutierrez, G. (2018). Compact Data Structures to Represent and Query Data Warehouses into Main Memory. *IEEE Latin America Transactions*, 16(9), 2328–2335. <https://doi.org/10.1109/TLA.2018.8789552>
- Younas, M. (2019). Research challenges of big data. In *Service Oriented Computing and Applications* (Vol. 13, Issue 2, pp. 105–107). Springer London. <https://doi.org/10.1007/s11761-019-00265-x>
- Zagni, R. (2023). *Data engineering with dbt: a practical guide to building a cloud-based, pragmatic, and dependable data platform with SQL*. Packt Publishing Ltd.

LAMPIRAN

Lampiran 1 Formulir Checklist Etika Penulisan dengan Penggunaan AI

PERNYATAAN KODE ETIK PENGUNAAN GENERATIVE AI

Code of Conduct Statement Generative AI or AI-Assisted Usage

Saya yang bertanda tangan di bawah ini:

I, the undersigned:

Nama Lengkap / *Full name* : Luthfan Aryananda Purwito
NRP / *Student ID* : 5026221166
Program Studi / *Study Program* : Sistem Informasi
Judul / *Title* : Analisis Kinerja Arsitektur Integrasi Data ETL, ELT, dan Hibrida pada *Data Warehouse* Berbasis *PostgreSQL*
Pembimbing / *Supervisors* : Faizal Johan Atletiko, S.Kom., MT.

Menyatakan dalam pengerjaan Tugas Akhir dengan judul tersebut di atas:

State that during the finalization of my thesis with the title above:

No.	Pernyataan / <i>Statement</i>	Checklist (☒)
1	Hanya menggunakan Generative AI sebagai alat bantu untuk memperbaiki tata bahasa pada Tugas Akhir saya. Ini tidak digunakan untuk membuat keseluruhan isi pekerjaan saya. <i>Generative AI is used only as a tool to improve the readability or language of the text in my Thesis. It is not used to generate a complete text of my work.</i>	☒
2	Telah memeriksa dan/atau memperbaiki seluruh bagian dari pekerjaan saya yang dibantu oleh Generative AI agar sesuai dengan baku mutu penulisan karya ilmiah <i>I have checked and/or refined all parts of my work that is assisted by Generative AI in such a way that it complies with the standard of academic writing</i>	☒

No.	Pernyataan / Statement	Checklist (☒)
3	Tidak menggunakan Generative AI untuk pembuatan data primer, grafik dan/atau tabel pada pekerjaan saya. <i>Generative AI was not used to generate primary data, figure and/or table in my works.</i>	☒
4	Penulis telah memberikan atribusi/pengakuan terhadap alat AI yang digunakan, secara rinci pada suatu bagian pada lampiran. <i>The author has acknowledged the use of Generative AI in any part of the work in the specific appendix page.</i>	☒
5	Buku Tugas Akhir telah mematuhi Baku Mutu Akademik ITS terkait keaslian karya dan etika penulisan. <i>The thesis has complied with the Academic Standard of ITS related to the originality of scientific work and scientific writing code of conduct</i>	☒
6	Penulis memastikan tidak ada plagiarisme, termasuk yang berasal dari penggunaan AI. <i>The authors have ensured that there is no plagiarism issue in the work, including any parts generated by AI.</i>	☒

Tempat/ Place	: Surabaya
Tanggal/ Date	: 18 Desember 2025
Nama Lengkap/ Full Name	: Luthfan Aryananda Purwito
Tanda tangan/ Signature	: 

BIODATA PENULIS

Photo closed-up



Penulis dilahirkan di Madiun, 29 Januari 1985, merupakan anak pertama dari 4 bersaudara. Penulis telah menempuh pendidikan formal yaitu di TK ABA 18 Madiun, SDN Beteng 1 Madiun, SMPN 2 Madiun dan SMAN 2 Madiun. Setelah lulus dari SMAN tahun 2020, Penulis mengikuti SBMPTN dan diterima di Departemen Teknik Mesin FTIRS - ITS pada tahun 2020 dan terdaftar dengan NRP 02112040000130.

Di Departemen Teknik Mesin Penulis sempat aktif di beberapa kegiatan Seminar yang diselenggarakan oleh Departemen, Himpunan Mahasiswa Teknik Mesin (HMM) dan aktif sebagai Asisten Praktikum Mesin Konversi Energi maupun Grader mata kuliah Termodinamika.